องค์ประกอบเชิงสถาปัตยกรรมของเว็ปไซต์

เว็บไซต์และระบบนี้ถูกพัฒนาขึ้นเพื่อใช้เป็นระบบสั่งอาหารที่ร้านผ่านแท็บเล็ตประจำโต๊ะและบริหารจัดการร้านอาหาร โดยออกแบบมาเพื่อรองรับการใช้งานของผู้ใช้งาน 4 กลุ่มหลัก ได้แก่ ลูกค้า ,พนักงานครัว , พนักงานบริการ , และพนักงานแคชเชียร์ ระบบทำงานร่วมกับเว็บเซิร์ฟเวอร์และฐานข้อมูล เพื่อให้สามารถส่งต่อข้อมูลคำสั่งซื้อ จัดเตรียมอาหาร เสิร์ฟอาหาร และคิดเงินได้อย่างแม่นยำ รวดเร็ว โดยองค์ประกอบหลักของระบบสามารถแบ่งออกตามโครงสร้างสถาปัตยกรรมซอฟต์แวร์ได้ดังต่อไปนี้

1. Client Layer (ส่วนผู้ใช้งาน)
   ส่วนที่ผู้ใช้งานแต่ละบทบาทใช้ติดต่อและโต้ตอบกับระบบผ่านเว็บเบราว์เซอร์หรืออุปกรณ์แท็บเล็ตและคอมพิวเตอร์ประจำจุดต่างๆ
   - ผู้ใช้งานหลัก
     - ลูกค้า
     - พนักงานครัว
     - พนักงานบริการ
     - พนักงานแคชเชียร์
   - หน้าที่
     - แสดงหน้าเว็บและเมนูอาหารแยกตามหมวดหมู่ (ซูชิ, ของทอด, เส้น/ซุป, ของหวาน , เครื่องดื่ม)
     - รับคำสั่งซื้อ การปรับแต่งตัวเลือกวัตถุดิบ และการจัดการตะกร้าสินค้าจากลูกค้า
     - แสดงรายการออเดอร์เข้าครัวแบบเรียลไทม์ และรับการกดเปลี่ยนสถานะ (กำลังทำ, พร้อมเสิร์ฟ, เสิร์ฟแล้ว)
     - แสดงข้อมูลบิล คำนวณยอดเงินรวม ยอดเงินทอน และสร้าง QR Code สำหรับชำระเงิน
   - เทคโนโลยีที่ใช้
     - HTML
     - CSS
     - JavaScript

2. Business Logic Layer (Web Server / Backend)

   เป็นส่วนประมวลผลหลักของระบบ ทำหน้าที่ควบคุมและจัดการตรรกะการทำงานทั้งหมด

   - หน้าที่
     - ประมวลผลคำสั่งซื้อจากแท็บเล็ตประจำโต๊ะ และส่งต่อไปยังระบบแสดงผลในครัว
     - จัดการตรรกะของระบบ เช่น การคำนวณราคารวม ส่วนลด ภาษี และเงินทอน
     - ตรวจสอบสิทธิ์และการเข้าถึงข้อมูลตามบทบาท
     - จัดการและอัปเดตสถานะโต๊ะอาหาร
     - จัดการและอัปเดตสถานะของแต่ละรายการอาหาร (กำลังเตรียมอาหาร, พร้อมเสิร์ฟ , เสิร์ฟแล้ว)
     - ติดต่อและทำธุรกรรมข้อมูลกับฐานข้อมูล
   - เทคโนโลยีที่ใช้
     - Node.js
     - Express.js
     - EJS (Embedded JavaScript Templates)
     - Express-session

3. Data Layer

   ส่วนที่ใช้จัดเก็บข้อมูลทั้งหมดของระบบ

   - ข้อมูลที่จัดเก็บ
     - ข้อมูลรายการอาหารและราคา
     - ข้อมูลตัวเลือกเสริมและการปรับแต่งวัตถุดิบ
     - ข้อมูลสถานะโต๊ะทั้ง 16 โต๊ะ
     - ข้อมูลคำสั่งซื้อหลัก
     - ข้อมูลรายการย่อยในคำสั่งซื้อพร้อม Add-on
     - ข้อมูลประวัติการชำระเงินและเงินทอน
   - เทคโนโลยีฐานข้อมูล
     - SQLite

กระบวนการทำงานของระบบ (System Workflows)

1. **การเปิดโต๊ะและเริ่มการสั่งอาหาร**

   พนักงานแคชเชียร์เลือกเปิดโต๊ะ (โต๊ะ 01 - 16) ระบบจะบันทึกหมายเลขโต๊ะ และเปลี่ยนสถานะโต๊ะเป็น “ไม่ว่าง”

2. **การเลือกดูเมนูอาหาร**

   ระบบดึงรายการอาหารทั้งหมด 50 รายการจากฐานข้อมูล menu แยกตาม 5 หมวดหมู่หลัก ได้แก่ ซูชิ, ของทอด, เส้น/ซุป, ของทานเล่น และเครื่องดื่ม พร้อมแสดงรูปภาพ ชื่อเมนู และราคา

3. **การปรับแต่งรายการ**

   เมื่อลูกค้ากดปุ่ม '+' บนเมนูอาหาร ระบบจะนำไปยังหน้าเลือก Add-on ดึงข้อมูลตัวเลือกตามหมวดหมู่ เช่น ระดับความเผ็ด, Topping, วาซาบิ/โชยุ, ซอส, ผงคลุก หรือระดับความหวาน พร้อมระบุราคาเพิ่มเติม

4. **การจัดการตะกร้าสินค้า**

   เมื่อกดเพิ่มลงตะกร้า ข้อมูลอาหารและ Add-on จะถูกเก็บลงใน Session cart ลูกค้าสามารถตรวจสอบรายการในหน้า /cart เพิ่มและลดจำนวน ลบรายการเดี่ยว หรือล้างตะกร้าทั้งหมดได้ โดยระบบคำนวณราคารวมให้อัตโนมัติ

5. **การยืนยันคำสั่งซื้อส่งเข้าครัว**

   เมื่อกดยืนยันการสั่งอาหาร ระบบจะบันทึกข้อมูลลงตาราง orders (สถานะเริ่มต้น 'กำลังทำ') และ order_items พร้อมอัปเดตสถานะโต๊ะใน tables_status เป็น 'ไม่ว่าง' จากนั้นล้างตะกร้าและนำไปยังหน้าประวัติคำสั่งซื้อ

6. **การจัดเตรียมอาหารในครัว**

   หน้าจอพนักงานครัวจะรีเฟรชข้อมูลอัตโนมัติทุก 8 วินาที แสดงรายการอาหารที่กำลังทำ พนักงานครัวดูรายละเอียดและ Add-on เมื่อปรุงเสร็จสิ้นจะกดปุ่ม 'เสร็จสิ้น' ระบบจะอัปเดตสถานะเป็น 'พร้อมเสิร์ฟ'

7. **การนำเสิร์ฟอาหาร**

   หน้าจอพนักงานบริการ (/service) จะแสดงเฉพาะรายการอาหารที่อยู่ในสถานะ 'พร้อมเสิร์ฟ' เมื่อพนักงานนำอาหารไปเสิร์ฟที่โต๊ะลูกค้าแล้ว จะกดปุ่ม 'เสิร์ฟแล้ว' เพื่อนำรายการออกจากหน้าจอรอเสิร์ฟ

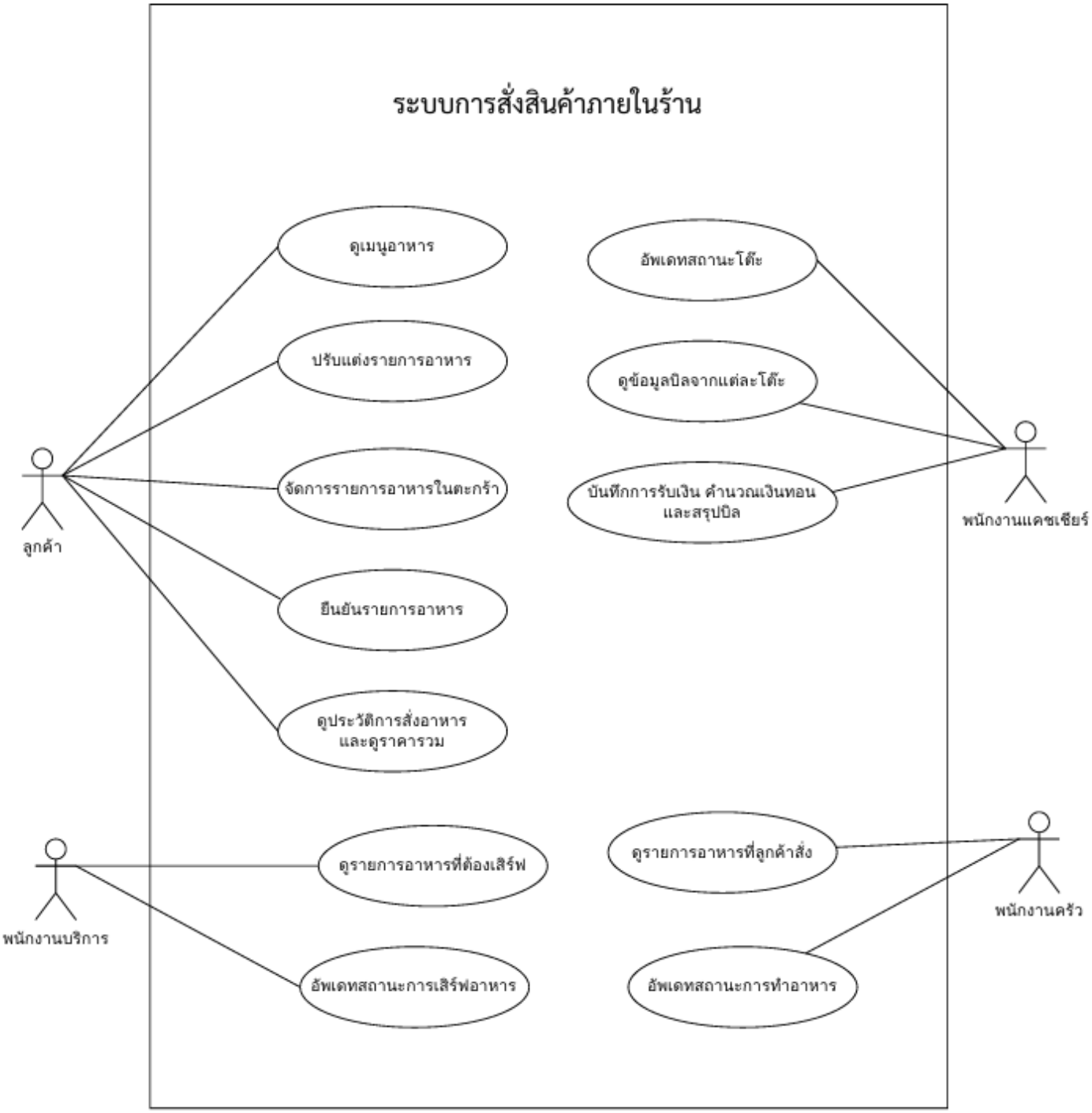
8. **การตรวจสอบประวัติคำสั่งซื้อ**

   ลูกค้าสามารถเข้าดูหน้า /orders เพื่อติดตามสถานะของอาหารแต่ละจาน (กำลังทำ, พร้อมเสิร์ฟ, เสิร์ฟแล้ว) และตรวจสอบยอดรวมค่าอาหารทั้งหมดของโต๊ะได้ตลอดเวลา

9. **การเรียกดูบิล ชำระเงิน และปิดโต๊ะ**

   พนักงานแคชเชียร์เข้าหน้า /cashier เลือกหมายเลขโต๊ะเพื่อตรวจสอบรายการอาหารและยอดรวมทั้งหมด เมื่อลูกค้าต้องการชำระเงิน แคชเชียร์เข้าสู่หน้ารับเงิน (/payment/:table) เลือกระบุจำนวนเงินสดหรือ QR CODE ระบบตรวจสอบยอดเงิน คำนวณเงินทอน บันทึกประวัติลงตาราง payments ปรับสถานะออเดอร์เป็น 'ชำระแล้ว' และรีเซ็ตสถานะโต๊ะกลับเป็น 'ว่าง'

Use case diagram



เทคนิคและเครื่องมือที่ใช้ในการพัฒนา (Technology Stack)

Frontend

- **HTML** ใช้สำหรับสร้างโครงสร้างหน้าเว็บ เช่น เมนูอาหาร ตารางรายการอาหาร ตะกร้าสินค้า และหน้าจอควบคุมของพนักงาน
- **CSS** ใช้สำหรับกำหนดรูปแบบความสวยงาม และการจัดวางองค์ประกอบทั้งหมดบนหน้าเว็บไซต์ เช่น การเลือกใช้โทนสี รูปแบบและขนาดตัวอักษร การเว้นระยะห่างของเนื้อหา รวมถึงการวางผังโครงสร้าง เพื่อให้เว็บไซต์มีความสวยงาม สบายตา
- **JavaScript** ใช้สำหรับเพิ่มความสามารถในการโต้ตอบ ตรวจสอบเงื่อนไข และส่งคำขอไปยังเซิร์ฟเวอร์
- **EJS (Embedded JavaScript Templates)** ใช้เป็น **Template Engine** สำหรับสร้างหน้าเว็บแบบ **Dynamic** ดึงข้อมูลจากเซิร์ฟเวอร์มาเรนเดอร์ในหน้า **HTML**

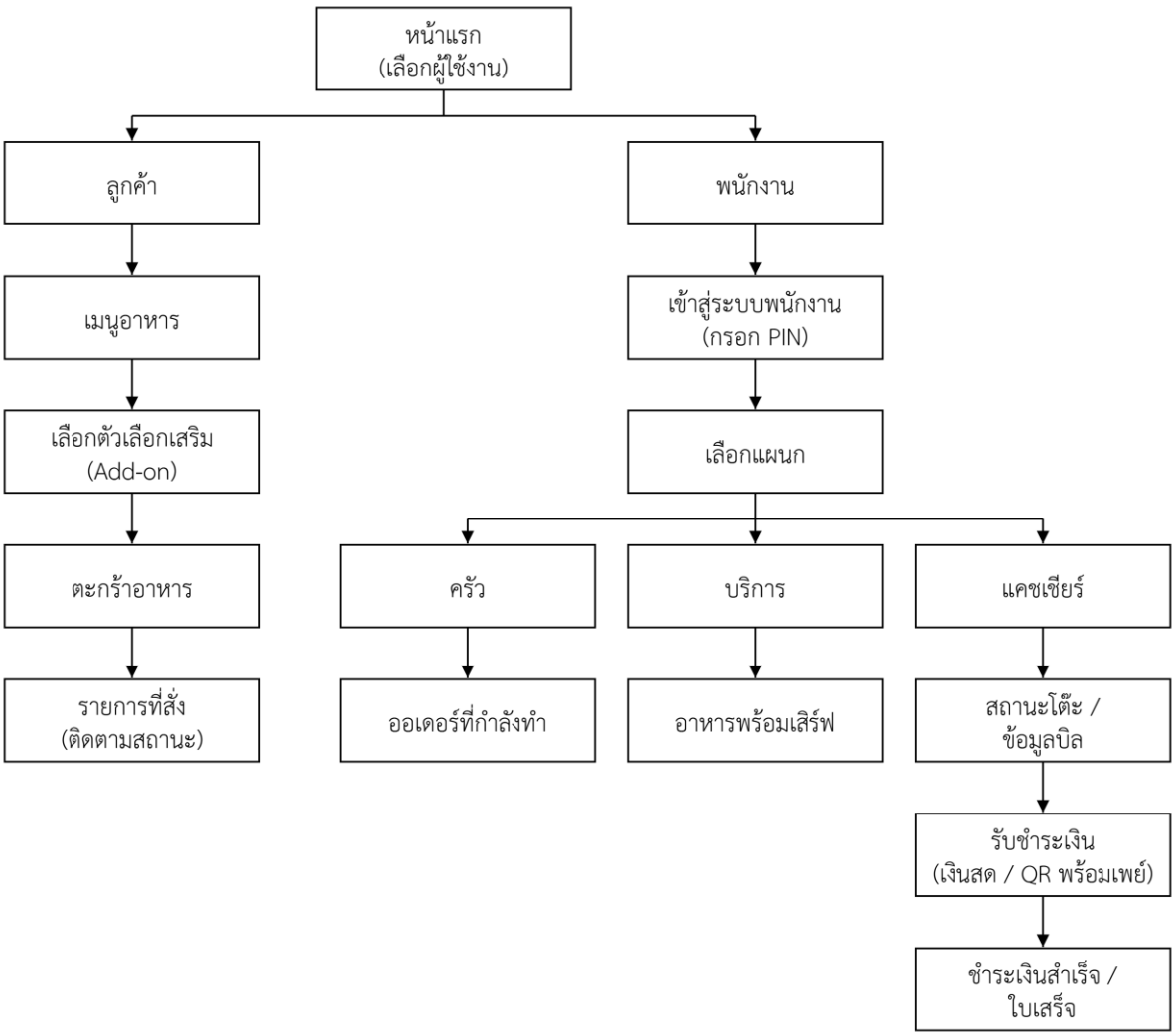
Backend

- **Node.js** เป็น **Runtime Environment** สำหรับรัน **JavaScript** บนฝั่งเซิร์ฟเวอร์ จัดการการทำงานของระบบและฐานข้อมูล
- **Express.js** เป็น **Framework** สำหรับสร้าง **Web Server** บน **Node.js** ช่วยจัดการ **Routing, Middleware** และ **HTTP Request/Response**
- **SQLite3** เป็นไลบรารีสำหรับเชื่อมต่อและจัดการคำสั่ง **SQL** กับฐานข้อมูล **SQLite**
- **Express-session** ใช้สำหรับจัดการ **Session** ของผู้ใช้ในระบบ เช่น การเก็บข้อมูลหมายเลขโต๊ะ และรายการสินค้าในตะกร้า

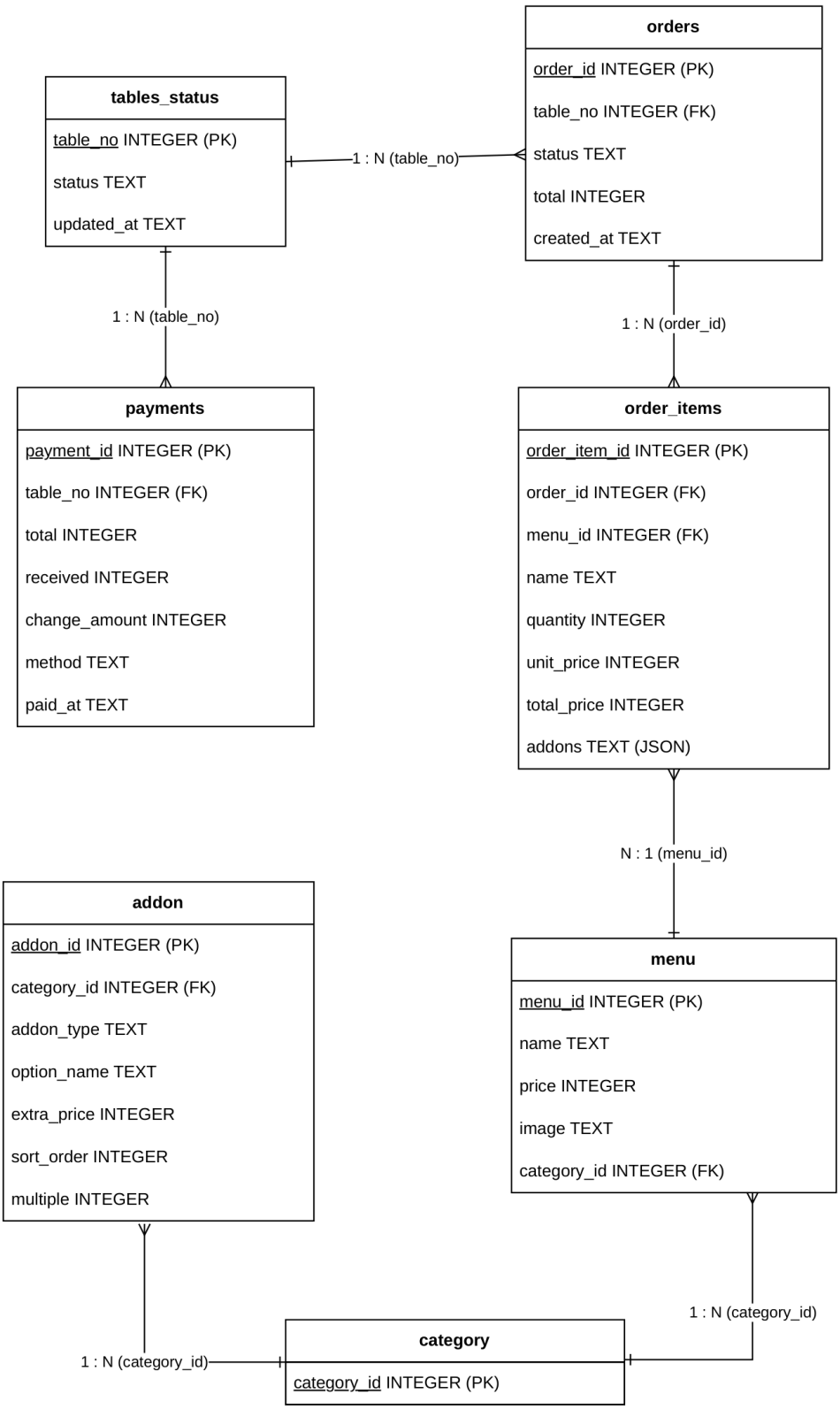
Database

- **SQLite** ระบบฐานข้อมูลแบบไฟล์ ใช้จัดเก็บข้อมูลของระบบในไฟล์ **food.db** สะดวก รวดเร็ว และไม่ต้องติดตั้งเซิร์ฟเวอร์ฐานข้อมูลแยก

แผนผังเว็บไซต์



การออกแบบ Entity Relationship Diagram



ตัวอย่างการเขียนโค้ดและใช้งานระบบสั่งอาหารผ่านแท็บเล็ต

หัวข้อนี้แบ่งออกเป็น 2 ส่วน ส่วนแรกอธิบายการทำงานของโค้ดโปรแกรมทั้งหมดของระบบ ได้แก่ โค้ดฝั่งเซิร์ฟเวอร์ (index.js) เทมเพลตหน้าเว็บ (EJS) สคริปต์ฝั่งหน้าเว็บ และไฟล์ตกแต่ง (CSS) ส่วนที่สองเป็นคู่มือการใช้งานหน้าเว็บของผู้ใช้แต่ละบทบาท ได้แก่ ลูกค้า พนักงานครัว พนักงานบริการ และพนักงานแคชเชียร์ พร้อมภาพหน้าจอประกอบทุกขั้นตอน

ส่วนที่ 1 การอธิบายโค้ดของระบบ

รูปโค้ดในส่วนนี้มีหมายเลขบรรทัดตรงกับไฟล์จริงในโปรเจกต์ ใช้อ้างอิงเมื่อเปิดดูโค้ดในโปรแกรม Visual Studio Code ได้ แต่ละหัวข้อประกอบด้วยคำอธิบายภาพรวม รูปโค้ด และคำอธิบายโค้ดทีละข้อ

1.1 โครงสร้างไฟล์ของโปรเจกต์

ระบบถูกพัฒนาด้วย Node.js และ Express.js โดยรวมโค้ดฝั่งเซิร์ฟเวอร์ทั้งหมดไว้ในไฟล์ index.js เพียงไฟล์เดียว ทั้งการตั้งค่า การสร้างฐานข้อมูล ฟังก์ชันช่วย และเส้นทาง (Route) ของทุกบทบาท ส่วนหน้าเว็บแยกเป็นเทมเพลต EJS ในโฟลเดอร์ views และไฟล์ static (CSS, JavaScript, รูปภาพ) อยู่ในโฟลเดอร์ public

ตัวอย่างโค้ดที่ 1 โครงสร้างไฟล์และโฟลเดอร์ของระบบ

โครงสร้างโฟลเดอร์

```
Ject_sakura_v5/
├── index.js              ← เซิร์ฟเวอร์ทั้งหมด (Express + SQLite)
├── food.db               ← ไฟล์ฐานข้อมูล SQLite
├── package.json          ← รายชื่อไลบรารีและคำสั่ง npm start / npm run dev
├── public/               ← ไฟล์ static เรียกผ่าน URL /public/...
│   ├── style.css         ← สไตล์ของทุกหน้า
│   ├── customer.js       ← ปุ่มลับกดค้างที่โลโก้ (สำหรับพนักงาน)
│   ├── qrcode.min.js     ← ไลบรารีวาด QR Code (ใช้ได้แม้ไม่มีอินเทอร์เน็ต)
│   └── Pic/              ← รูปอาหาร sushi_1.jpg, fried_1.jpg, ...
└── views/                ← เทมเพลต EJS ของแต่ละหน้า
    ├── landing.ejs       ← หน้าแรก เลือก ลูกค้า / พนักงาน
    ├── home.ejs          ← หน้าเมนูอาหาร
    ├── addon.ejs         ← หน้าเลือก Add-on และจำนวน
    ├── cart.ejs          ← หน้าตะกร้า
    ├── orders.ejs        ← หน้ารายการที่สั่ง (ติดตามสถานะ)
    ├── _floatingcart.ejs ← ปุ่มตะกร้าลอย (ใช้ซ้ำหลายหน้า)
    ├── staff-login.ejs   ← หน้ากรอก PIN พนักงาน
    ├── staff-select.ejs  ← หน้าเลือกแผนก
    ├── kitchen.ejs       ← หน้าจอครัว
    ├── service.ejs       ← หน้าจอพนักงานบริการ
    ├── cashier.ejs       ← หน้าจอแคชเชียร์ (สถานะโต๊ะ + บิล)
    ├── payment.ejs       ← หน้ารับชำระเงิน (เงินสด / QR พร้อมเพย์)
    └── payment-success.ejs ← หน้าชำระเงินสำเร็จ / ใบเสร็จ
```

คำอธิบายโค้ด

1) index.js เป็นจุดเริ่มต้นของระบบ เมื่อรันคำสั่ง npm start จะเรียก node index.js เพื่อเปิดเว็บเซิร์ฟเวอร์ที่พอร์ต 3000
2) food.db เป็นไฟล์ฐานข้อมูล SQLite ที่ระบบสร้างและเติมข้อมูลตั้งต้นให้อัตโนมัติเมื่อเปิดเซิร์ฟเวอร์
3) โฟลเดอร์ public ถูกเปิดให้เบราว์เซอร์เข้าถึงผ่าน URL /public/... เช่น /public/style.css และ /public/Pic/sushi_1.jpg
4) โฟลเดอร์ views เก็บเทมเพลต EJS ซึ่งเซิร์ฟเวอร์จะนำข้อมูลจากฐานข้อมูลมาเติมแล้วส่งออกเป็นหน้า HTML ให้ผู้ใช้แต่ละบทบาท

1.2 การตั้งค่าเริ่มต้นของเซิร์ฟเวอร์

ส่วนบนสุดของ index.js ทำหน้าที่นำเข้าไลบรารี สร้างแอป Express เชื่อมต่อฐานข้อมูล และกำหนดค่าคงที่ของร้าน เช่น ชื่อร้าน เบอร์พร้อมเพย์ และ PIN ของพนักงาน

ตัวอย่างโค้ดที่ 2 index.js – นำเข้าไลบรารีและกำหนดค่าคงที่ของร้าน

index.js

```
const express = require("express");
const session = require("express-session");
const sqlite3 = require("sqlite3").verbose();
const path = require("path");

const app = express();
const PORT = process.env.PORT || 3000;
const DB_FILE = path.join(__dirname, "food.db");
const db = new sqlite3.Database(DB_FILE);

// ---------- ข้อมูลร้าน / การชำระเงิน ----------
// แก้ promptpay เป็นเบอร์พร้อมเพย์ หรือเลขบัตร/เลขผู้เสียภาษีของร้านจริง
// (เบอร์มือถือ 10 หลัก เช่น "0812345678" หรือเลข 13 หลัก)
const RESTAURANT = {
  name: "ซากุระ",
  nameEn: "SAKURA",
  tagline: "ร้านอาหารญี่ปุ่น",
  promptpay: "0812345678"
};

// PIN สำหรับเข้าหน้าพนักงาน (ครัว/บริการ/แคชเชียร์) กันลูกค้าเผลอเข้า
// ตั้งเป็น "" (ค่าว่าง) เพื่อปิดการใช้ PIN
const STAFF_PIN = "1234";
```

คำอธิบายโค้ด

1) require() นำเข้าไลบรารี express (เว็บเซิร์ฟเวอร์), express-session (จัดการ Session), sqlite3 (ฐานข้อมูล) และ path (จัดการเส้นทางไฟล์)

2) const app = express() สร้างแอปพลิเคชันเว็บ ส่วน PORT ใช้ค่าจากตัวแปรสภาพแวดล้อม PORT ถ้าไม่มีจะใช้ 3000

3) new sqlite3.Database(DB_FILE) เปิดไฟล์ food.db ที่อยู่โฟลเดอร์เดียวกับ index.js หากยังไม่มีไฟล์ SQLite จะสร้างให้ใหม่

4) ออบเจกต์ RESTAURANT เก็บชื่อร้าน (ซากุระ / SAKURA) คำโปรย และหมายเลขพร้อมเพย์ที่ใช้สร้าง QR Code รับเงิน

5) STAFF_PIN = "1234" คือรหัสสำหรับเข้าหน้าพนักงาน ป้องกันลูกค้าเผลอกดเข้าหน้าครัวหรือแคชเชียร์ หากตั้งเป็นค่าว่าง "" จะปิดการใช้ PIN

1.3 การสร้างข้อมูล QR Code พร้อมเพย์

ระบบสร้างข้อความ (Payload) ตามมาตรฐาน EMVCo ของ Thai QR Payment ที่ฝั่งเซิร์ฟเวอร์ โดยฝังยอดเงินของบิลลงไปด้วย จากนั้นหน้าเว็บจะนำข้อความนี้ไปวาดเป็น QR Code ให้ลูกค้าสแกนจ่ายด้วยแอปธนาคาร

ตัวอย่างโค้ดที่ 3 index.js – ฟังก์ชัน emv(), crc16() และ promptpayPayload()

index.js

```
// สร้าง payload มาตรฐาน PromptPay (EMVCo) ตามยอดเงิน -> ให้ฝั่งหน้าเว็บเรนเดอร์เป็น QR
function emv(id, value) { return id + String(value.length).padStart(2, "0") + value; }
function crc16(str) {
  let crc = 0xffff;
  for (let i = 0; i < str.length; i++) {
    crc ^= str.charCodeAt(i) << 8;
    for (let j = 0; j < 8; j++) {
      crc = (crc & 0x8000) ? ((crc << 1) ^ 0x1021) : (crc << 1);
      crc &= 0xffff;
    }
  }
  return crc.toString(16).toUpperCase().padStart(4, "0");
}
function promptpayPayload(target, amount) {
  const acc = String(target || "").replace(/[^0-9]/g, "");
  if (!acc) return "";
  const merchantId = acc.length >= 13
    ? emv("02", acc.slice(0, 13))
    : emv("01", "0066" + acc.replace(/^0/, ""));
  const merchant = emv("29", emv("00", "A000000677010111") + merchantId);
  const amt = Number(amount) || 0;
  let payload =
    emv("00", "01") +
    emv("01", amt > 0 ? "12" : "11") +
    merchant +
    emv("53", "764") +
    (amt > 0 ? emv("54", amt.toFixed(2)) : "") +
    emv("58", "TH");
  payload += "6304" + crc16(payload + "6304");
  return payload;
}
```

คำอธิบายโค้ด

1) emv(id, value) ประกอบข้อมูลแต่ละช่องในรูปแบบ ID 2 หลัก + ความยาว 2 หลัก + ค่า เช่น emv("58", "TH") ได้ "5802TH"
2) crc16() คำนวณค่าตรวจสอบความถูกต้อง CRC-16/CCITT (polynomial 0x1021 ค่าเริ่มต้น 0xFFFF) แล้วคืนค่าเป็นเลขฐาน 16 จำนวน 4 หลัก แอปธนาคารใช้ค่านี้ตรวจว่า QR ไม่ผิดเพี้ยน
3) promptpayPayload() ตัดอักขระที่ไม่ใช่ตัวเลขออกจากหมายเลขบัญชี ถ้ายาว 13 หลักถือเป็นเลขบัตรประชาชน/เลขผู้เสียภาษี (ช่อง 02) ถ้าเป็นเบอร์มือถือจะแปลง 0 นำหน้าเป็นรหัสประเทศ 0066 (ช่อง 01)
4) ช่อง 29 คือข้อมูลผู้รับเงินพร้อมเพย์ (AID A000000677010111) ช่อง 53 = สกุลเงินบาท (764) ช่อง 54 = ยอดเงินทศนิยม 2 ตำแหน่ง และช่อง 58 = ประเทศไทย
5) ถ้ามียอดเงิน ช่อง 01 จะเป็น "12" (QR แบบใช้ครั้งเดียว) และสุดท้ายต่อท้ายด้วย "6304" + ค่า CRC ของข้อความทั้งหมด

1.4 การแปลงเวลาของออเดอร์

ฐานข้อมูลบันทึกเวลาด้วยฟังก์ชัน datetime('now') ของ SQLite ซึ่งเป็นเวลา UTC ฟังก์ชัน orderTime() จึงแปลงเป็นเวลาไทย และคำนวณว่าออเดอร์ผ่านไปแล้วกี่นาที เพื่อแสดงบนหน้าครัว บริการ แคชเชียร์ และหน้าของลูกค้า

ตัวอย่างโค้ดที่ 4 index.js – ฟังก์ชัน orderTime() แปลงเวลาเป็นเวลาไทย

index.js

```
58  // แปลงเวลา (เก็บเป็น UTC) -> เวลาไทย + "ผ่านไปแล้วกี่นาที"
59  function orderTime(createdAt) {
60    if (!createdAt) return { clock: "", ago: "", mins: 0 };
61    const d = new Date(String(createdAt).replace(" ", "T") + "Z");
62    if (isNaN(d.getTime())) return { clock: "", ago: "", mins: 0 };
63    const clock = d.toLocaleTimeString("th-TH", { hour: "2-digit", minute: "2-digit", hour12:
    false, timeZone: "Asia/Bangkok" });
64    const mins = Math.max(0, Math.floor((Date.now() - d.getTime()) / 60000));
65    let ago;
66    if (mins < 1) ago = "เพิ่งสั่ง";
67    else if (mins < 60) ago = mins + " นาทีที่แล้ว";
68    else ago = Math.floor(mins / 60) + " ชม. " + (mins % 60) + " นาทีที่แล้ว";
69    return { clock, ago, mins };
70  }
```

คำอธิบายโค้ด

1) แปลงข้อความเวลา "YYYY-MM-DD HH:MM:SS" ให้เป็นรูปแบบ ISO โดยแทนช่องว่างด้วย T และต่อท้าย Z เพื่อบอกว่าเป็นเวลา UTC
2) toLocaleTimeString("th-TH", { timeZone: "Asia/Bangkok" }) แสดงเวลาเป็นชั่วโมง:นาทีตามเวลาประเทศไทย
3) mins คือจำนวนนาทีที่ผ่านไปตั้งแต่สั่ง คำนวณจาก Date.now() ลบเวลาที่สั่ง แล้วหารด้วย 60000 มิลลิวินาที
4) ข้อความ ago จะเป็น "เพิ่งสั่ง" ถ้าไม่ถึง 1 นาที, "X นาทีที่แล้ว" ถ้าไม่ถึงชั่วโมง หรือ "X ชม. Y นาทีที่แล้ว"
5) ฟังก์ชันคืนค่า { clock, ago, mins } โดยหน้าครัวใช้ mins กำหนดสีเตือนของการ์ดออเดอร์ที่รอนาน

1.5 การตั้งค่า Template Engine และ Middleware

Middleware คือฟังก์ชันที่ทำงานก่อนถึง Route ทุกครั้งที่มีคำขอเข้ามา ส่วนนี้กำหนดให้ใช้ EJS แสดงผลหน้าเว็บ อ่านข้อมูลจากฟอร์ม เปิดโฟลเดอร์ public และเปิดใช้ Session สำหรับเก็บตะกร้าสินค้า

ตัวอย่างโค้ดที่ 5 index.js – ตั้งค่า EJS, Session และข้อมูลที่ทุกหน้าใช้ร่วมกัน

index.js

```
app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));
app.use(express.urlencoded({ extended: true }));
app.use("/public", express.static(path.join(__dirname, "public")));
app.use(session({
  secret: "food-order-project-secret",
  resave: false,
  saveUninitialized: true
}));

// ใส่ข้อมูลร้าน + สรุปตะกร้า ให้ทุกหน้าเรียกใช้ได้
app.use((req, res, next) => {
  const cart = req.session.cart || [];
  res.locals.restaurant = RESTAURANT;
  res.locals.cartCount = cart.reduce((s, x) => s + Number(x.quantity || 0), 0);
  res.locals.cartTotal = cart.reduce((s, x) => s + Number(x.total_price || 0), 0);
  next();
});
```

คำอธิบายโค้ด

1) app.set("view engine", "ejs") กำหนดให้ res.render() ใช้เทมเพลต EJS ในโฟลเดอร์ views
2) express.urlencoded() แปลงข้อมูลจากฟอร์ม HTML (method="post") ให้อ่านได้ผ่าน req.body
3) express.static() ให้บริการไฟล์ในโฟลเดอร์ public ภายใต้ URL /public
4) session() สร้าง Session ให้แต่ละเบราว์เซอร์ (แท็บเล็ตแต่ละโต๊ะ) ใช้เก็บตะกร้า (req.session.cart) หมายเลขโต๊ะ และสถานะการล็อกอินของพนักงาน
5) Middleware บรรทัด 83–89 คำนวณจำนวนชิ้นในตะกร้า (cartCount) และยอดรวม (cartTotal) แล้วใส่ไว้ใน res.locals ทำให้ทุกเทมเพลตเรียกใช้ restaurant, cartCount และ cartTotal ได้ทันที โดยไม่ต้องส่งเองทุก Route

1.6 การอัปเดตหน้าจอแบบเรียลไทม์ (Server-Sent Events)

เมื่อลูกค้าสั่งอาหารหรือพนักงานเปลี่ยนสถานะ หน้าจออื่นต้องเห็นข้อมูลใหม่ทันที ระบบใช้ Server-Sent Events (SSE) ซึ่งเบราว์เซอร์จะเปิดการเชื่อมต่อค้างไว้ และเซิร์ฟเวอร์ส่งสัญญาณกลับมาได้ทุกเมื่อ

ตัวอย่างโค้ดที่ 6 index.js – ฟังก์ชัน broadcast() และเส้นทาง /events

index.js

```
// ---------- Real-time (Server-Sent Events) ----------
// ทุกจอ (ครัว/บริการ/แคชเชียร์/ลูกค้า) เปิด EventSource ไว้ พอมีการเปลี่ยนแปลงจะอัปเดตทันที
const sseClients = new Set();
function broadcast() {
  for (const res of sseClients) { try { res.write("data: update\n\n"); } catch (e) { } }
}
app.get("/events", (req, res) => {
  res.writeHead(200, {
    "Content-Type": "text/event-stream",
    "Cache-Control": "no-cache",
    "Connection": "keep-alive",
    "X-Accel-Buffering": "no"
  });
  res.write("retry: 3000\n\n");
  res.write(": connected\n\n");
  sseClients.add(res);
  const ping = setInterval(() => { try { res.write(": ping\n\n"); } catch (e) { } }, 25000);
  req.on("close", () => { clearInterval(ping); sseClients.delete(res); });
});
```

คำอธิบายโค้ด

1) sseClients เป็น Set เก็บการเชื่อมต่อ (res) ของทุกหน้าจอที่เปิด EventSource อยู่
2) broadcast() วนส่งข้อความ "data: update" ไปยังทุกการเชื่อมต่อ โดยไม่ส่งข้อมูลอื่น หน้าจอที่ได้รับจะโหลดข้อมูลใหม่เอง ครอบด้วย try/catch กันกรณีการเชื่อมต่อหลุดไปแล้ว
3) GET /events ตอบกลับด้วย Content-Type: text/event-stream และไม่ปิดการเชื่อมต่อ ส่วน retry: 3000 บอกเบราว์เซอร์ให้เชื่อมต่อใหม่ภายใน 3 วินาทีหากหลุด
4) setInterval() ส่งข้อความ ": ping" ทุก 25 วินาที เพื่อไม่ให้การเชื่อมต่อถูกตัดเพราะเงียบนานเกินไป
5) เมื่อผู้ใช้ปิดหน้าหรือเปลี่ยนหน้า (req.on("close")) จะหยุด ping และลบการเชื่อมต่อออกจาก sseClients
6) ทุก Route ที่เปลี่ยนข้อมูลออเดอร์หรือสถานะโต๊ะจะเรียก broadcast() เสมอ

1.7 การป้องกันหน้าพนักงานและค่าคงที่ของระบบ

หน้าครัว บริการ และแคชเชียร์ต้องผ่านการกรอก PIN ก่อน ระบบใช้ Middleware ชื่อ requireStaff ใส่ไว้หน้า Route ของพนักงานทุกเส้นทาง

ตัวอย่างโค้ดที่ 7 index.js – Middleware requireStaff และค่าคงที่โต๊ะ/หมวดหมู่

index.js

```
// ---------- กันหน้าพนักงานด้วย PIN ----------
function requireStaff(req, res, next) {
  if (!STAFF_PIN || req.session.staff) return next();
  return res.redirect("/staff/login?next=" + encodeURIComponent(req.originalUrl));
}

const TABLES = Array.from({ length: 16 }, (_, i) => i + 1);
const CATEGORIES = ["ซูชิ", "ของทอด", "เส้น/ซุป", "ของทานเล่น", "เครื่องดื่ม"];
```

คำอธิบายโค้ด

1) requireStaff() ตรวจว่า Session มีค่า staff = true (เคยกรอก PIN ถูก) หรือระบบปิดการใช้ PIN อยู่ ถ้าใช่จะเรียก next() ให้ทำงานต่อ
2) ถ้ายังไม่ล็อกอิน จะ redirect ไปหน้า /staff/login พร้อมแนบ ?next= เป็น URL เดิม เพื่อพากลับมาหน้าเดิมหลังกรอก PIN สำเร็จ
3) TABLES สร้างอาร์เรย์หมายเลขโต๊ะ 1 ถึง 16 ด้วย Array.from({ length: 16 })
4) CATEGORIES กำหนดหมวดหมู่อาหาร 5 หมวดตามลำดับที่แสดงบนหน้าเมนู ได้แก่ ซูชิ ของทอด เส้น/ซุป ของทานเล่น และเครื่องดื่ม

1.8 ข้อมูลตั้งต้นของ Add-on และเมนูอาหาร

ข้อมูลตัวเลือกเสริม (Add-on) และรายการอาหาร 50 รายการถูกกำหนดเป็นอาร์เรย์ในโค้ด แล้วนำไปบันทึกลงฐานข้อมูลตอนเปิดเซิร์ฟเวอร์ ทำให้แก้ไขเมนูได้ง่ายจากที่เดียว

ตัวอย่างโค้ดที่ 8 index.js – อาร์เรย์ addonData ข้อมูลตัวเลือกเสริมแยกตามหมวด

index.js

```
const addonData = [
  ["เส้น/ซุป", "ระดับความเผ็ด", "ไม่เผ็ด", 0, 1, 0],
  ["เส้น/ซุป", "ระดับความเผ็ด", "เผ็ดน้อย", 0, 2, 0],
  ["เส้น/ซุป", "ระดับความเผ็ด", "เผ็ดกลาง", 0, 3, 0],
  ["เส้น/ซุป", "ระดับความเผ็ด", "เผ็ดมาก", 0, 4, 0],
  ["เส้น/ซุป", "Topping", "เพิ่มไข่ต้ม", 10, 1, 1],
  ["เส้น/ซุป", "Topping", "เพิ่มไข่ดอง", 10, 2, 1],
  ["เส้น/ซุป", "Topping", "เพิ่มหน่อไม้", 10, 3, 1],
  ["เส้น/ซุป", "Topping", "เพิ่มสาหร่าย", 10, 4, 1],
  ["เส้น/ซุป", "Topping", "เพิ่มหมูชาชู", 20, 5, 1],

  // วาซาบิ/โชยุ เสิร์ฟให้ทุกโต๊ะอยู่แล้ว จึงตัดตัวเลือกรับ/ไม่รับออก เหลือเฉพาะ "เพิ่ม" แบบจ่ายเงิน
  ["ซูชิ", "เครื่องเคียง", "เพิ่มวาซาบิ", 10, 1, 1],
  ["ซูชิ", "เครื่องเคียง", "เพิ่มโชยุ", 10, 2, 1],
  ["ซูชิ", "เครื่องเคียง", "เพิ่มขิงดอง", 10, 3, 1],

  ["ของทอด", "ซอส", "ซอสมะเขือเทศ", 0, 1, 0],
  ["ของทอด", "ซอส", "ซอสพริก", 0, 2, 0],
  ["ของทอด", "ซอส", "มายองเนส", 0, 3, 0],
  ["ของทอด", "ซอส", "ไม่รับซอส", 0, 4, 0],
  ["ของทอด", "ผงคลุก", "ผงชีส", 10, 1, 1],
  ["ของทอด", "ผงคลุก", "ผงปาปริก้า", 10, 2, 1],
  ["ของทอด", "ผงคลุก", "ผงบาร์บีคิว", 10, 3, 1],

  ["ของทานเล่น", "ซอส", "ซอสมะเขือเทศ", 0, 1, 0],
  ["ของทานเล่น", "ซอส", "ซอสพริก", 0, 2, 0],
  ["ของทานเล่น", "ซอส", "มายองเนส", 0, 3, 0],
  ["ของทานเล่น", "ซอส", "ไม่รับซอส", 0, 4, 0],
  ["ของทานเล่น", "ผงคลุก", "ผงชีส", 10, 1, 1],
  ["ของทานเล่น", "ผงคลุก", "ผงปาปริก้า", 10, 2, 1],
  ["ของทานเล่น", "ผงคลุก", "ผงบาร์บีคิว", 10, 3, 1]
  // เครื่องดื่ม: ตัด add-on ทั้งหมด (ระดับความหวาน, เพิ่มไข่มุก, เพิ่มบุก) ออกตามที่กำหนด
];
```

คำอธิบายโค้ด

1) แต่ละแถวมีรูปแบบ [หมวดหมู่, ชนิดตัวเลือก, ชื่อตัวเลือก, ราคาเพิ่ม, ลำดับการแสดง, เลือกได้หลายอย่าง]
2) หมวด เส้น/ซุป มี "ระดับความเผ็ด" (เลือกได้อย่างเดียว ค่า multiple = 0 ไม่มีค่าใช้จ่าย) และ "Topping" (เลือกได้หลายอย่าง ค่า multiple = 1 ราคา 10–20 บาท)
3) หมวด ซูชิ มี "เครื่องเคียง" แบบจ่ายเงินเพิ่ม เช่น เพิ่มวาซาบิ เพิ่มโชยุ เพิ่มขิงดอง
4) หมวด ของทอด และ ของทานเล่น มี "ซอส" (เลือกได้อย่างเดียว) และ "ผงคลุก" (เลือกได้หลายอย่าง ราคา 10 บาท)
5) หมวด เครื่องดื่ม ไม่มี Add-on เมื่อกดเลือกเครื่องดื่มจึงมีแค่การเลือกจำนวน

ตัวอย่างโค้ดที่ 9 index.js – อาร์เรย์ menuNames รายการอาหาร 50 รายการ

index.js

```
const menuNames = [
  ["ซูชิแซลมอน", 25, "ซูชิ"], ["ซูชิทูนา", 35, "ซูชิ"], ["ซูชิซาบะดอง", 20, "ซูชิ"], ["ซูชิกุ้งสด", 20, "ซูชิ"], ["ซูชิไข่ปลาแซลมอน", 60, "ซูชิ"], ["ซูชิแซลมอนย่าง", 30, "ซูชิ"], ["ซูชิไข่กุ้ง", 20, "ซูชิ"], ["ซูชิไข่หวาน", 20, "ซูชิ"], ["แซลมอนโรล", 180, "ซูชิ"], ["ทูน่าโรล", 180, "ซูชิ"],
  ["ปีกไก่ทอดน้ำปลา", 100, "ของทอด"], ["ชีสบอล", 80, "ของทอด"], ["ทอดมันกุ้ง", 120, "ของทอด"], ["ไก่คาราอาเกะ", 80, "ของทอด"], ["ปลาหมึกชุบแป้งทอด", 100, "ของทอด"], ["เฟรนช์ฟรายส์", 70, "ของทอด"], ["หอมหัวใหญ่ทอด", 70, "ของทอด"], ["เกี๊ยวซ่าทอด", 80, "ของทอด"], ["เต้าหู้ทอด", 60, "ของทอด"], ["กุ้งเทมปุระ", 130, "ของทอด"],
  ["หงคัตสึราเมน", 170, "เส้น/ซุป"], ["โชยุราเมน", 150, "เส้น/ซุป"], ["มิโซะราเมน", 140, "เส้น/ซุป"], ["ชาชูเมน", 160, "เส้น/ซุป"], ["โซเมน", 100, "เส้น/ซุป"], ["ยากิโซบะ", 130, "เส้น/ซุป"], ["ซารุโซบะ", 120, "เส้น/ซุป"], ["โอเด้ง", 100, "เส้น/ซุป"], ["ซุปหัวไชเท้าต้มซีอิ๊ว", 70, "เส้น/ซุป"], ["ซุปมิโซะ", 30, "เส้น/ซุป"],
  ["ทาโกยากิ", 80, "ของทานเล่น"], ["พิซซ่าญี่ปุ่น", 130, "ของทานเล่น"], ["ถั่วแระญี่ปุ่น", 40, "ของทานเล่น"], ["ไข่ตุ๋นญี่ปุ่น", 40, "ของทานเล่น"], ["ยำสาหร่าย", 30, "ของทานเล่น"], ["ยากิโทริ", 40, "ของทานเล่น"], ["มันหวานญี่ปุ่นเผา", 100, "ของทานเล่น"], ["ทูน่าทาทากิ", 170, "ของทานเล่น"], ["เต้าหู้เย็นญี่ปุ่น", 60, "ของทานเล่น"], ["ฟักทองญี่ปุ่นต้มซีอิ๊ว", 60, "ของทานเล่น"],
  ["น้ำผึ้งมะนาวโซดา", 50, "เครื่องดื่ม"], ["โค้ก", 30, "เครื่องดื่ม"], ["สไปรท์", 30, "เครื่องดื่ม"], ["ชามะนาวเย็น", 50, "เครื่องดื่ม"], ["ชาดำเย็น", 50, "เครื่องดื่ม"], ["ชาไทยเย็น", 50, "เครื่องดื่ม"], ["โกโก้เย็น", 50, "เครื่องดื่ม"], ["ชาเขียว", 40, "เครื่องดื่ม"], ["น้ำส้ม", 50, "เครื่องดื่ม"], ["น้ำเปล่า", 20, "เครื่องดื่ม"]
];
```

คำอธิบายโค้ด

1) แต่ละรายการมีรูปแบบ [ชื่ออาหาร, ราคา, หมวดหมู่] แบ่งเป็น 5 หมวด หมวดละ 10 รายการ
2) ลำดับของรายการในแต่ละหมวดมีความสำคัญ เพราะใช้จับคู่กับไฟล์รูป เช่น รายการแรกของหมวดซูชิ (ซูชิแซลมอน) ใช้รูป sushi_1
3) เมื่อต้องการเพิ่มหรือลดเมนู ให้แก้ที่อาร์เรย์นี้ ระบบจะสร้างตาราง menu ใหม่เมื่อจำนวนรายการไม่ตรงกับในฐานข้อมูล

ตัวอย่างโค้ดที่ 10 index.js – ฟังก์ชัน localImage() หาไฟล์รูปของเมนู

index.js

```
function localImage(category, index) {
  const map = {
    "ของทอด": "fried_" + (index + 1),
    "ของทานเล่น": "snack_" + (index + 1),
    "ซูชิ": "sushi_" + (index + 1),
    "เครื่องดื่ม": "drink_" + (index + 1),
    "เส้น/ซุป": "noodle_" + (index + 1)
  };
  const base = map[category];
  const files = require("fs").readdirSync(path.join(__dirname, "public", "Pic"));
  const found = files.find(f => path.parse(f).name === base);
  return found ? "/public/Pic/" + encodeURIComponent(found).replace(/%2F/g, "/") : "";
}
```

คำอธิบายโค้ด

1) map แปลงชื่อหมวดเป็นคำนำหน้าชื่อไฟล์ ได้แก่ fried_, snack_, sushi_, drink_ และ noodle_ ต่อด้วยลำดับของเมนูในหมวด (เริ่มที่ 1)
2) fs.readdirSync() อ่านรายชื่อไฟล์ทั้งหมดในโฟลเดอร์ public/Pic แล้วหาไฟล์ที่ชื่อ (ไม่รวมนามสกุล) ตรงกัน จึงรองรับรูปได้ทั้ง .jpg .png .webp
3) คืนค่าเป็น URL เช่น /public/Pic/sushi_1.jpg สำหรับเก็บลงคอลัมน์ image ของตาราง menu หากไม่พบไฟล์จะคืนค่าว่าง

1.9 ฟังก์ชันช่วยจัดการฐานข้อมูล

ไลบรารี sqlite3 ทำงานแบบ Callback ระบบจึงสร้างฟังก์ชันห่อเป็น Promise เพื่อให้เขียนโค้ดด้วย async/await ได้อ่านง่ายขึ้น

ตัวอย่างโค้ดที่ 11 index.js – ฟังก์ชัน run(), get(), all(), tableNo() และ money()

index.js

```
176  function run(sql, params = []) {
177    return new Promise((resolve, reject) => db.run(sql, params, function (err) {
178      if (err) reject(err); else resolve(this);
179    }));
180  }
181  function get(sql, params = []) {
182    return new Promise((resolve, reject) => db.get(sql, params, (err, row) => err ? reject(err)
     : resolve(row)));
183  }
184  function all(sql, params = []) {
185    return new Promise((resolve, reject) => db.all(sql, params, (err, rows) => err ? reject(err)
     : resolve(rows)));
186  }
187  function tableNo(v) {
188    const n = Number(v);
189    return TABLES.includes(n) ? n : 1;
190  }
191  function money(n) { return Number(n || 0); }
```

คำอธิบายโค้ด

1) run(sql, params) ใช้กับคำสั่ง INSERT / UPDATE / DELETE และคืนค่า this ของ sqlite3 ซึ่งมี lastID (รหัสแถวที่เพิ่งเพิ่ม) ใช้ต่อในการบันทึกรายการอาหารของออเดอร์
2) get(sql, params) คืนข้อมูลแถวเดียว ส่วน all(sql, params) คืนข้อมูลทุกแถวเป็นอาร์เรย์
3) ทุกฟังก์ชันใช้เครื่องหมาย ? แทนค่าในคำสั่ง SQL (Parameterized Query) ป้องกันการโจมตีแบบ SQL Injection
4) tableNo(v) แปลงค่าเป็นตัวเลขและตรวจว่าอยู่ในช่วงโต๊ะ 1–16 หากไม่ถูกต้องจะใช้โต๊ะ 1 ป้องกันการส่งหมายเลขโต๊ะแปลกปลอมมาทาง URL

5) money(n) แปลงค่าให้เป็นตัวเลขเสมอ (ค่าว่างกลายเป็น 0) ใช้ตอนรวมยอดเงิน

1.10 การสร้างตารางและข้อมูลตั้งต้น (initDb)

ฟังก์ชัน initDb() ทำงานทุกครั้งก่อนเซิร์ฟเวอร์เริ่มรับคำขอ ทำหน้าที่สร้างตารางทั้ง 7 ตาราง ปรับโครงสร้างฐานข้อมูลเก่า และเติมข้อมูลตั้งต้น ความสัมพันธ์ของตารางเป็นไปตาม ER Diagram ในหัวข้อก่อนหน้า

ตัวอย่างโค้ดที่ 12 index.js – สร้างตาราง category, tables_status, menu และ addon

index.js

```
193 async function initDb() {
194   // ตารางหมวดหมู่ (lookup) — menu และ addon อ้างอิงด้วย category_id เป็น FK
195   await run(`CREATE TABLE IF NOT EXISTS category(
196     category_id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL UNIQUE)`);
197
198   // รองรับ DB เก่า: ถ้า menu/addon เดิมเก็บ category เป็น TEXT (ยังไม่มี category_id) ให้สร้างใหม่
199   const menuCols = await all("PRAGMA table_info(menu)");
200   if (menuCols.length && !menuCols.some(c => c.name === "category_id")) await run("DROP TABLE
      menu");
201   const addonCols = await all("PRAGMA table_info(addon)");
202   if (addonCols.length && !addonCols.some(c => c.name === "category_id")) await run("DROP
      TABLE addon");
203
204   await run(`CREATE TABLE IF NOT EXISTS tables_status(
205     table_no INTEGER PRIMARY KEY, status TEXT NOT NULL DEFAULT 'ว่าง', updated_at TEXT)`);
206   await run(`CREATE TABLE IF NOT EXISTS menu(
207     menu_id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, price INTEGER NOT NULL,
208     image TEXT, category_id INTEGER NOT NULL,
209     FOREIGN KEY(category_id) REFERENCES category(category_id))`);
210   await run(`CREATE TABLE IF NOT EXISTS addon(
211     addon_id INTEGER PRIMARY KEY AUTOINCREMENT, category_id INTEGER NOT NULL, addon_type TEXT
      NOT NULL,
212     option_name TEXT NOT NULL, extra_price INTEGER NOT NULL DEFAULT 0, sort_order INTEGER NOT
      NULL DEFAULT 0,
213     multiple INTEGER NOT NULL DEFAULT 0,
214     FOREIGN KEY(category_id) REFERENCES category(category_id))`);
```

คำอธิบายโค้ด

1) CREATE TABLE IF NOT EXISTS สร้างตารางเฉพาะเมื่อยังไม่มี ข้อมูลเดิมจึงไม่หายเมื่อเปิดเซิร์ฟเวอร์ซ้ำ
2) ตาราง category เก็บชื่อหมวดหมู่ (UNIQUE) และเป็นตารางหลักที่ menu และ addon อ้างอิงผ่าน category_id (Foreign Key)
3) PRAGMA table_info() ตรวจโครงสร้างตาราง menu และ addon เดิม ถ้าเป็นแบบเก่าที่ยังไม่มีคอลัมน์ category_id จะลบทิ้งเพื่อสร้างใหม่ (Migration)
4) ตาราง tables_status เก็บสถานะของโต๊ะแต่ละตัว ค่าเริ่มต้นคือ 'ว่าง'

5) ตาราง addon มีคอลัมน์ multiple บอกว่าเลือกได้หลายอย่าง (1) หรืออย่างเดียว (0) และ sort_order ใช้เรียงลำดับการแสดงผล

ตัวอย่างโค้ดที่ 13 index.js – สร้างตาราง orders, order_items และ payments

index.js

```
215  await run(`CREATE TABLE IF NOT EXISTS orders(
216    order_id INTEGER PRIMARY KEY AUTOINCREMENT, table_no INTEGER NOT NULL,
217    status TEXT NOT NULL DEFAULT 'กำลังทำ', total INTEGER NOT NULL DEFAULT 0,
218    created_at TEXT NOT NULL,
219    FOREIGN KEY(table_no) REFERENCES tables_status(table_no))`);
220  await run(`CREATE TABLE IF NOT EXISTS order_items(
221    order_item_id INTEGER PRIMARY KEY AUTOINCREMENT, order_id INTEGER NOT NULL,
222    menu_id INTEGER NOT NULL, name TEXT NOT NULL, quantity INTEGER NOT NULL,
223    unit_price INTEGER NOT NULL, total_price INTEGER NOT NULL, addons TEXT,
224    FOREIGN KEY(order_id) REFERENCES orders(order_id),
225    FOREIGN KEY(menu_id) REFERENCES menu(menu_id))`);
226  await run(`CREATE TABLE IF NOT EXISTS payments(
227    payment_id INTEGER PRIMARY KEY AUTOINCREMENT, table_no INTEGER NOT NULL,
228    total INTEGER NOT NULL, received INTEGER NOT NULL, change_amount INTEGER NOT NULL,
229    method TEXT NOT NULL, paid_at TEXT NOT NULL,
230    FOREIGN KEY(table_no) REFERENCES tables_status(table_no))`);
```

คำอธิบายโค้ด

1) ตาราง orders เก็บออเดอร์แต่ละครั้งที่ลูกค้ากดยืนยัน ได้แก่ หมายเลขโต๊ะ สถานะ (ค่าเริ่มต้น 'กำลังทำ') ยอดรวม และเวลาที่สั่ง
2) ตาราง order_items เก็บรายการอาหารในออเดอร์ โดยบันทึกชื่อและราคาไว้ด้วย แม้ภายหลังแก้ราคาในเมนู ประวัติการขายก็ยังถูกต้อง ส่วนคอลัมน์ addons เก็บตัวเลือกที่เลือกในรูปแบบ JSON
3) ตาราง payments เก็บประวัติการชำระเงิน ได้แก่ ยอดรวม เงินที่รับ เงินทอน ช่องทาง (เงินสด / QR CODE) และเวลาที่ชำระ
4) ทุกความสัมพันธ์เป็นแบบหนึ่งต่อกลุ่ม (1:N) ผ่าน FOREIGN KEY เช่น ออเดอร์หนึ่งมีได้หลายรายการอาหาร โต๊ะหนึ่งมีได้หลายออเดอร์

ตัวอย่างโค้ดที่ 14 index.js – เติมข้อมูลหมวดหมู่ เมนู Add-on และโต๊ะ

index.js

```
232   // seed หมวดหมู่ก่อน แล้วทำ map ชื่อ -> category_id
233   for (const name of CATEGORIES) await run("INSERT OR IGNORE INTO category(name) VALUES(?)",
    [name]);
234   const catRows = await all("SELECT category_id, name FROM category");
235   const catId = {};
236   catRows.forEach(r => { catId[r.name] = r.category_id; });
237
238   const menuCount = await get("SELECT COUNT(*) AS c FROM menu");
239   if (menuCount.c !== menuNames.length) {
240     await run("DELETE FROM menu");
241     for (let i = 0; i < menuNames.length; i++) {
242       const [name, price, category] = menuNames[i];
243       const sameCategoryIndex = menuNames.slice(0, i).filter(x => x[2] === category).length;
244       await run("INSERT INTO menu(name,price,image,category_id) VALUES(?,?,?,?)",
245         [name, price, localImage(category, sameCategoryIndex), catId[category]]);
246     }
247   }
248
249   await run("DELETE FROM addon");
250   for (const a of addonData) {
251     const [category, addon_type, option_name, extra_price, sort_order, multiple] = a;
252     await run("INSERT INTO
    addon(category_id,addon_type,option_name,extra_price,sort_order,multiple)
    VALUES(?,?,?,?,?,?)",
253       [catId[category], addon_type, option_name, extra_price, sort_order, multiple]);
254   }
255
256   for (const t of TABLES) {
257     await run("INSERT OR IGNORE INTO tables_status(table_no,status,updated_at)
    VALUES(?,?,datetime('now'))", [t, "ว่าง"]);
258   }
259 }
```

คำอธิบายโค้ด

1) INSERT OR IGNORE เพิ่มหมวดหมู่เฉพาะที่ยังไม่มี แล้วสร้างออบเจกต์ catId สำหรับแปลงชื่อหมวดเป็น category_id
2) ตาราง menu จะถูกล้างและเติมใหม่เฉพาะเมื่อจำนวนแถวในฐานข้อมูลไม่เท่ากับจำนวนรายการใน menuNames เพื่อไม่ให้ menu_id ของเมนูเปลี่ยนโดยไม่จำเป็น
3) sameCategoryIndex นับว่ารายการนี้เป็นลำดับที่เท่าไรของหมวด แล้วส่งให้ localImage() หาไฟล์รูป
4) ตาราง addon ถูกลบและเติมใหม่จาก addonData ทุกครั้งที่เปิดเซิร์ฟเวอร์ การแก้ราคาหรือชื่อ Add-on ในโค้ดจึงมีผลทันทีหลังรีสตาร์ต
5) สุดท้ายเพิ่มโต๊ะ 1–16 ลงตาราง tables_status ด้วยสถานะ 'ว่าง' (เฉพาะโต๊ะที่ยังไม่มี)

1.11 หน้าแรกและหน้าเมนูอาหาร

เมื่อแท็บเล็ตเปิดเว็บที่ /?table=N ระบบจะจำหมายเลขโต๊ะไว้ใน Session แล้วแสดงหน้าเลือกผู้ใช้งาน เมื่อลูกค้ากดเข้าหน้าเมนู ระบบจะดึงรายการอาหารทั้งหมดพร้อมชื่อหมวดหมู่จากฐานข้อมูล

ตัวอย่างโค้ดที่ 15 index.js – ฟังก์ชันตะกร้า และเส้นทาง / กับ /menu

index.js

```
async function cartItems(req) {
  return req.session.cart || [];
}
function cartTotal(cart) {
  return cart.reduce((s, x) => s + money(x.total_price), 0);
}

// หน้าแรก: เลือกบทบาท (ลูกค้า / พนักงาน)
// แท็บเล็ตแต่ละโต๊ะตั้งค่าเปิดที่ /?table=<เลขโต๊ะ> ครั้งเดียว ระบบจะจำโต๊ะไว้ให้
app.get("/", (req, res) => {
  const table = tableNo(req.query.table || req.session.table);
  req.session.table = table;
  res.render("landing", { table });
});

// หน้าเมนูของลูกค้า (เดิมอยู่ที่ /)
app.get("/menu", async (req, res) => {
  const table = tableNo(req.query.table || req.session.table);
  req.session.table = table;
  const menus = await all(`SELECT m.menu_id, m.name, m.price, m.image, m.category_id, c.name
AS category
    FROM menu m JOIN category c ON m.category_id = c.category_id
    ORDER BY m.category_id, m.menu_id`);
  res.render("home", { menus, table, categories: CATEGORIES, added: req.query.added || "" });
});
```

คำอธิบายโค้ด

1) cartItems() คืนตะกร้าจาก Session (ถ้ายังไม่มีคืนอาร์เรย์ว่าง) และ cartTotal() รวมราคาทุกรายการในตะกร้า
2) GET / อ่านหมายเลขโต๊ะจาก ?table= หรือจาก Session แล้วบันทึกกลับลง Session ทำให้ตั้งค่าแท็บเล็ตครั้งเดียวก็จำโต๊ะไว้ได้ จากนั้นแสดงหน้า landing.ejs
3) GET /menu ใช้คำสั่ง SQL JOIN ตาราง menu กับ category เพื่อได้ชื่อหมวดหมู่ของแต่ละเมนู เรียงตามหมวดและรหัสเมนู
4) ส่งค่า menus, table, categories และ added (ชื่ออาหารที่เพิ่งเพิ่มลงตะกร้า ใช้แสดงข้อความแจ้งเตือน) ไปให้เทมเพลต home.ejs

1.12 การเข้าสู่ระบบของพนักงาน

หน้าพนักงานแยกจากหน้าลูกค้าด้วย PIN เมื่อกรอกถูกต้องระบบจะจำสถานะไว้ใน Session จนกว่าจะกดออกจากระบบ

ตัวอย่างโค้ดที่ 16 index.js – เส้นทาง /staff, /staff/login และ /staff/logout

index.js

```
// หน้าเลือกแผนกของพนักงาน
app.get("/staff", requireStaff, (req, res) => {
  res.render("staff-select", {});
});
app.get("/staff/login", (req, res) => {
  if (!STAFF_PIN || req.session.staff) return res.redirect("/staff");
  res.render("staff-login", { error: "", next: req.query.next || "/staff" });
});
app.post("/staff/login", (req, res) => {
  const next = typeof req.body.next === "string" && req.body.next.startsWith("/") ? req.body.next : "/staff";
  if (String(req.body.pin || "") === STAFF_PIN) { req.session.staff = true; return res.redirect(next); }
  res.render("staff-login", { error: "PIN ไม่ถูกต้อง", next });
});
app.post("/staff/logout", (req, res) => { req.session.staff = false; res.redirect("/"); });
```

คำอธิบายโค้ด

1) GET /staff มี requireStaff คั่นไว้ จึงต้องล็อกอินก่อน แล้วจึงแสดงหน้าเลือกแผนก (staff-select.ejs)
2) GET /staff/login ถ้าปิดการใช้ PIN หรือล็อกอินแล้วจะพาไปหน้า /staff ทันที ไม่เช่นนั้นแสดงฟอร์มกรอก PIN
3) POST /staff/login เปรียบเทียบ PIN ที่กรอกกับ STAFF_PIN ถ้าตรงกันตั้งค่า req.session.staff = true แล้ว redirect ไปหน้าที่ต้องการ (next) ถ้าไม่ตรงแสดงข้อความ "PIN ไม่ถูกต้อง"
4) ค่า next ต้องขึ้นต้นด้วย / เท่านั้น ป้องกันการใช้ลิงก์พาไปเว็บไซต์ภายนอก (Open Redirect)
5) POST /staff/logout ตั้งค่า staff = false แล้วกลับไปหน้าแรก

1.13 การเลือก Add-on และเพิ่มอาหารลงตะกร้า

เมื่อลูกค้ากดปุ่ม + ที่เมนู ระบบจะแสดงตัวเลือกเสริมเฉพาะของหมวดหมู่นั้น และเมื่อกดเพิ่มลงตะกร้า เซิร์ฟเวอร์จะคำนวณราคาใหม่เองจากฐานข้อมูล ไม่ใช้ราคาที่ส่งมาจากหน้าเว็บ

ตัวอย่างโค้ดที่ 17 index.js – เส้นทาง GET /addon แสดงตัวเลือกเสริมของเมนู

index.js

```
301  app.get("/addon", async (req, res) => {
302    const menu = await get(`SELECT m.*, c.name AS category FROM menu m
303      JOIN category c ON m.category_id = c.category_id WHERE m.menu_id=?`,
     [Number(req.query.menu_id)]);
304    if (!menu) return res.status(404).send("ไม่พบเมนู");
305    const table = tableNo(req.query.table);
306    const addons = await all("SELECT * FROM addon WHERE category_id=? ORDER BY addon_type,
     sort_order", [menu.category_id]);
307    const groups = [];
308    for (const a of addons) {
309      let g = groups.find(x => x.type === a.addon_type);
310      if (!g) { g = { type: a.addon_type, multiple: Boolean(a.multiple), options: [] };
     groups.push(g); }
311      g.options.push(a);
312    }
313    res.render("addon", { menu, table, groups });
314  });
```

คำอธิบายโค้ด

1) ค้นหาเมนูจาก menu_id ที่ส่งมาทาง URL ถ้าไม่พบตอบกลับ 404 "ไม่พบเมนู"
2) ดึง Add-on เฉพาะที่ category_id ตรงกับหมวดของเมนู เรียงตามชนิดและลำดับการแสดง
3) วนลูปจัดกลุ่ม Add-on ตามชนิด (addon_type) เช่น กลุ่ม "ระดับความเผ็ด" และกลุ่ม "Topping" แต่ละกลุ่มจำค่า multiple ไว้ เพื่อให้หน้าเว็บเลือกใช้ Radio หรือ Checkbox
4) ส่ง menu, table และ groups ให้เทมเพลต addon.ejs

ตัวอย่างโค้ดที่ 18 index.js – เส้นทาง POST /add-to-cart บันทึกรายการลงตะกร้า

index.js

```
316 app.post("/add-to-cart", async (req, res) => {
317   const menu = await get("SELECT * FROM menu WHERE menu_id=?", [Number(req.body.menu_id)]);
318   if (!menu) return res.status(404).send("ไม่พบเมนู");
319   const table = tableNo(req.body.table);
320   const quantity = Math.max(1, Math.min(99, Number(req.body.quantity) || 1));
321   let ids = [];
322   Object.keys(req.body).filter(k => k.startsWith("addon_")).forEach(k => {
323     const v = req.body[k]; ids.push(...(Array.isArray(v) ? v : [v]));
324   });
325   ids = [...new Set(ids.map(Number).filter(Number.isInteger))];
326   let addons = [];
327   if (ids.length) addons = await all(`SELECT * FROM addon WHERE category_id=? AND addon_id IN
    (${ids.map(() => "?").join(",")})`,
328     [menu.category_id, ...ids]);
329   const addonPrice = addons.reduce((s, a) => s + a.extra_price, 0);
330   if (!req.session.cart) req.session.cart = [];
331   req.session.cart.push({
332     cart_id: Date.now() + Math.floor(Math.random() * 100000),
333     menu_id: menu.menu_id, name: menu.name, image: menu.image, base_price: menu.price,
334     addons: addons.map(a => ({ addon_id: a.addon_id, type: a.addon_type, name: a.option_name,
    price: a.extra_price })),
335     addon_price: addonPrice, quantity, unit_price: menu.price + addonPrice,
336     total_price: (menu.price + addonPrice) * quantity
337   });
338   req.session.table = table;
339   // เพิ่มลงตะกร้าแล้ว -> กลับไปหน้าเมนูเพื่อเลือกต่อ พร้อมแจ้งเตือน (มีแถบตะกร้าลอยให้กดไปชำระ)
340   res.redirect("/menu?table=" + table + "&added=" + encodeURIComponent(menu.name));
341 });
```

คำอธิบายโค้ด

1) จำกัดจำนวนให้อยู่ระหว่าง 1–99 ด้วย Math.max() และ Math.min()
2) รวบรวมรหัส Add-on จากทุกช่องที่ชื่อขึ้นต้นด้วย addon_ (ทั้ง Checkbox ที่อาจส่งมาหลายค่าและ Radio) แปลงเป็นตัวเลขและตัดค่าซ้ำด้วย Set
3) ดึง Add-on จากฐานข้อมูลอีกครั้งโดยกรองด้วย category_id ของเมนู ป้องกันการส่งรหัส Add-on ของหมวดอื่นหรือแก้ราคามาจากหน้าเว็บ
4) คำนวณ unit_price = ราคาอาหาร + ราคา Add-on และ total_price = unit_price × จำนวน แล้วเพิ่มเข้า req.session.cart พร้อม cart_id ที่ไม่ซ้ำกันสำหรับใช้ลบรายการ
5) เสร็จแล้ว redirect กลับหน้าเมนูพร้อม ?added=ชื่ออาหาร เพื่อแสดงข้อความแจ้งว่าเพิ่มสำเร็จ

1.14 การจัดการตะกร้าและหน้ารายการที่สั่ง

ตะกร้าสินค้าอยู่ใน Session ของแท็บเล็ตจนกว่าลูกค้าจะกดยืนยัน ลูกค้าดูตะกร้า ลบรายการ ล้างตะกร้า และติดตามสถานะออเดอร์ของโต๊ะตัวเองได้

ตัวอย่างโค้ดที่ 19 index.js – เส้นทาง /cart, /remove-from-cart, /clear-cart และ /orders

index.js

```
343  app.get("/cart", async (req, res) => {
344    const table = tableNo(req.query.table || req.session.table);
345    const cart = await cartItems(req);
346    res.render("cart", { cart, total: cartTotal(cart), table });
347  });
348  app.post("/remove-from-cart", (req, res) => {
349    const id = Number(req.body.cart_id);
350    req.session.cart = (req.session.cart || []).filter(x => x.cart_id !== id);
351    res.redirect("/cart?table=" + tableNo(req.body.table));
352  });
353  app.post("/clear-cart", (req, res) => { req.session.cart = []; res.redirect("/cart?table=" +
     tableNo(req.body.table)); });
354
355  app.get("/orders", async (req, res) => {
356    const table = tableNo(req.query.table || req.session.table);
357    // แสดงเฉพาะออเดอร์ที่ยังไม่ชำระ (พอจ่ายเงิน/ปิดบิลแล้ว ประวัติจะหายจากหน้าลูกค้า แต่ยังเก็บใน DB)
358    const orders = await all("SELECT * FROM orders WHERE table_no=? AND status!='ชำระแล้ว' ORDER
     BY order_id DESC", [table]);
359    for (const o of orders) { o.items = await all("SELECT * FROM order_items WHERE order_id=?",
     [o.order_id]); o.time = orderTime(o.created_at); }
360    res.render("orders", { orders, table, placed: req.query.placed || "" });
361  });
```

คำอธิบายโค้ด

1) GET /cart ส่งรายการในตะกร้าและยอดรวมให้เทมเพลต cart.ejs
2) POST /remove-from-cart ใช้ filter() เก็บเฉพาะรายการที่ cart_id ไม่ตรงกับที่ต้องการลบ
3) POST /clear-cart ตั้งตะกร้าเป็นอาร์เรย์ว่าง แล้วกลับไปหน้าตะกร้า
4) GET /orders ดึงออเดอร์ของโต๊ะที่สถานะยังไม่ใช่ 'ชำระแล้ว' เรียงจากล่าสุด พร้อมรายการอาหารและเวลาของแต่ละออเดอร์ เมื่อปิดบิลแล้วประวัติจะหายจากหน้าลูกค้า แต่ข้อมูลยังเก็บอยู่ในฐานข้อมูล

1.15 การยืนยันคำสั่งซื้อ

เมื่อลูกค้ากด "ยืนยันการสั่งอาหาร" ข้อมูลในตะกร้าจะถูกบันทึกลงฐานข้อมูลเป็นออเดอร์ใหม่ ส่งเข้าครัว และเปลี่ยนโต๊ะเป็นสถานะไม่ว่าง

ตัวอย่างโค้ดที่ 20 index.js – เส้นทาง POST /confirm-order

index.js

```
app.post("/confirm-order", async (req, res) => {
  const table = tableNo(req.body.table);
  const cart = req.session.cart || [];
  if (!cart.length) return res.redirect("/cart?table=" + table);
  const total = cartTotal(cart);
  const r = await run("INSERT INTO orders(table_no,status,total,created_at)
VALUES(?,?,?,datetime('now'))", [table, "กำลังทำ", total]);
  for (const item of cart) {
    await run("INSERT INTO
order_items(order_id,menu_id,name,quantity,unit_price,total_price,addons)
VALUES(?,?,?,?,?,?,?)",
      [r.lastID, item.menu_id, item.name, item.quantity, item.unit_price, item.total_price,
JSON.stringify(item.addons)]);
  }
  await run("UPDATE tables_status SET status='ไม่ว่าง',updated_at=datetime('now') WHERE
table_no=?", [table]);
  req.session.cart = [];
  broadcast(); // แจ้งครัว/แคชเชียร์แบบเรียลไทม์
  res.redirect("/orders?table=" + table + "&placed=1");
});
```

คำอธิบายโค้ด

1) ถ้าตะกร้าว่างจะกลับไปหน้าตะกร้า ไม่สร้างออเดอร์เปล่า
2) INSERT ข้อมูลลงตาราง orders ด้วยสถานะ 'กำลังทำ' ยอดรวม และเวลาปัจจุบัน แล้วใช้ r.lastID เป็นรหัสออเดอร์
3) วนลูปบันทึกทุกรายการในตะกร้าลงตาราง order_items โดยแปลงรายการ Add-on เป็นข้อความ JSON ด้วย JSON.stringify()
4) UPDATE ตาราง tables_status ให้โต๊ะนี้เป็น 'ไม่ว่าง' จากนั้นล้างตะกร้า
5) เรียก broadcast() เพื่อให้หน้าจอครัวและแคชเชียร์แสดงออเดอร์ใหม่ทันที แล้วพาลูกค้าไปหน้ารายการที่สั่ง (?placed=1 ใช้แสดงแถบแจ้งว่าส่งออเดอร์สำเร็จ)

1.16 การทำงานของครัวและพนักงานบริการ

สถานะของออเดอร์ไหลไปทางเดียวคือ กำลังทำ ⟶ พร้อมเสิร์ฟ ⟶ เสิร์ฟแล้ว ⟶ ชำระแล้ว ครัวและพนักงานบริการแต่ละฝ่ายเห็นเฉพาะออเดอร์ในสถานะที่ตัวเองต้องจัดการ

ตัวอย่างโค้ดที่ 21 index.js – ฟังก์ชัน getWorkOrders() และเส้นทางของครัว

index.js

```
// ---------- KITCHEN ----------
async function getWorkOrders(status) {
  const orders = await all("SELECT * FROM orders WHERE status=? ORDER BY order_id ASC",
[status]);
  for (const o of orders) { o.items = await all("SELECT * FROM order_items WHERE order_id=?",
[o.order_id]); o.time = orderTime(o.created_at); }
  return orders;
}
app.get("/kitchen", requireStaff, async (req, res) => {
  const orders = await getWorkOrders("กำลังทำ");
  res.render("kitchen", { orders, role: "พนักงานครัว" });
});
app.post("/kitchen/done", requireStaff, async (req, res) => {
  // เปลี่ยนสถานะเฉพาะออเดอร์ที่ยัง 'กำลังทำ' เท่านั้น กันหน้าค้าง/กดซ้ำย้อนสถานะ
  await run("UPDATE orders SET status='พร้อมเสิร์ฟ' WHERE order_id=? AND status='กำลังทำ'",
[Number(req.body.order_id)]);
  broadcast(); // ส่งต่อให้หน้าบริการทันที
  res.redirect("/kitchen");
});
```

คำอธิบายโค้ด

1) getWorkOrders(status) ดึงออเดอร์ตามสถานะที่ระบุ เรียงจากเก่าไปใหม่ (มาก่อนทำก่อน) พร้อมรายการอาหารและเวลา
2) GET /kitchen แสดงออเดอร์สถานะ 'กำลังทำ' ให้พนักงานครัว
3) POST /kitchen/done เปลี่ยนสถานะเป็น 'พร้อมเสิร์ฟ' โดยมีเงื่อนไข WHERE status='กำลังทำ' หากหน้าจอค้างหรือกดซ้ำ ออเดอร์จะไม่ถูกย้อนสถานะ
4) เรียก broadcast() เพื่อให้หน้าพนักงานบริการเห็นออเดอร์ที่พร้อมเสิร์ฟทันที

ตัวอย่างโค้ดที่ 22 index.js – เส้นทางของพนักงานบริการ

index.js

```
// ---------- SERVICE ----------
app.get("/service", requireStaff, async (req, res) => {
  const orders = await getWorkOrders("พร้อมเสิร์ฟ");
  res.render("service", { orders, role: "พนักงานบริการ" });
});
app.post("/service/served", requireStaff, async (req, res) => {
  // เปลี่ยนสถานะเฉพาะออเดอร์ที่ 'พร้อมเสิร์ฟ' เท่านั้น กันหน้าค้าง/กดซ้ำย้อนสถานะ
  await run("UPDATE orders SET status='เสิร์ฟแล้ว' WHERE order_id=? AND status='พร้อมเสิร์ฟ'",
[Number(req.body.order_id)]);
  broadcast(); // แจ้งแคชเชียร์ว่าเสิร์ฟครบแล้วพร้อมปิดบิล
  res.redirect("/service");
});
```

คำอธิบายโค้ด

1) GET /service แสดงออเดอร์สถานะ 'พร้อมเสิร์ฟ' โดยใช้ getWorkOrders() ร่วมกับหน้าครัว
2) POST /service/served เปลี่ยนสถานะเป็น 'เสิร์ฟแล้ว' เฉพาะออเดอร์ที่ยังเป็น 'พร้อมเสิร์ฟ' (ป้องกันการกดซ้ำแบบเดียวกับครัว)
3) broadcast() แจ้งหน้าแคชเชียร์ว่าเสิร์ฟแล้ว ทำให้ปุ่มรับชำระเงินพร้อมใช้งานเมื่อเสิร์ฟครบทุกออเดอร์

1.17 หน้าแคชเชียร์และการจัดการสถานะโต๊ะ

แคชเชียร์เห็นสถานะของโต๊ะทั้ง 16 โต๊ะ เลือกโต๊ะเพื่อดูบิล และปรับสถานะโต๊ะเองได้ในโหมดเปลี่ยนสถานะ

ตัวอย่างโค้ดที่ 23 index.js – ฟังก์ชัน billForTable() และเส้นทาง GET /cashier

index.js

```
// ---------- CASHIER ----------
async function tableOverview() {
  return await all("SELECT * FROM tables_status ORDER BY table_no");
}
async function billForTable(table) {
  const orders = await all("SELECT * FROM orders WHERE table_no=? AND status!='ชำระแล้ว' ORDER BY order_id", [table]);
  const items = [];
  for (const o of orders) {
    o.items = await all("SELECT * FROM order_items WHERE order_id=?", [o.order_id]);
    o.time = orderTime(o.created_at);
    o.items.forEach(i => items.push({ ...i, order_id: o.order_id }));
  }
  // ปิดบิลได้ต่อเมื่อทุกออเดอร์ของโต๊ะเสิร์ฟครบแล้ว
  const allServed = orders.length > 0 && orders.every(o => o.status === "เสิร์ฟแล้ว");
  return { orders, items, total: orders.reduce((s, o) => s + o.total, 0), allServed };
}
app.get("/cashier", requireStaff, async (req, res) => {
  const tables = await tableOverview();
  const selected = req.query.selected ? tableNo(req.query.selected) : null;
  const mode = req.query.mode === "change";
  const bill = selected ? await billForTable(selected) : null;
  res.render("cashier", { tables, selected, bill, mode, role: "พนักงานแคชเชียร์", notice: req.query.notice || "" });
});
```

คำอธิบายโค้ด

1) tableOverview() ดึงสถานะของทุกโต๊ะเรียงตามหมายเลข
2) billForTable(table) ดึงทุกออเดอร์ของโต๊ะที่ยังไม่ชำระ รวมรายการอาหารของทุกออเดอร์ และรวมยอดเงินของทั้งบิล
3) allServed เป็นจริงเมื่อมีอย่างน้อย 1 ออเดอร์และทุกออเดอร์มีสถานะ 'เสิร์ฟแล้ว' ใช้เป็นเงื่อนไขว่าปิดบิลได้หรือยัง

4) GET /cashier รับ ?selected= (โต๊ะที่เลือกดูบิล), ?mode=change (โหมดเปลี่ยนสถานะโต๊ะ) และ ?notice= (ข้อความเตือน) แล้วส่งให้ cashier.ejs

ตัวอย่างโค้ดที่ 24 index.js – โหมดเปลี่ยนสถานะโต๊ะของแคชเชียร์

index.js

```
app.post("/cashier/toggle-mode", requireStaff, (req, res) => {
  const selected = req.body.selected || "";
  res.redirect("/cashier?mode=change" + (selected ? "&selected=" + selected : ""));
});
app.post("/cashier/table-status", requireStaff, async (req, res) => {
  // เลือกโต๊ะในโหมดเปลี่ยนสถานะเท่านั้น ยังไม่เปลี่ยนสถานะจริง
  const table = tableNo(req.body.table);
  res.redirect("/cashier?mode=change&selected=" + table);
});

app.post("/cashier/confirm-status", requireStaff, async (req, res) => {
  const table = tableNo(req.body.table);
  const row = await get("SELECT status FROM tables_status WHERE table_no=?", [table]);
  if (!row) return res.redirect("/cashier");
  const next = row.status === "ว่าง" ? "ไม่ว่าง" : "ว่าง";
  await run("UPDATE tables_status SET status=?,updated_at=datetime('now') WHERE table_no=?",
[next, table]);
  broadcast();
  // ยืนยันแล้วกลับหน้าสถานะโต๊ะปกติทันที
  res.redirect("/cashier");
});
```

คำอธิบายโค้ด

1) POST /cashier/toggle-mode เข้าสู่โหมดเปลี่ยนสถานะโต๊ะ โดยจำโต๊ะที่เลือกอยู่ไว้
2) POST /cashier/table-status เป็นเพียงการเลือกโต๊ะในโหมดนี้ ยังไม่เปลี่ยนสถานะจริง เพื่อลดการกดผิด
3) POST /cashier/confirm-status อ่านสถานะปัจจุบันของโต๊ะแล้วสลับค่า ว่าง ↔ ไม่ว่าง บันทึกเวลาที่แก้ไข เรียก broadcast() แล้วกลับหน้าแคชเชียร์ปกติ
4) การเปลี่ยนสถานะโต๊ะด้วยวิธีนี้ไม่ส่งผลต่อออเดอร์ของโต๊ะ ใช้กรณีลูกค้านั่งแต่ยังไม่สั่ง หรือแก้สถานะที่ค้างอยู่

1.18 การรับชำระเงินและปิดบิล

การชำระเงินทำได้เมื่ออาหารทุกออเดอร์ของโต๊ะเสิร์ฟครบแล้ว รองรับทั้งเงินสดและการโอนผ่าน QR พร้อมเพย์ เมื่อชำระสำเร็จระบบจะบันทึกประวัติ ปิดออเดอร์ และคืนสถานะโต๊ะเป็นว่าง

ตัวอย่างโค้ดที่ 25 index.js – เส้นทาง POST /payment/confirm

index.js

```
app.post("/payment/confirm", requireStaff, async (req, res) => {
  const table = tableNo(req.body.table);
  const bill = await billForTable(table);
  const method = req.body.method === "QR CODE" ? "QR CODE" : "เงินสด";
  if (!bill.total) return res.redirect("/cashier?selected=" + table);
  // กันปิดบิลถ้ายังเสิร์ฟไม่ครบ
  if (!bill.allServed) return res.redirect("/cashier?selected=" + table + "&notice=" +
encodeURIComponent("ยังปิดบิลไม่ได้ ต้องเสิร์ฟอาหารครบทุกรายการก่อน"));
  // QR พร้อมเพย์ = จ่ายตามยอดบิลพอดี ไม่ต้องกรอกเงินรับ/ไม่มีเงินทอน
  const received = method === "QR CODE" ? bill.total : Math.max(0, Number(req.body.received)
|| 0);
  if (received < bill.total) return res.redirect(`/payment/${table}?error=ยอดรับเงินไม่
พอ&received=${received}&method=${encodeURIComponent(method)}`);
  const change = received - bill.total;
  await run("INSERT INTO payments(table_no,total,received,change_amount,method,paid_at)
VALUES(?,?,?,?,?,datetime('now'))",
    [table, bill.total, received, change, method]);
  await run("UPDATE orders SET status='ชำระแล้ว' WHERE table_no=? AND status!='ชำระแล้ว'",
[table]);
  await run("UPDATE tables_status SET status='ว่าง',updated_at=datetime('now') WHERE
table_no=?", [table]);
  broadcast(); // โต๊ะว่าง + ประวัติหายจากหน้าลูกค้าทันที
  res.redirect(`/payment/${table}/success?
total=${bill.total}&received=${received}&change=${change}&method=${encodeURIComponent(method)}
`);
});
```

คำอธิบายโค้ด

1) ช่องทางชำระเงินรับได้เพียง 2 ค่า ถ้าไม่ใช่ "QR CODE" จะถือเป็น "เงินสด" เสมอ
2) ตรวจซ้ำที่เซิร์ฟเวอร์ว่าบิลมียอดเงินและเสิร์ฟครบแล้ว (allServed) ถ้ายังไม่ครบจะกลับหน้าแคชเชียร์ พร้อมข้อความ "ยังปิดบิลไม่ได้ ต้องเสิร์ฟอาหารครบทุกรายการก่อน"
3) ถ้าจ่ายด้วย QR ยอดรับเงินเท่ากับยอดบิลพอดี ถ้าจ่ายเงินสดใช้ยอดที่แคชเชียร์กรอก ถ้าน้อยกว่ายอดบิลจะกลับไปหน้าชำระเงินพร้อมข้อความ "ยอดรับเงินไม่พอ"
4) บันทึกลงตาราง payments (ยอดรวม เงินรับ เงินทอน ช่องทาง เวลา) เปลี่ยนทุกออเดอร์ของโต๊ะเป็น 'ชำระแล้ว' และเปลี่ยนโต๊ะเป็น 'ว่าง'
5) broadcast() ทำให้หน้ารายการที่สั่งของลูกค้าว่างลงทันที แล้วพาไปหน้าชำระเงินสำเร็จพร้อมยอดเงินสำหรับแสดงใบเสร็จ

ตัวอย่างโค้ดที่ 26 index.js – หน้าชำระเงิน หน้าสำเร็จ และการเริ่มเซิร์ฟเวอร์

index.js

```
469  app.get("/payment/:table", requireStaff, async (req, res) => {
470    const table = tableNo(req.params.table);
471    const bill = await billForTable(table);
472    if (!bill.total) return res.redirect("/cashier?selected=" + table);
473    // กันเปิดหน้าชำระถ้ายังเสิร์ฟไม่ครบ
474    if (!bill.allServed) return res.redirect("/cashier?selected=" + table + "&notice=" +
     encodeURIComponent("ยังปิดบิลไม่ได้ ต้องเสิร์ฟอาหารครบทุกรายการก่อน"));
475    const qrPayload = promptpayPayload(RESTAURANT.promptpay, bill.total);
476    res.render("payment", { table, bill, query: req.query, qrPayload });
477  });
478  app.get("/payment/:table/success", requireStaff, (req, res) => {
479    const table = tableNo(req.params.table);
480    res.render("payment-success", { table, total: Number(req.query.total) || 0, received:
     Number(req.query.received) || 0, change: Number(req.query.change) || 0, method:
     req.query.method || "เงินสด", paidAt: orderTime(new Date().toISOString().slice(0,
     19).replace("T", " ")) });
481  });
482
483  app.get("/receipt/:table", (req, res) => res.redirect("/cashier?selected=" +
     tableNo(req.params.table)));
484
485  initDb().then(() => {
486    app.listen(PORT, () => console.log(`Server running at http://localhost:${PORT}`));
487  }).catch(err => { console.error(err); process.exit(1); });
```

คำอธิบายโค้ด

1) GET /payment/:table ตรวจเงื่อนไขเดียวกับตอนยืนยัน แล้วสร้าง Payload พร้อมเพย์ตามยอดบิลด้วย promptpayPayload() ส่งให้ payment.ejs
2) GET /payment/:table/success รับยอดรวม เงินรับ เงินทอน และช่องทางจาก URL มาแสดงในใบเสร็จ พร้อมเวลาปัจจุบัน
3) GET /receipt/:table เป็นเส้นทางสำรองที่พากลับไปหน้าแคชเชียร์โดยเลือกโต๊ะนั้นไว้
4) initDb().then() สร้างฐานข้อมูลให้เสร็จก่อน แล้วจึงเรียก app.listen() เปิดเซิร์ฟเวอร์ ถ้าเกิดข้อผิดพลาดจะแสดงใน console และหยุดโปรแกรม

1.19 เทมเพลตหน้าแรก (landing.ejs)

ทุกไฟล์ EJS ขึ้นต้นด้วยส่วน <head> แบบเดียวกัน คือกำหนดภาษาไทย ปรับขนาดให้พอดีหน้าจอแท็บเล็ต โหลดฟอนต์ Noto Sans Thai / Noto Serif Thai และไฟล์ style.css จึงอธิบายเพียงครั้งเดียวในหัวข้อนี้

ตัวอย่างโค้ดที่ 27 views/landing.ejs – หน้าเลือกผู้ใช้งาน

views/landing.ejs

```
<!doctype html>
<html lang="th">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title><%= restaurant.name %> · เลือกผู้ใช้งาน</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=Noto+Sans+Thai:wght@400;500;600;700&family=Noto+Serif+Thai:wght@600;700&display=swap"
rel="stylesheet">
  <link rel="stylesheet" href="/public/style.css">
</head>

<body>
  <div class="role-shell">
    <div class="role-brand">
      <span class="brand-mark big">咲</span>
      <h1><%= restaurant.name %></h1>
      <p><%= restaurant.tagline %></p>
    </div>
    <div class="role-grid">
      <a class="role-card customer" href="/menu?table=<%= table %>">
        <div class="role-emoji">🍽️</div>
        <h2>ลูกค้า</h2>
        <p>ดูเมนูและสั่งอาหาร</p>
        <span class="role-table">โต๊ะ <%= String(table).padStart(2,"0") %></span>
      </a>
      <a class="role-card staff" href="/staff">
        <div class="role-emoji">👨‍🍳</div>
        <h2>พนักงาน</h2>
        <p>ครัว · บริการ · แคชเชียร์</p>
        <span class="role-lock">🔒 ต้องใช้ PIN</span>
      </a>
    </div>
    <p class="role-hint">แท็บเล็ตของแต่ละโต๊ะถูกตั้งค่าเลขโต๊ะไว้แล้ว — กด “ลูกค้า” เพื่อเริ่มสั่งอาหารได้เลย</p>
  </div>
</body>

</html>
```

คำอธิบายโค้ด

1) <meta name="viewport"> ทำให้หน้าเว็บปรับขนาดตามความกว้างของแท็บเล็ต ส่วน <%= restaurant.name %> เป็นแท็ก EJS ที่แทรกชื่อร้านจากเซิร์ฟเวอร์ลงในหัวเรื่อง
2) การ์ด "ลูกค้า" ลิงก์ไปที่ /menu?table=<เลขโต๊ะ> และแสดงหมายเลขโต๊ะ 2 หลักด้วย String(table).padStart(2, "0") เช่น โต๊ะ 05
3) การ์ด "พนักงาน" ลิงก์ไปที่ /staff ซึ่งจะถามหา PIN ก่อน

4) 咲 คือโลโก้ตัวอักษร ซากุ (咲) ซึ่งใช้เป็นปุ่มลับกดค้างในหน้าของลูกค้า

1.20 เทมเพลตหน้าเมนูอาหาร (home.ejs)

หน้าเมนูแบ่งเป็นแถบด้านบน ปุ่มเลื่อนไปยังหมวดหมู่ และตารางการ์ดเมนูแยกตามหมวด

ตัวอย่างโค้ดที่ 28 views/home.ejs – แถบด้านบนและปุ่มหมวดหมู่

views/home.ejs

```
<body>
  <div class="app-shell">
    <header class="topbar">
      <div class="brand">
        <span class="brand-mark">咲</span>
        <span class="brand-text"><b><%= restaurant.name %></b><small><%= restaurant.tagline %>
</small></span>
      </div>
      <nav class="top-links">
        <a class="active" href="/menu?table=<%= table %>">เมนู</a>
        <a href="/orders?table=<%= table %>">รายการที่สั่ง</a>
        <a class="cart-link" href="/cart?table=<%= table %>">ตะกร้า<% if (cartCount > 0) { %>
<span class="cart-badge"><%= cartCount %></span><% } %></a>
        <span class="table-pill">โต๊ะ <%= String(table).padStart(2,"0") %></span>
      </nav>
    </header>
    <main class="content">
      <div class="hero-row">
        <div>
          <h1>เมนูอาหาร</h1>
          <p>เลือกอาหารอร่อยๆ สำหรับโต๊ะ <strong><%= String(table).padStart(2,"0") %></strong></p>
        </div>
      </div>

      <div class="cat-nav">
        <% categories.forEach(category=>{ if(menus.some(m=>m.category===category)){ %>
          <a href="#cat-<%= encodeURIComponent(category) %>"><%= category %></a>
        <% }}) %>
      </div>
```

คำอธิบายโค้ด

1) <header class="topbar"> แสดงโลโก้ ชื่อร้าน และเมนูนำทาง เมนู / รายการที่สั่ง / ตะกร้า ทุกลิงก์แนบ ?table= ไปด้วยเพื่อไม่ให้หมายเลขโต๊ะหาย
2) ถ้ามีสินค้าในตะกร้า (cartCount > 0) จะแสดงตัวเลขจำนวนชิ้น (cart-badge) ข้างคำว่าตะกร้า
3) ส่วน cat-nav วนลูป categories สร้างลิงก์ #cat-ชื่อหมวด เพื่อเลื่อนหน้าไปยังหมวดนั้น โดยแสดงเฉพาะหมวดที่มีเมนูอยู่จริง

ตัวอย่างโค้ดที่ 29 views/home.ejs – การ์ดเมนู ตะกร้าลอย และข้อความแจ้งเตือน

views/home.ejs

```
      <% categories.forEach(category=>{ const
categoryMenus=menus.filter(m=>m.category===category);
        if(!categoryMenus.length) return; %>
        <section class="category" id="cat-<%= encodeURIComponent(category) %>">
          <h2><%= category %></h2>
          <div class="menu-grid">
            <% categoryMenus.forEach(menu=>{ %>
              <article class="menu-card">
                <div class="menu-thumb">
                  <img src="<%= menu.image %>" alt="<%= menu.name %>" loading="lazy">
                </div>
                <div class="menu-info">
                  <div class="menu-meta">
                    <b><%= menu.name %></b>
                    <span><%= menu.price %> บาท</span>
                  </div>
                  <a class="add-button" href="/addon?menu_id=<%= menu.menu_id %>&table=<%=
table %>" aria-label="เพิ่ม <%= menu.name %>">+</a>
                </div>
              </article>
            <% }) %>
          </div>
        </section>
      <% }) %>
    </main>
  </div>

  <%- include("_floatingcart", { table: table }) %>

  <% if (added) { %>
    <div class="toast" id="toast">✓ เพิ่ม <b><%= added %></b> ลงตะกร้าแล้ว</div>
    <script>
      setTimeout(function(){ var t=document.getElementById('toast'); if(t){
t.classList.add('hide'); } }, 2600);
    </script>
  <% } %>
  <script src="/public/customer.js"></script>
</body>

</html>
```

คำอธิบายโค้ด

1) วนลูปแต่ละหมวด กรองเมนูของหมวดด้วย menus.filter() แล้วสร้าง <section> พร้อม id สำหรับลิงก์เลื่อนหน้า

2) การ์ดเมนู (menu-card) แสดงรูป (loading="lazy" โหลดเมื่อเลื่อนมาถึง) ชื่อ และราคา ปุ่ม + ลิงก์ไปที่ /addon?menu_id=...

3) <%- include("_floatingcart") %> แทรกปุ่มตะกร้าลอยที่ใช้ร่วมกับหน้ารายการที่สั่ง

4) ถ้ามีค่า added จะแสดง Toast "✓ เพิ่ม ... ลงตะกร้าแล้ว" แล้วใช้ setTimeout() ซ่อนอัตโนมัติหลัง 2.6 วินาที

ตัวอย่างโค้ดที่ 30 views/_floatingcart.ejs – ปุ่มตะกร้าลอย

views/_floatingcart.ejs

```
<% if (typeof cartCount !== "undefined" && cartCount > 0) { %>
  <a class="floating-cart" href="/cart?table=<%= table %>">
    <span class="fc-icon">🛒<span class="fc-count"><%= cartCount %></span></span>
    <span class="fc-text">ดูตะกร้า</span>
    <span class="fc-total"><%= cartTotal %> ฿</span>
  </a>
<% } %>
```

คำอธิบายโค้ด

1) เป็นเทมเพลตย่อย (Partial) แสดงเฉพาะเมื่อมีสินค้าในตะกร้า
2) แสดงไอคอนตะกร้าพร้อมจำนวนชิ้น ข้อความ "ดูตะกร้า" และยอดรวม (cartTotal) โดยค่าทั้งสองมาจาก res.locals ที่ Middleware เตรียมไว้

1.21 เทมเพลตหน้าเลือก Add-on (addon.ejs)

หน้าปรับแต่งรายการอาหาร สร้างตัวเลือกตามกลุ่ม Add-on ที่ได้จากเซิร์ฟเวอร์ และคำนวณราคารวมให้ลูกค้าเห็นทันทีเมื่อเปลี่ยนตัวเลือก

ตัวอย่างโค้ดที่ 31 views/addon.ejs – ข้อมูลเมนูและฟอร์มตัวเลือก

views/addon.ejs

```
<main class="content narrow">
  <a class="back-link" href="/menu?table=<%= table %>">← กลับไปเลือกเมนู</a>
  <div class="product-head">
    <img src="<%= menu.image %>" alt="<%= menu.name %>">
    <div>
      <span class="product-cat"><%= menu.category %></span>
      <h1><%= menu.name %></h1>
      <div class="price"><%= menu.price %> บาท</div>
    </div>
  </div>
  <form action="/add-to-cart" method="post" id="addonForm">
    <input type="hidden" name="menu_id" value="<%= menu.menu_id %>">
    <input type="hidden" name="table" value="<%= table %>">
    <% groups.forEach((group,gi)=>{ %>
      <section class="option-box">
        <h2><%= group.type %>
          <small class="addon-rule <%= group.multiple ? 'multi' : 'single' %>">
            <%= group.multiple ? "เลือกได้หลายอย่าง" : "เลือกได้อย่างใดอย่างหนึ่ง" %>
          </small>
        </h2>
        <div class="options">
          <% group.options.forEach((option,oi)=>{ %>
            <label class="option-row">
              <% if (option.multiple) { %>
                <input type="checkbox" name="addon_<%= gi %>" value="<%= option.addon_id %>"
data-price="<%= option.extra_price %>">
              <% } else { %>
                <input type="radio" name="addon_single_<%= gi %>" value="<%= option.addon_id
%>" data-price="<%= option.extra_price %>" <%= oi===0 ? 'checked' : '' %>>
              <% } %>
              <span class="option-name"><%= option.option_name %></span>
              <span class="option-price <%= option.extra_price ? '' : 'free' %>"><%=
option.extra_price ? '+' + option.extra_price + ' บาท' : 'ฟรี' %></span>
            </label>
          <% }) %>
        </div>
      </section>
    <% }) %>
```

คำอธิบายโค้ด

1) ส่วน product-head แสดงรูป หมวดหมู่ ชื่อ และราคาของเมนูที่เลือก
2) ฟอร์มส่งข้อมูลไปที่ /add-to-cart แบบ POST พร้อม hidden input ของ menu_id และหมายเลขโต๊ะ
3) แต่ละกลุ่มแสดงป้ายกำกับ "เลือกได้หลายอย่าง" หรือ "เลือกได้อย่างใดอย่างหนึ่ง" ตามค่า group.multiple
4) ตัวเลือกแบบเลือกได้หลายอย่างใช้ <input type="checkbox"> ส่วนแบบเลือกอย่างเดียวใช้ <input type="radio"> และเลือกตัวแรกไว้ให้ (checked)
5) data-price เก็บราคาเพิ่มของตัวเลือกไว้ให้ JavaScript ใช้คำนวณ ถ้าไม่มีค่าใช้จ่ายจะแสดงคำว่า "ฟรี"

ตัวอย่างโค้ดที่ 32 views/addon.ejs – ปุ่มจำนวนและแถบเพิ่มลงตะกร้า

views/addon.ejs

```
63        <section class="option-box quantity-row">
64          <span>จำนวน</span>
65          <div class="qty-stepper">
66            <button type="button" class="qty-btn" data-step="-1">-</button>
67            <input type="number" name="quantity" id="qty" value="1" min="1" max="99">
68            <button type="button" class="qty-btn" data-step="1">+</button>
69          </div>
70        </section>
71        <div class="addon-actionbar">
72          <div class="addon-sum">รวม <strong id="sumPrice"><%= menu.price %></strong> บาท</div>
73          <button class="primary-btn" type="submit">เพิ่มลงตะกร้า</button>
74        </div>
75      </form>
76    </main>
77  </div>
```

คำอธิบายโค้ด

1) qty-stepper ประกอบด้วยปุ่ม – / + (type="button" จึงไม่ส่งฟอร์ม) และช่องจำนวน (min=1, max=99)

2) แถบ addon-actionbar ติดด้านล่างจอ แสดงราคารวมใน #sumPrice (ค่าเริ่มต้นคือราคาอาหาร) และปุ่ม "เพิ่มลงตะกร้า" ซึ่งเป็นปุ่มส่งฟอร์ม

ตัวอย่างโค้ดที่ 33 views/addon.ejs – สคริปต์คำนวณราคารวม

views/addon.ejs

```
  <script>
    (function () {
      var base = <%= menu.price %>;
      var form = document.getElementById('addonForm');
      var qty = document.getElementById('qty');
      var sum = document.getElementById('sumPrice');
      function recalc() {
        var add = 0;
        form.querySelectorAll('input[data-price]').forEach(function (el) {
          if ((el.type === 'checkbox' || el.type === 'radio') && el.checked) add +=
Number(el.dataset.price) || 0;
        });
        var q = Math.max(1, Math.min(99, Number(qty.value) || 1));
        sum.textContent = (base + add) * q;
      }
      form.addEventListener('change', recalc);
      document.querySelectorAll('.qty-btn').forEach(function (b) {
        b.addEventListener('click', function () {
          qty.value = Math.max(1, Math.min(99, (Number(qty.value) || 1) +
Number(b.dataset.step)));
          recalc();
        });
      });
      recalc();
    })();
  </script>
  <script src="/public/customer.js"></script>
</body>

</html>
```

คำอธิบายโค้ด

1) var base = <%= menu.price %> ฝังราคาอาหารจากเซิร์ฟเวอร์ลงในตัวแปร JavaScript
2) ฟังก์ชัน recalc() รวมราคา data-price ของตัวเลือกที่ถูกเลือก แล้วคูณด้วยจำนวน แสดงผลใน #sumPrice
3) form.addEventListener('change', recalc) คำนวณใหม่ทุกครั้งที่เปลี่ยนตัวเลือก ส่วนปุ่ม – / + ปรับจำนวนตาม data-step และจำกัดค่า 1–99
4) ราคาที่แสดงนี้มีไว้ให้ลูกค้าดูเท่านั้น ราคาจริงคำนวณใหม่ที่เซิร์ฟเวอร์ตอนเพิ่มลงตะกร้า

1.22 เทมเพลตหน้าตะกร้า (cart.ejs)

หน้าตะกร้าแสดงรายการอาหารที่เลือกไว้ทั้งหมด พร้อมปุ่มลบ ล้างตะกร้า และยืนยันการสั่งอาหาร

ตัวอย่างโค้ดที่ 34 views/cart.ejs – รายการอาหารในตะกร้า

views/cart.ejs

```
<main class="content narrow">
  <a class="back-link" href="/menu?table=<%= table %>">← เลือกอาหารเพิ่ม</a>
  <div class="section-title">
    <h1>ตะกร้าอาหาร</h1>
    <span class="pill-count"><%= cart.length %> รายการ</span>
  </div>
  <% if(!cart.length){ %>
    <div class="empty">
      <div class="empty-emoji">🍜</div>
      <p>ยังไม่มีรายการอาหารในตะกร้า</p>
      <a class="primary-btn" href="/menu?table=<%= table %>">เลือกอาหาร</a>
    </div>
  <% } else { %>
    <div class="cart-list">
      <% cart.forEach(item=>{ %>
        <article class="cart-card">
          <img src="<%= item.image %>" alt="<%= item.name %>">
          <div class="cart-main">
            <h2><%= item.name %> <span class="cart-qty">× <%= item.quantity %></span></h2>
            <div class="cart-base">อาหาร <%= item.base_price %> บาท</div>
            <% if(item.addons.length){ %>
              <div class="addons">
                <% item.addons.forEach(a=>{ %>
                  <span class="addon-chip"><%= a.name %><%= a.price ? ' +' + a.price : '' %>
</span>
                <% }) %>
              </div>
            <% } %>
          </div>
```

คำอธิบายโค้ด

1) แสดงจำนวนรายการด้วย cart.length ถ้าตะกร้าว่างแสดงข้อความ "ยังไม่มีรายการอาหารในตะกร้า" พร้อมปุ่มเลือกอาหาร

2) การ์ดแต่ละรายการแสดงรูป ชื่อ จำนวน (× quantity) และราคาอาหารเดิม (base_price)

3) Add-on ที่เลือกแสดงเป็นป้าย (addon-chip) ถ้ามีราคาเพิ่มจะแสดง +ราคา ต่อท้าย

ตัวอย่างโค้ดที่ 35 views/cart.ejs – ปุ่มลบ ยอดรวม และปุ่มยืนยัน

views/cart.ejs

```
56                <div class="cart-price">
57                  <b><%= item.total_price %> ฿</b>
58                  <form action="/remove-from-cart" method="post">
59                    <input type="hidden" name="cart_id" value="<%= item.cart_id %>">
60                    <input type="hidden" name="table" value="<%= table %>">
61                    <button class="danger-btn" type="submit">ลบ</button>
62                  </form>
63                </div>
64              </article>
65            <% }) %>
66          </div>
67          <div class="total-box">
68            <div class="total-row"><span>ยอดรวมทั้งหมด</span><strong><%= total %> บาท</strong>
    </div>
69            <div class="actions">
70              <form action="/clear-cart" method="post">
71                <input type="hidden" name="table" value="<%= table %>">
72                <button class="secondary-btn" type="submit">ล้างตะกร้า</button>
73              </form>
74              <form action="/confirm-order" method="post">
75                <input type="hidden" name="table" value="<%= table %>">
76                <button class="primary-btn" type="submit">ยืนยันการสั่งอาหาร</button>
77              </form>
78            </div>
79          </div>
80        <% } %>
81      </main>
82    </div>
83    <script src="/public/customer.js"></script>
84  </body>
85
86  </html>
```

คำอธิบายโค้ด

1) ปุ่ม "ลบ" อยู่ในฟอร์มเล็กที่ส่ง cart_id ไปที่ /remove-from-cart
2) กล่อง total-box แสดง "ยอดรวมทั้งหมด" ของตะกร้า
3) ปุ่ม "ล้างตะกร้า" ส่งไป /clear-cart และปุ่ม "ยืนยันการสั่งอาหาร" ส่งไป /confirm-order ทั้งสองฟอร์มส่งหมายเลขโต๊ะไปด้วย

1.23 เทมเพลตหน้ารายการที่สั่ง (orders.ejs)

หน้ารายการที่สั่งให้ลูกค้าติดตามสถานะอาหารของโต๊ะ และอัปเดตเองแบบเรียลไทม์เมื่อครัวหรือพนักงานเปลี่ยนสถานะ

ตัวอย่างโค้ดที่ 36 views/orders.ejs – รายการออเดอร์และสถานะ

views/orders.ejs

```
  <main class="content narrow">
    <% if(placed){ %>
      <div class="banner-success">✓ ส่งออเดอร์เข้าครัวเรียบร้อยแล้ว กำลังเตรียมอาหารให้คุณ</div>
    <% } %>
    <div class="section-title">
      <h1>รายการที่สั่ง</h1>
      <a class="outline-btn sm" href="/menu?table=<%= table %>">สั่งเพิ่ม</a>
    </div>
    <% if(!orders.length){ %>
      <div class="empty">
        <div class="empty-emoji">🍽️</div>
        <p>ยังไม่มีรายการสั่งอาหาร</p>
        <a class="primary-btn" href="/menu?table=<%= table %>">เริ่มสั่งอาหาร</a>
      </div>
    <% } %>
    <% orders.forEach(order=>{ %>
      <article class="order-history">
        <div class="order-head">
          <div class="order-id">
            <b>ออเดอร์ #<%= order.order_id %></b>
            <span class="order-time">🕒 <%= order.time.clock %> · <%= order.time.ago %></span>
          </div>
          <span class="status status-<%= order.status==='กำลังทำ'?'cooking':order.status==='พร้อมเสิร์ฟ'?'ready':order.status==='เสิร์ฟแล้ว'?'served':'paid' %>">
            <%= order.status %>
          </span>
        </div>
        <% order.items.forEach(item=>{ let ads=[]; try{ads=JSON.parse(item.addons||"[]")}catch(e){} %>
          <div class="order-line">
            <span><%= item.name %> × <%= item.quantity %>
              <% if(ads.length){ %><small><%= ads.map(a=>a.name).join(", ") %></small><% } %>
            </span>
            <b><%= item.total_price %> บาท</b>
          </div>
        <% }) %>
        <div class="order-total">รวม <%= order.total %> บาท</div>
      </article>
    <% }) %>
  </main>
</div>

<%- include("_floatingcart", { table: table }) %>
```

คำอธิบายโค้ด

1) ถ้าเพิ่งยืนยันออเดอร์ (placed) แสดงแถบสีเขียว "ส่งออเดอร์เข้าครัวเรียบร้อยแล้ว"

2) แต่ละออเดอร์แสดงหมายเลข เวลาที่สั่ง เวลาที่ผ่านไป และป้ายสถานะ

3) คลาสของป้ายสถานะแปลงจากข้อความภาษาไทย เช่น 'กำลังทำ' ⟶ status-cooking (สีส้ม), 'พร้อมเสิร์ฟ' ⟶ status-ready (สีเหลือง), 'เสิร์ฟแล้ว' ⟶ status-served (สีเขียว)

4) คอลัมน์ addons ถูกแปลงกลับด้วย JSON.parse() ภายใน try/catch แล้วแสดงชื่อ Add-on ใต้ชื่ออาหาร

ตัวอย่างโค้ดที่ 37 views/orders.ejs – สคริปต์อัปเดตแบบเรียลไทม์

views/orders.ejs

```
69    <script>
70      // Real-time: อัปเดตทันทีเมื่อมีการเปลี่ยนแปลงออเดอร์/โต๊ะ (ผ่าน Server-Sent Events)
71      (function () {
72        var t, es;
73        function connect() {
74          try {
75            es = new EventSource('/events');
76            es.onmessage = function () { clearTimeout(t); t = setTimeout(function () {
    location.reload(); }, 300); };
77          } catch (e) { }
78        }
79        connect();
80        // ปิดการเชื่อมต่อทันทีก่อนเปลี่ยนหน้า กันเบราว์เซอร์ค้าง (จำกัดจำนวน connection ต่อเว็บ)
81        window.addEventListener('pagehide', function () { try { if (es) es.close(); } catch (e) {
    } });
82        window.addEventListener('beforeunload', function () { try { if (es) es.close(); } catch
    (e) { } });
83        // สำรอง: รีเฟรชทุก 60 วิ เผื่อการเชื่อมต่อ real-time หลุด
84        setInterval(function () { location.reload(); }, 60000);
85      })();
86    </script>
87    <script src="/public/customer.js"></script>
88  </body>
89
90  </html>
```

คำอธิบายโค้ด

1) new EventSource('/events') เชื่อมต่อกับเซิร์ฟเวอร์ค้างไว้ เมื่อได้รับข้อความจะเรียก location.reload() เพื่อโหลดข้อมูลใหม่
2) ใช้ clearTimeout() + setTimeout(300) หน่วงเวลาเล็กน้อย (Debounce) ถ้ามีหลายเหตุการณ์ติดกันจะรีโหลดเพียงครั้งเดียว
3) ปิดการเชื่อมต่อในเหตุการณ์ pagehide / beforeunload เพราะเบราว์เซอร์จำกัดจำนวนการเชื่อมต่อค้างต่อเว็บไซต์
4) setInterval() รีโหลดทุก 60 วินาทีเป็นระบบสำรองในกรณีการเชื่อมต่อเรียลไทม์หลุด สคริปต์ชุดเดียวกันนี้ใช้ในหน้าครัว บริการ และแคชเชียร์

1.24 สคริปต์ปุ่มลับของพนักงาน (customer.js)

แท็บเล็ตของลูกค้าไม่มีปุ่มกลับไปหน้าแรกให้เห็น เพื่อป้องกันลูกค้าเข้าหน้าพนักงาน พนักงานจึงใช้วิธีกดค้างที่โลโก้แทน

ตัวอย่างโค้ดที่ 38 public/customer.js – กดค้างที่โลโก้เพื่อกลับหน้าแรก

public/customer.js

```
// ปุ่มลับสำหรับพนักงาน: "กดค้าง" ที่โลโก้ 呋 ประมาณ 1.2 วินาที เพื่อย้อนกลับไปหน้าเลือกผู้ใช้ (/)
// ซ่อนจากลูกค้าทั่วไป (ไม่มีปุ่มให้เห็น ต้องกดค้างเท่านั้น)
(function () {
  var mark = document.querySelector(".brand-mark");
  if (!mark) return;
  var timer = null, HOLD = 1200;
  mark.style.cursor = "default";
  mark.style.webkitUserSelect = "none";
  mark.style.userSelect = "none";
  function start() { cancel(); timer = setTimeout(function () { window.location.href = "/"; },
HOLD); }
  function cancel() { if (timer) { clearTimeout(timer); timer = null; } }
  mark.addEventListener("touchstart", start, { passive: true });
  mark.addEventListener("touchend", cancel);
  mark.addEventListener("touchmove", cancel, { passive: true });
  mark.addEventListener("touchcancel", cancel);
  mark.addEventListener("mousedown", start);
  mark.addEventListener("mouseup", cancel);
  mark.addEventListener("mouseleave", cancel);
  // กัน context menu ตอนกดค้างบนมือถือ
  mark.addEventListener("contextmenu", function (e) { e.preventDefault(); });
})();
```

คำอธิบายโค้ด

1) ค้นหาโลโก้ .brand-mark ถ้าหน้านั้นไม่มีโลโก้จะหยุดทำงาน
2) start() ตั้งเวลา 1.2 วินาที (HOLD = 1200) ถ้ายังกดค้างอยู่ครบเวลาจะพาไปหน้า / ส่วน cancel() ยกเลิกการตั้งเวลา
3) รองรับทั้งหน้าจอสัมผัส (touchstart / touchend / touchmove) และเมาส์ (mousedown / mouseup / mouseleave)
4) ปิดการเลือกข้อความ (user-select: none) และป้องกันเมนูคลิกขวา (contextmenu) ที่จะเด้งขึ้นเวลากดค้างบนแท็บเล็ต

1.25 เทมเพลตหน้าพนักงาน (staff-login.ejs และ staff-select.ejs)

พนักงานกรอก PIN แล้วเลือกแผนกที่ต้องการเข้าใช้งาน

ตัวอย่างโค้ดที่ 39 views/staff-login.ejs – ฟอร์มกรอก PIN

views/staff-login.ejs

```
<body>
  <div class="role-shell">
    <div class="pin-card">
      <div class="role-emoji">🔒</div>
      <h1>เข้าสู่ระบบพนักงาน</h1>
      <p>กรอก PIN เพื่อเข้าหน้าครัว / บริการ / แคชเชียร์</p>
      <% if (error) { %><div class="error center"><%= error %></div><% } %>
      <form action="/staff/login" method="post" class="pin-form">
        <input type="hidden" name="next" value="<%= next %>">
        <input type="password" name="pin" inputmode="numeric" autocomplete="off" placeholder="• • • •" autofocus>
        <button class="primary-btn full" type="submit">เข้าสู่ระบบ</button>
      </form>
      <a class="back-link dark" href="/">← กลับหน้าแรก</a>
    </div>
  </div>
</body>

</html>
```

คำอธิบายโค้ด

1) ถ้ามีข้อความ error (กรอก PIN ผิด) จะแสดงกล่องสีแดงด้านบนฟอร์ม

2) hidden input ชื่อ next เก็บหน้าที่ต้องการไปหลังล็อกอินสำเร็จ

3) ช่อง PIN ใช้ type="password" ซ่อนตัวเลข และ inputmode="numeric" ให้แท็บเล็ตแสดงแป้นตัวเลข autofocus ทำให้พิมพ์ได้ทันที

ตัวอย่างโค้ดที่ 40 views/staff-select.ejs – หน้าเลือกแผนก

views/staff-select.ejs

```
<body>
  <div class="role-shell">
    <div class="role-brand">
      <span class="brand-mark big">咲</span>
      <h1>พนักงาน</h1>
      <p>เลือกแผนกที่ต้องการเข้าใช้งาน</p>
    </div>
    <div class="role-grid three">
      <a class="role-card dept cook" href="/kitchen">
        <div class="role-emoji">👨‍🍳</div>
        <h2>ครัว</h2>
        <p>รับออเดอร์ · ทำอาหาร</p>
      </a>
      <a class="role-card dept serve" href="/service">
        <div class="role-emoji">🍽️</div>
        <h2>บริการ</h2>
        <p>เสิร์ฟอาหารให้ลูกค้า</p>
      </a>
      <a class="role-card dept cash" href="/cashier">
        <div class="role-emoji">💵</div>
        <h2>แคชเชียร์</h2>
        <p>สถานะโต๊ะ · ชำระเงิน</p>
      </a>
    </div>
    <div class="role-foot">
      <a class="back-link dark" href="/">← หน้าแรก</a>
      <form action="/staff/logout" method="post"><button class="secondary-btn"
type="submit">ออกจากระบบพนักงาน</button></form>
    </div>
  </div>
</body>

</html>
```

คำอธิบายโค้ด

1) role-grid three แสดงการ์ด 3 ใบ ได้แก่ ครัว (/kitchen) บริการ (/service) และแคชเชียร์ (/cashier) แต่ละใบมีสีขอบบนต่างกัน

2) ด้านล่างมีลิงก์กลับหน้าแรก และปุ่ม "ออกจากระบบพนักงาน" ส่งไป /staff/logout

1.26 เทมเพลตหน้าครัว (kitchen.ejs)

หน้าจอครัว (Kitchen Display) แสดงออเดอร์ที่ต้องทำเป็นการ์ด เรียงตามลำดับที่สั่ง และเปลี่ยนสีเตือนเมื่อออเดอร์รอนาน

ตัวอย่างโค้ดที่ 41 views/kitchen.ejs – แถบเมนูพนักงานและหัวข้อ

views/kitchen.ejs

```
14  <body class="staff-body">
15    <div class="staff-shell">
16      <header class="staff-top">
17        <h1><span class="staff-dot cook"></span>พนักงานครัว</h1>
18        <nav>
19          <a class="active" href="/kitchen">ครัว</a>
20          <a href="/service">บริการ</a>
21          <a href="/cashier">แคชเชียร์</a>
22          <form action="/staff/logout" method="post" class="nav-logout"><button type="submit">ออก
    ⎋</button></form>
23        </nav>
24      </header>
25      <main class="staff-content">
26        <div class="staff-head-row">
27          <h2>ออเดอร์ที่กำลังทำ</h2>
28          <span class="count-pill"><%= orders.length %> ออเดอร์</span>
29        </div>
30        <% if(!orders.length){ %>
31          <div class="staff-empty">
32            <div class="empty-emoji">🍳</div>
33            ยังไม่มีออเดอร์ที่กำลังทำ
34          </div>
35        <% } else { %>
```

คำอธิบายโค้ด

1) staff-top เป็นแถบด้านบนที่มีลิงก์ไปหน้าครัว บริการ แคชเชียร์ และปุ่ม "ออก" ลิงก์ของหน้าปัจจุบันมีคลาส active

2) count-pill แสดงจำนวนออเดอร์ที่กำลังทำ ถ้าไม่มีออเดอร์แสดงข้อความ "ยังไม่มีออเดอร์ที่กำลังทำ"

ตัวอย่างโค้ดที่ 42 views/kitchen.ejs – การ์ดออเดอร์และปุ่มเสร็จสิ้น

views/kitchen.ejs

```
      <div class="order-board">
        <% orders.forEach(order=>{ %>
          <article class="work-card <%= order.time.mins>=15?'urgent':
(order.time.mins>=8?'warn':'') %>">
            <div class="work-title">
              <span class="wt-table">โต๊ะ <%= String(order.table_no).padStart(2,"0") %></span>
              <span class="wt-id">#<%= order.order_id %></span>
            </div>
            <div class="work-time">🕒 <%= order.time.clock %> · <%= order.time.ago %></div>
            <ul>
              <% order.items.forEach(item=>{ let ads=[]; try{ads=JSON.parse(item.addons||"
[]")}catch(e){} %>
                <li>
                  <span class="li-name"><%= item.name %></span>
                  <span class="li-qty">x <%= item.quantity %></span>
                  <% if(ads.length){ %><small><%= ads.map(a=>a.name).join(" · ") %></small><% }
%>
                </li>
              <% }) %>
            </ul>
            <form action="/kitchen/done" method="post">
              <input type="hidden" name="order_id" value="<%= order.order_id %>">
              <button class="orange-btn full" type="submit">เสร็จสิ้น → ส่งเสิร์ฟ</button>
            </form>
          </article>
        <% }) %>
      </div>
    <% } %>
  </main>
</div>
```

คำอธิบายโค้ด

1) คลาสของการ์ดกำหนดจากเวลาที่รอ: ตั้งแต่ 8 นาทีได้คลาส warn (ขอบเหลือง) ตั้งแต่ 15 นาทีได้คลาส urgent (ขอบแดง) ช่วยให้ครัวเห็นออเดอร์ที่ต้องเร่ง

2) การ์ดแสดงหมายเลขโต๊ะ หมายเลขออเดอร์ เวลา รายการอาหาร จำนวน และ Add-on (แปลงจาก JSON)

3) ปุ่ม "เสร็จสิ้น ⟶ ส่งเสิร์ฟ" ส่ง order_id ไปที่ /kitchen/done

ตัวอย่างโค้ดที่ 43 views/kitchen.ejs – สคริปต์อัปเดตแบบเรียลไทม์

views/kitchen.ejs

```
63    <script>
64      // Real-time: อัปเดตทันทีเมื่อมีการเปลี่ยนแปลงออเดอร์/โต๊ะ (ผ่าน Server-Sent Events)
65      (function () {
66        var t, es;
67        function connect() {
68          try {
69            es = new EventSource('/events');
70            es.onmessage = function () { clearTimeout(t); t = setTimeout(function () {
    location.reload(); }, 300); };
71          } catch (e) { }
72        }
73        connect();
74        // ปิดการเชื่อมต่อทันทีก่อนเปลี่ยนหน้า กันเบราว์เซอร์ค้าง (จำกัดจำนวน connection ต่อเว็บ)
75        window.addEventListener('pagehide', function () { try { if (es) es.close(); } catch (e) {
    } });
76        window.addEventListener('beforeunload', function () { try { if (es) es.close(); } catch
    (e) { } });
77        // สำรอง: รีเฟรชทุก 60 วิ เผื่อการเชื่อมต่อ real-time หลุด
78        setInterval(function () { location.reload(); }, 60000);
79      })();
80    </script>
81  </body>
82
83  </html>
```

คำอธิบายโค้ด

1) เป็นสคริปต์ชุดเดียวกับหน้ารายการที่สั่งของลูกค้า: เปิด EventSource('/events') และรีโหลดหน้าเมื่อได้รับสัญญาณ
2) เมื่อลูกค้ายืนยันออเดอร์ เซิร์ฟเวอร์เรียก broadcast() การ์ดออเดอร์ใหม่จึงขึ้นบนจอครัวทันทีโดยไม่ต้องกดรีเฟรช
3) รีโหลดสำรองทุก 60 วินาที ช่วยอัปเดตเวลาที่ผ่านไปและสีเตือนของการ์ดด้วย

1.27 เทมเพลตหน้าพนักงานบริการ (service.ejs)

หน้าพนักงานบริการมีโครงสร้างเหมือนหน้าครัว แต่แสดงออเดอร์สถานะพร้อมเสิร์ฟ

ตัวอย่างโค้ดที่ 44 views/service.ejs – การ์ดอาหารพร้อมเสิร์ฟ

views/service.ejs

```
  <main class="staff-content">
    <div class="staff-head-row">
      <h2>อาหารพร้อมเสิร์ฟ</h2>
      <span class="count-pill"><%= orders.length %> ออเดอร์</span>
    </div>
    <% if(!orders.length){ %>
      <div class="staff-empty">
        <div class="empty-emoji">🍽️</div>
        ยังไม่มีอาหารที่พร้อมเสิร์ฟ
      </div>
    <% } else { %>
      <div class="order-board">
        <% orders.forEach(order=>{ %>
          <article class="work-card ready <%= order.time.mins>=15?'urgent':
(order.time.mins>=8?'warn':'') %>">
            <div class="work-title">
              <span class="wt-table">โต๊ะ <%= String(order.table_no).padStart(2,"0") %></span>
              <span class="wt-id">#<%= order.order_id %></span>
            </div>
            <div class="work-time">🕒 สั่งเมื่อ <%= order.time.clock %> · <%= order.time.ago %>
</div>
            <ul>
              <% order.items.forEach(item=>{ let ads=[]; try{ads=JSON.parse(item.addons||"
[]")}catch(e){} %>
                <li>
                  <span class="li-name"><%= item.name %></span>
                  <span class="li-qty">× <%= item.quantity %></span>
                  <% if(ads.length){ %><small><%= ads.map(a=>a.name).join(" · ") %></small><% }
%>
                </li>
              <% }) %>
            </ul>
            <form action="/service/served" method="post">
              <input type="hidden" name="order_id" value="<%= order.order_id %>">
              <button class="green-btn full" type="submit">เสิร์ฟแล้ว</button>
            </form>
          </article>
        <% }) %>
      </div>
    <% } %>
  </main>
</div>
```

คำอธิบายโค้ด

1) หัวข้อ "อาหารพร้อมเสิร์ฟ" พร้อมจำนวนออเดอร์ ถ้าไม่มีแสดง "ยังไม่มีอาหารที่พร้อมเสิร์ฟ"

2) การ์ดมีคลาส ready (ขอบบนสีเขียว) และใช้เงื่อนไขเวลา warn / urgent แบบเดียวกับครัว

3) ปุ่ม "เสิร์ฟแล้ว" ส่ง order_id ไปที่ /service/served เมื่อเสิร์ฟเสร็จการ์ดจะหายไปจากหน้านี้

4) ส่วนท้ายไฟล์ (บรรทัด 63–83) เป็นสคริปต์เรียลไทม์ชุดเดียวกับหน้าครัว

1.28 เทมเพลตหน้าแคชเชียร์ (cashier.ejs)

หน้าแคชเชียร์แบ่งเป็น 2 แผง คือแผงสถานะโต๊ะทั้ง 16 โต๊ะ และแผงข้อมูลบิลของโต๊ะที่เลือก

ตัวอย่างโค้ดที่ 45 views/cashier.ejs – ข้อความเตือนและปุ่มโหมดเปลี่ยนสถานะ

views/cashier.ejs

```
<main class="cashier-content">
  <% if(typeof notice !== "undefined" && notice){ %>
    <div class="banner-warn">⚠️ <%= notice %></div>
  <% } %>
  <section class="panel">
    <div class="panel-head">
      <h2>สถานะโต๊ะ</h2>
      <div class="status-actions">
        <div class="legend">
          <span><i class="dot-vacant"></i>ว่าง</span>
          <span><i class="dot-occupied"></i>ไม่ว่าง</span>
        </div>
        <form action="/cashier/toggle-mode" method="post">
          <input type="hidden" name="selected" value="<%= selected||'' %>">
          <button type="submit" class="<%= mode?'orange-btn':'blue-btn' %>">
            <%= mode?'● กำลังเปลี่ยนสถานะโต๊ะ':'เปลี่ยนสถานะโต๊ะ' %>
          </button>
        </form>
        <% if(mode && selected){ %>
          <form action="/cashier/confirm-status" method="post">
            <input type="hidden" name="table" value="<%= selected %>">
            <button type="submit" class="green-btn">ยืนยันการเปลี่ยนสถานะ</button>
          </form>
        <% } %>
      </div>
    </div>
```

คำอธิบายโค้ด

1) ถ้ามี notice (เช่น พยายามปิดบิลทั้งที่ยังเสิร์ฟไม่ครบ) จะแสดงแถบเตือนสีส้ม

2) legend อธิบายสีของโต๊ะ: สีขาว = ว่าง, สีฟ้า = ไม่ว่าง

3) ปุ่ม "เปลี่ยนสถานะโต๊ะ" ส่งไป /cashier/toggle-mode เมื่ออยู่ในโหมดนี้ปุ่มจะเปลี่ยนเป็นสีส้มพร้อมข้อความ "● กำลังเปลี่ยนสถานะโต๊ะ"

4) ปุ่ม "ยืนยันการเปลี่ยนสถานะ" แสดงเฉพาะเมื่ออยู่ในโหมดเปลี่ยนสถานะและเลือกโต๊ะแล้ว

ตัวอย่างโค้ดที่ 46 views/cashier.ejs – ตารางโต๊ะ 16 โต๊ะ

views/cashier.ejs

```
51    <div class="tables-grid">
52      <% tables.forEach(t=>{ %>
53        <% if(mode){ %>
54          <form class="table-form" action="/cashier/table-status" method="post">
55            <input type="hidden" name="table" value="<%= t.table_no %>">
56            <button class="table-tile <%= t.status==='ว่าง'?'vacant':'occupied' %> <%=
    selected===t.table_no?'selected':'' %>" type="submit">
57              <b><%= String(t.table_no).padStart(2,'0') %></b>
58              <span><%= t.status %></span>
59            </button>
60          </form>
61        <% } else { %>
62          <a class="table-tile <%= t.status==='ว่าง'?'vacant':'occupied' %> <%=
    selected===t.table_no?'selected':'' %>" href="/cashier?selected=<%= t.table_no %>">
63            <b><%= String(t.table_no).padStart(2,'0') %></b>
64            <span><%= t.status %></span>
65          </a>
66        <% } %>
67      <% }) %>
68    </div>
69    <% if(mode){ %>
70      <div class="hint">โหมดเปลี่ยนสถานะโต๊ะ: กดเลือกโต๊ะแล้วกด “ยืนยันการเปลี่ยนสถานะ” เพื่อสลับ ว่าง / ไม่ว่าง · หากปิดโหมดนี้ การกดโต๊ะจะเป็นการดูข้อมูลบิล</div>
71    <% } %>
72  </section>
```

คำอธิบายโค้ด

1) วนลูปทุกโต๊ะ กำหนดคลาส vacant / occupied ตามสถานะ และคลาส selected ให้โต๊ะที่เลือกอยู่ (กรอบสีแดง)
2) ในโหมดเปลี่ยนสถานะ แต่ละโต๊ะเป็นปุ่มในฟอร์มที่ส่งไป /cashier/table-status ส่วนโหมดปกติเป็นลิงก์ /cashier?selected=เลขโต๊ะ เพื่อดูบิล
3) ข้อความ hint อธิบายวิธีใช้โหมดเปลี่ยนสถานะ

ตัวอย่างโค้ดที่ 47 views/cashier.ejs – แผงข้อมูลบิล

views/cashier.ejs

```
  <% if(selected){ %>
    <section class="panel bill-panel">
      <div class="panel-head">
        <h2>ข้อมูลบิล — โต๊ะ <%= String(selected).padStart(2,"0") %></h2>
      </div>
      <% if(bill && bill.items.length){ %>
        <div class="bill-lines">
          <% bill.orders.forEach(order=>{ if(!order.items.length) return; %>
            <div class="bill-order-head">
              <span>ออเดอร์ #<%= order.order_id %></span>
              <span class="bill-order-time">🕑 <%= order.time.clock %> · <%= order.time.ago
%></span>
            </div>
            <% order.items.forEach(item=>{ let ads=[]; try{ads=JSON.parse(item.addons||"
[]")}catch(e){} %>
              <div class="bill-line">
                <span><%= item.name %> × <%= item.quantity %>
                  <% if(ads.length){ %><small><%= ads.map(a=>a.name).join(" · ") %></small><%
} %>
                </span>
                <b><%= item.total_price %> ฿</b>
              </div>
            <% }) %>
          <% }) %>
        </div>
        <div class="bill-total"><span>ยอดรวม</span><b><%= bill.total %> ฿</b></div>
        <% if(bill.allServed){ %>
          <a class="blue-btn full-btn" href="/payment/<%= selected %>">รับชำระเงิน</a>
        <% } else { %>
          <div class="bill-wait">⏳ ยังปิดบิลไม่ได้ — ต้องเสิร์ฟอาหารครบทุกรายการก่อน</div>
          <button class="blue-btn full-btn" type="button" disabled>รับชำระเงิน</button>
        <% } %>
      <% } else { %>
        <div class="empty bill-empty">ยังไม่มีรายการอาหารของโต๊ะนี้</div>
      <% } %>
    </section>
  <% } %>
</main>
```

คำอธิบายโค้ด

1) แสดงหัวข้อ "ข้อมูลบิล — โต๊ะ XX" เมื่อมีโต๊ะที่เลือก

2) แยกรายการตามออเดอร์ (ออเดอร์ #, เวลา) แล้วแสดงชื่ออาหาร จำนวน Add-on และราคา

3) bill-total แสดงยอดรวมของทั้งบิล

4) ถ้า bill.allServed เป็นจริง ปุ่ม "รับชำระเงิน" จะลิงก์ไป /payment/เลขโต๊ะ ถ้ายังไม่ครบ แสดงข้อความ "⏳ ยังปิดบิลไม่ได้" และปุ่มถูกปิด (disabled)

5) ไฟล์นี้ปิดท้ายด้วยสคริปต์เรียลไทม์ชุดเดียวกับหน้าครัว (บรรทัด 110–127)

1.29 เทมเพลตหน้ารับชำระเงิน (payment.ejs)

หน้ารับชำระเงินรองรับ 2 ช่องทาง คือเงินสด (มีปุ่มกรอกเงินด่วนและคำนวณเงินทอน) และ QR พร้อมเพย์ (วาด QR Code ตามยอดบิล)

ตัวอย่างโค้ดที่ 48 views/payment.ejs – ยอดชำระและปุ่มเลือกช่องทาง

views/payment.ejs

```
  <script src="/public/qrcode.min.js"></script>
</head>

<body class="staff-body">
  <div class="staff-shell">
    <header class="staff-top">
      <h1><span class="staff-dot cash"></span>รับชำระเงิน</h1>
      <nav><a href="/cashier?selected=<%= table %>">← กลับสถานะโต๊ะ</a></nav>
    </header>
    <main class="cashier-content">
      <section class="panel payment-panel">
        <div class="panel-head">
          <h2>โต๊ะ <%= String(table).padStart(2,"0") %></h2>
        </div>

        <div class="pay-amount">
          <span>ยอดที่ต้องชำระ</span>
          <b><%= bill.total %> ฿</b>
        </div>

        <% if(query && query.error){ %>
          <div class="error"><%= query.error %></div>
        <% } %>

        <div class="method-title">เลือกช่องทางชำระเงิน</div>
        <div class="method-buttons">
          <button type="button" class="method active" data-method="เงินสด">💵 เงินสด</button>
          <button type="button" class="method" data-method="QR CODE">▦ QR พร้อมเพย์</button>
        </div>

        <form action="/payment/confirm" method="post" id="paymentForm">
          <input type="hidden" name="table" value="<%= table %>">
          <input type="hidden" name="method" id="method" value="เงินสด">

```

คำอธิบายโค้ด

1) โหลด qrcode.min.js จากโฟลเดอร์ public ของเครื่องเซิร์ฟเวอร์ ไม่ต้องใช้อินเทอร์เน็ตในการวาด QR

2) pay-amount แสดง "ยอดที่ต้องชำระ" ของโต๊ะ ถ้ามี query.error (เช่น ยอดรับเงินไม่พอ) แสดงกล่องข้อความสีแดง

3) ปุ่มเลือกช่องทาง " 💵 เงินสด" และ " ▦ QR พร้อมเพย์" ใช้ data-method เก็บค่าช่องทาง

4) ฟอร์มส่งไป /payment/confirm พร้อม hidden input ของโต๊ะและช่องทางที่เลือก (ค่าเริ่มต้นเงินสด)

ตัวอย่างโค้ดที่ 49 views/payment.ejs – ส่วนเงินสดและส่วน QR พร้อมเพย์

views/payment.ejs

```
        <!-- โหมดเงินสด -->
        <div id="cashPane">
          <div class="payment-row">
            <label for="received">รับเงินมา</label>
            <div class="received-wrap">
              <input id="received" name="received" type="number" min="0" step="1"
                value="<%= query && query.received ? query.received : bill.total %>"> ฿
            </div>
          </div>
          <div class="quick-cash">
            <button type="button" data-cash="<%= bill.total %>">พอดี</button>
            <button type="button" data-add="100">+100</button>
            <button type="button" data-add="500">+500</button>
            <button type="button" data-add="1000">+1000</button>
            <button type="button" data-cash="0">ล้าง</button>
          </div>
          <div class="payment-row change-row">
            <span>เงินทอน</span>
            <strong id="change"><%= Math.max(0,(Number(query&&query.received||bill.total)-
bill.total)) %> ฿</strong>
          </div>
        </div>

        <!-- โหมด QR พร้อมเพย์ -->
        <div id="qrPane" hidden>
          <div class="qr-card">
            <div class="qr-head">
              <span class="qr-ppay">PromptPay</span>
              <span><%= restaurant.name %></span>
            </div>
            <div class="qr-box"><div id="qrcode"></div></div>
            <div class="qr-amount">สแกนเพื่อจ่าย <b><%= bill.total %> บาท</b></div>
            <div class="qr-note">เมื่อลูกค้าโอนสำเร็จแล้ว จึงกดยืนยันการชำระเงิน</div>
          </div>
        </div>

        <button class="blue-btn full-btn big" type="submit" id="confirmBtn">ยืนยันการชำระ
เงิน</button>
      </form>
    </section>
  </main>
</div>
```

คำอธิบายโค้ด

1) cashPane มีช่อง "รับเงินมา" ค่าเริ่มต้นเท่ากับยอดบิล และปุ่มลัด พอดี / +100 / +500 / +1000 / ล้าง

2) change-row แสดงเงินทอน คำนวณครั้งแรกจากฝั่งเซิร์ฟเวอร์และอัปเดตด้วย JavaScript

3) qrPane (ซ่อนไว้ก่อนด้วย hidden) มีกล่อง #qrcode สำหรับวาด QR ยอดเงินที่ต้องสแกน และคำแนะนำให้กดยืนยันเมื่อโอนสำเร็จ

4) ปุ่ม "ยืนยันการชำระเงิน" เป็นปุ่มส่งฟอร์มของทั้งสองช่องทาง

ตัวอย่างโค้ดที่ 50 views/payment.ejs – คำนวณเงินทอนและสร้าง QR Code

views/payment.ejs

```
<script>
  var total = <%= bill.total %>;
  var payload = <%- JSON.stringify(qrPayload || "") %>;
  var received = document.getElementById('received');
  var change = document.getElementById('change');
  var methodInput = document.getElementById('method');
  var cashPane = document.getElementById('cashPane');
  var qrPane = document.getElementById('qrPane');
  var confirmBtn = document.getElementById('confirmBtn');

  function renderChange() {
    change.textContent = Math.max(0, (Number(received.value) || 0) - total) + ' ฿';
  }
  received.addEventListener('input', renderChange);

  document.querySelectorAll('.quick-cash button').forEach(function (b) {
    b.addEventListener('click', function () {
      if (b.dataset.cash !== undefined) received.value = Number(b.dataset.cash);
      else received.value = (Number(received.value) || 0) + Number(b.dataset.add);
      renderChange();
    });
  });

  // สร้าง QR จาก payload พร้อมเพย์
  var qrDone = false;
  function buildQR() {
    if (qrDone || !payload || typeof QRCode === 'undefined') return;
    new QRCode(document.getElementById('qrcode'), {
      text: payload, width: 220, height: 220,
      colorDark: '#111', colorLight: '#fff',
      correctLevel: QRCode.CorrectLevel.M
    });
    qrDone = true;
  }
```

คำอธิบายโค้ด

1) total และ payload รับค่ามาจากเซิร์ฟเวอร์ โดย <%- JSON.stringify(qrPayload) %> แทรก Payload เป็นข้อความ JavaScript ที่ปลอดภัย

2) renderChange() คำนวณเงินทอน = เงินรับ – ยอดบิล (ไม่ต่ำกว่า 0) ทุกครั้งที่พิมพ์ในช่องรับเงิน

3) ปุ่ม data-cash ตั้งค่าเงินรับเป็นค่านั้นโดยตรง ส่วนปุ่ม data-add บวกเพิ่มจากค่าเดิม

4) buildQR() สร้าง QR Code ขนาด 220×220 พิกเซล จาก Payload ด้วยไลบรารี QRCode และวาดเพียงครั้งเดียว (qrDone)

ตัวอย่างโค้ดที่ 51 views/payment.ejs – สลับช่องทางชำระเงิน

views/payment.ejs

```
122      document.querySelectorAll('.method').forEach(function (b) {
123        b.addEventListener('click', function () {
124          document.querySelectorAll('.method').forEach(function (x) {
     x.classList.remove('active'); });
125          b.classList.add('active');
126          var m = b.dataset.method;
127          methodInput.value = m;
128          if (m === 'QR CODE') {
129            cashPane.hidden = true; qrPane.hidden = false;
130            confirmBtn.textContent = 'ยืนยันว่าได้รับเงินแล้ว';
131            buildQR();
132          } else {
133            cashPane.hidden = false; qrPane.hidden = true;
134            confirmBtn.textContent = 'ยืนยันการชำระเงิน';
135          }
136        });
137      });
138
139      // ถ้า reload กลับมาหลัง error ของ QR ให้คงโหมด QR
140      <% if (query && query.method === 'QR CODE') { %>
141        document.querySelector('.method[data-method="QR CODE"]').click();
142      <% } %>
143    </script>
144  </body>
145
146  </html>
```

คำอธิบายโค้ด

1) เมื่อกดปุ่มช่องทาง จะย้ายคลาส active มาที่ปุ่มนั้นและเก็บค่าใน hidden input method
2) ถ้าเลือก QR CODE ซ่อนส่วนเงินสด แสดงส่วน QR เปลี่ยนข้อความปุ่มเป็น "ยืนยันว่าได้รับเงินแล้ว" และเรียก buildQR()
3) ถ้าหน้าถูกโหลดใหม่พร้อม error ในโหมด QR โค้ด EJS ส่วนท้ายจะกดปุ่ม QR ให้อัตโนมัติเพื่อคงโหมดเดิม

1.30 เทมเพลตหน้าชำระเงินสำเร็จ (payment-success.ejs)

หลังยืนยันการชำระเงิน ระบบแสดงใบเสร็จสรุปยอด และสามารถสั่งพิมพ์ได้

ตัวอย่างโค้ดที่ 52 views/payment-success.ejs – ใบเสร็จรับเงิน

views/payment-success.ejs

```
<body class="staff-body">
  <div class="staff-shell">
    <header class="staff-top no-print">
      <h1><span class="staff-dot cash"></span>พนักงานแคชเชียร์</h1>
    </header>
    <main class="cashier-content">
      <section class="panel success-panel">
        <div class="checkmark">✓</div>
        <h2>ชำระเงินสำเร็จ · ปิดบิลแล้ว</h2>

        <div class="receipt" id="receipt">
          <div class="receipt-top">
            <div class="receipt-logo">咲</div>
            <div class="receipt-name"><%= restaurant.name %></div>
            <div class="receipt-sub"><%= restaurant.tagline %></div>
          </div>
          <div class="receipt-row"><span>โต๊ะ</span><b><%= String(table).padStart(2,"0") %></b>
</div>
          <div class="receipt-row"><span>เวลา</span><b><%= paidAt.clock %></b></div>
          <div class="receipt-row"><span>ช่องทาง</span><b><%= method %></b></div>
          <div class="receipt-divider"></div>
          <div class="receipt-row big"><span>ยอดรวม</span><b><%= total %> ฿</b></div>
          <% if(method !== 'QR CODE'){ %>
            <div class="receipt-row"><span>รับเงิน</span><b><%= received %> ฿</b></div>
            <div class="receipt-row"><span>เงินทอน</span><b><%= change %> ฿</b></div>
          <% } %>
          <div class="receipt-divider"></div>
          <div class="receipt-thanks">ขอบคุณที่ใช้บริการ 🙏</div>
        </div>

        <div class="success-status no-print">
          <span>สถานะโต๊ะ <%= String(table).padStart(2,"0") %> ตอนนี้</span><b>ว่าง</b>
        </div>
        <div class="actions no-print">
          <a class="blue-btn full-btn" href="/cashier">กลับหน้าสถานะโต๊ะ</a>
          <button class="secondary-btn full-btn" type="button" onclick="window.print()">🖨️ พิมพ์
ใบเสร็จ</button>
        </div>
      </section>
    </main>
  </div>
</body>

</html>
```

คำอธิบายโค้ด

1) แสดงเครื่องหมายถูกและหัวข้อ "ชำระเงินสำเร็จ · ปิดบิลแล้ว"

2) กล่อง receipt แสดงชื่อร้าน โต๊ะ เวลา ช่องทาง และยอดรวม ถ้าเป็นเงินสดจะแสดงเงินรับและเงินทอนเพิ่ม

3) success-status ยืนยันว่าโต๊ะกลับเป็นสถานะ "ว่าง" แล้ว

4) ปุ่ม " พิมพ์ใบเสร็จ" เรียก window.print() ส่วนที่มีคลาส no-print (แถบเมนู ปุ่มต่าง ๆ) จะถูกซ่อนตอนพิมพ์ ให้เหลือเฉพาะใบเสร็จ

1.31 การตกแต่งหน้าเว็บ (style.css)

ไฟล์ style.css กำหนดรูปแบบของทุกหน้าในระบบ โดยใช้โทนสีแดงเข้มและสีทองตามธีมร้านอาหารญี่ปุ่น แบ่งเป็นกลุ่มตามหน้าที่ใช้งาน

ตัวอย่างโค้ดที่ 53 public/style.css – ตัวแปรสีและสไตล์พื้นฐาน (บรรทัด 1–37)

public/style.css

```
:root {
  --red: #a21d22;
  --red-deep: #7c1216;
  --red-dark: #5e0f12;
  --gold: #e3b34f;
  --gold-soft: #f0d08a;
  --cream: #fff8ef;
  --paper: #fffdf9;
  --ink: #241612;
  --muted: #8a7a72;
  --line: #ece2d6;
  --bg-staff: #f3ece2;
  --ok: #2e9b4f;
  --ok-d: #237c3e;
  --warn: #e8822a;
  --info: #2d5b96;
  --danger: #d23b3b;
  --shadow: 0 10px 30px rgba(60, 20, 10, .12);
  --shadow-sm: 0 4px 14px rgba(60, 20, 10, .10);
  --r: 16px;
}

* { box-sizing: border-box }

html, body {
  margin: 0;
  padding: 0;
  font-family: "Noto Sans Thai", system-ui, "Segoe UI", sans-serif;
  color: var(--ink);
  background: var(--red-dark);
  -webkit-font-smoothing: antialiased;
}

body { min-height: 100vh }
a { text-decoration: none; color: inherit }
button, input { font: inherit }
h1, h2, .brand-text b { font-family: "Noto Serif Thai", "Noto Sans Thai", serif }
```

คำอธิบายโค้ด

1) ใน :root ประกาศตัวแปร CSS (Custom Properties) เช่น --red, --gold, --paper ใช้ซ้ำทั้งไฟล์ เปลี่ยนสีทั้งระบบได้จากจุดเดียว

2) box-sizing: border-box ทำให้การคำนวณขนาดกล่องรวม padding และ border

3) กำหนดฟอนต์หลัก Noto Sans Thai และใช้ Noto Serif Thai กับหัวข้อ

ตัวอย่างโค้ดที่ 54 public/style.css – กรอบหน้าลูกค้า แถบด้านบน และโลโก้ (บรรทัด 39–80)

public/style.css

```
/* ==================== CUSTOMER SHELL ==================== */
.app-shell {
  width: min(1180px, 94%);
  margin: 26px auto;
  border-radius: 26px;
  overflow: hidden;
  background:
    radial-gradient(1200px 400px at 85% -10%, rgba(227, 179, 79, .18), transparent 60%),
    linear-gradient(160deg, var(--red) 0%, var(--red-deep) 55%, var(--red-dark) 100%);
  min-height: calc(100vh - 52px);
  box-shadow: var(--shadow);
  border: 1px solid rgba(0, 0, 0, .25);
}

.topbar {
  min-height: 70px;
  display: flex;
  align-items: center;
  padding: 0 26px;
  color: #fff;
  background: rgba(0, 0, 0, .14);
  border-bottom: 1px solid rgba(255, 255, 255, .12);
  position: sticky;
  top: 0;
  z-index: 20;
  backdrop-filter: blur(6px);
}

.brand { display: flex; align-items: center; gap: 12px }
.brand-mark {
  width: 42px; height: 42px;
  display: grid; place-items: center;
  border-radius: 12px;
  background: linear-gradient(145deg, var(--gold-soft), var(--gold));
  color: var(--red-dark);
  font-family: "Noto Serif Thai", serif;
  font-size: 24px; font-weight: 700;
  box-shadow: inset 0 0 0 1px rgba(255, 255, 255, .4);
}
.brand-text { display: flex; flex-direction: column; line-height: 1.15 }
.brand-text b { font-size: 18px; letter-spacing: .3px }
.brand-text small { font-size: 11px; color: var(--gold-soft); letter-spacing: 2px; text-
transform: uppercase }
```

คำอธิบายโค้ด

1) .app-shell เป็นกรอบหลักของหน้าลูกค้า ใช้พื้นหลังไล่สีแดง (linear-gradient) ซ้อนแสงสีทอง (radial-gradient)

2) .topbar ใช้ position: sticky ติดด้านบนเมื่อเลื่อนหน้า และ backdrop-filter เบลอพื้นหลัง

3) .brand-mark คือกล่องโลโก้ 咲 สีทองมุมโค้ง

ตัวอย่างโค้ดที่ 55 public/style.css – เมนูนำทาง ป้ายโต๊ะ และหัวเรื่อง (บรรทัด 82–114)

public/style.css

```
.top-links { margin-left: auto; display: flex; gap: 10px; align-items: center; font-size: 14px
}
.top-links a {
  padding: 8px 14px; border-radius: 999px; color: #fff;
  transition: background .15s; position: relative;
}
.top-links a:hover { background: rgba(255, 255, 255, .14) }
.top-links a.active { background: rgba(255, 255, 255, .20); font-weight: 600 }
.cart-link { display: inline-flex; align-items: center; gap: 6px }
.cart-badge {
  background: var(--gold); color: var(--red-dark);
  font-size: 12px; font-weight: 700;
  min-width: 20px; height: 20px; padding: 0 5px;
  border-radius: 999px; display: inline-grid; place-items: center;
}
.table-pill {
  border: 1px solid rgba(255, 255, 255, .55);
  border-radius: 999px; padding: 7px 14px; font-size: 13px;
  background: rgba(255, 255, 255, .08);
}

.content { padding: 28px 34px 60px; color: #fff }
.content.narrow { max-width: 920px; margin: auto }

.hero-row { display: flex; justify-content: space-between; align-items: flex-end; gap: 15px }
.hero-row h1 { margin: 0 0 6px; font-size: 30px }
.hero-row p { margin: 0; color: rgba(255, 255, 255, .82) }
.hero-row strong { color: var(--gold-soft) }

.back-link {
  display: inline-block; color: rgba(255, 255, 255, .85);
  margin-bottom: 16px; font-size: 14px; padding: 6px 0;
}
.back-link:hover { color: #fff }
```

คำอธิบายโค้ด

1) .top-links จัดลิงก์ด้วย Flexbox ลิงก์ที่ active มีพื้นหลังโปร่งแสง

2) .cart-badge คือวงกลมสีทองแสดงจำนวนชิ้นในตะกร้า และ .table-pill คือป้ายหมายเลขโต๊ะ

3) .content.narrow จำกัดความกว้างเนื้อหาของหน้าตะกร้า/Add-on ไม่เกิน 920px

ตัวอย่างโค้ดที่ 56 public/style.css – ปุ่มหมวดหมู่และหัวข้อหมวด (บรรทัด 116–138)

public/style.css

```
116 /* category quick nav */
117 .cat-nav {
118   display: flex; gap: 10px; flex-wrap: wrap;
119   margin: 22px 0 6px; position: sticky; top: 70px; z-index: 10;
120   padding: 10px 0;
121 }
122 .cat-nav a {
123   background: rgba(255, 255, 255, .12); color: #fff;
124   border: 1px solid rgba(255, 255, 255, .22);
125   padding: 8px 16px; border-radius: 999px; font-size: 14px;
126   transition: all .15s;
127 }
128 .cat-nav a:hover { background: var(--gold); color: var(--red-dark); border-color: var(--gold)
    }
129
130 .category { margin: 16px 0 34px; scroll-margin-top: 130px }
131 .category h2 {
132   font-size: 21px; margin: 0 0 16px; color: #fff;
133   display: inline-flex; align-items: center; gap: 10px;
134 }
135 .category h2::before {
136   content: ""; width: 5px; height: 22px; border-radius: 3px;
137   background: linear-gradient(var(--gold-soft), var(--gold));
138 }
```

คำอธิบายโค้ด

1) .cat-nav เป็นแถบปุ่มหมวดหมู่ที่ติดอยู่ใต้แถบด้านบน (sticky, top: 70px) เมื่อชี้จะเปลี่ยนเป็นสีทอง

2) scroll-margin-top เว้นระยะเมื่อกดเลื่อนไปยังหมวด ไม่ให้หัวข้อถูกแถบด้านบนบัง

3) .category h2::before สร้างแถบสีทองหน้าชื่อหมวดโดยไม่ต้องเพิ่มแท็ก HTML

ตัวอย่างโค้ดที่ 57 public/style.css – การ์ดเมนูอาหาร (บรรทัด 140–171)

public/style.css

```
.menu-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 18px }
.menu-card {
  background: var(--paper); color: var(--ink);
  border-radius: var(--r); overflow: hidden;
  box-shadow: var(--shadow-sm);
  transition: transform .16s, box-shadow .16s;
  display: flex; flex-direction: column;
}
.menu-card:hover { transform: translateY(-4px); box-shadow: 0 16px 34px rgba(30, 10, 5, .28) }
.menu-thumb { position: relative; overflow: hidden }
.menu-thumb img {
  display: block; width: 100%; height: 150px; object-fit: cover;
  background: #eee; transition: transform .35s;
}
.menu-card:hover .menu-thumb img { transform: scale(1.06) }
.menu-info {
  display: flex; align-items: center; justify-content: space-between;
  padding: 12px 12px 12px 14px; gap: 10px;
}
.menu-meta { min-width: 0 }
.menu-meta b {
  display: block; white-space: nowrap; overflow: hidden; text-overflow: ellipsis;
  font-size: 15px; font-weight: 600;
}
.menu-meta span { display: block; color: var(--red); font-size: 14px; font-weight: 700;
margin-top: 3px }
.add-button {
  width: 40px; height: 40px; border-radius: 12px; flex: none;
  background: linear-gradient(145deg, var(--ok), var(--ok-d)); color: #fff;
  display: grid; place-items: center; font-size: 26px; line-height: 0;
  box-shadow: 0 4px 10px rgba(35, 124, 62, .4); transition: transform .12s;
}
.add-button:hover { transform: scale(1.08) }
```

คำอธิบายโค้ด

1) .menu-grid ใช้ CSS Grid แสดง 4 คอลัมน์

2) .menu-card เมื่อชี้จะลอยขึ้น (translateY) และรูปซูมขยายเล็กน้อย (scale 1.06)

3) ชื่ออาหารยาวจะถูกตัดด้วย text-overflow: ellipsis ส่วน .add-button คือปุ่ม + สีเขียว

ตัวอย่างโค้ดที่ 58 public/style.css – ปุ่มรูปแบบต่าง ๆ (บรรทัด 173–192)

public/style.css

```
/* ==================== BUTTONS ==================== */
.outline-btn, .primary-btn, .secondary-btn, .blue-btn, .orange-btn, .green-btn, .danger-btn {
  border: 0; border-radius: 12px; padding: 12px 20px; cursor: pointer;
  display: inline-flex; justify-content: center; align-items: center; gap: 6px;
  font-weight: 600; transition: transform .12s, filter .15s; font-size: 15px;
}
.outline-btn:active, .primary-btn:active, .secondary-btn:active,
.blue-btn:active, .orange-btn:active, .green-btn:active { transform: translateY(1px) }
.primary-btn:hover, .blue-btn:hover, .orange-btn:hover, .green-btn:hover { filter: brightness(1.06) }
.outline-btn { background: #fff; border: 1.5px solid var(--ink); color: var(--ink) }
.primary-btn { background: linear-gradient(145deg, var(--ok), var(--ok-d)); color: #fff; box-shadow: 0 6px 16px rgba(35, 124, 62, .3) }
.secondary-btn { background: #efe7db; color: var(--ink) }
.blue-btn { background: linear-gradient(145deg, #3a6fb0, var(--info)); color: #fff; box-shadow: 0 6px 16px rgba(45, 91, 150, .3) }
.orange-btn { background: linear-gradient(145deg, #f79245, var(--warn)); color: #fff }
.green-btn { background: linear-gradient(145deg, var(--ok), var(--ok-d)); color: #fff }
.danger-btn { background: #fff; color: var(--danger); border: 1.5px solid #f0c7c7; padding: 8px 14px; font-size: 14px; border-radius: 10px }
.danger-btn:hover { background: var(--danger); color: #fff; border-color: var(--danger) }
.full { width: 100% }
.sm { padding: 8px 14px; font-size: 14px }
.big { padding: 16px 20px; font-size: 17px }
```

คำอธิบายโค้ด

1) กำหนดรูปแบบร่วมของปุ่มทุกชนิด (มุมโค้ง ตัวหนา) แล้วแยกสีตามหน้าที่: primary/green = ยืนยัน, blue = ชำระเงิน, orange = ครัว, danger = ลบ
2) เมื่อกดปุ่มจะเลื่อนลง 1px (:active) ให้รู้สึกว่ากดได้จริง
3) คลาสเสริม .full .sm .big ปรับความกว้างและขนาดปุ่ม

ตัวอย่างโค้ดที่ 59 public/style.css – หน้าเลือก Add-on: ข้อมูลเมนูและตัวเลือก (บรรทัด 194–223)

public/style.css

```
/* ==================== PRODUCT / ADDON ==================== */
.product-head {
  display: flex; align-items: center; gap: 22px; margin-bottom: 22px;
  background: rgba(255, 255, 255, .08); padding: 16px; border-radius: var(--r);
  border: 1px solid rgba(255, 255, 255, .12);
}
.product-head img { width: 180px; height: 130px; object-fit: cover; border-radius: 12px; flex: none }
.product-cat {
  display: inline-block; font-size: 12px; color: var(--red-dark);
  background: var(--gold-soft); padding: 3px 10px; border-radius: 999px; margin-bottom: 6px;
}
.product-head h1 { margin: 0 0 6px; color: #fff; font-size: 24px }
.price { font-size: 18px; color: var(--gold-soft); font-weight: 700 }

.option-box { background: var(--paper); color: var(--ink); border-radius: var(--r); padding: 16px 20px; margin-bottom: 14px; box-shadow: var(--shadow-sm) }
.option-box h2 { font-size: 17px; color: var(--red); margin: 0 0 10px; display: flex; align-items: center; gap: 10px }
.addon-rule { font-size: 11px; font-weight: 600; padding: 3px 9px; border-radius: 999px }
.addon-rule.multi { background: #e7f3ff; color: #1d5fa8 }
.addon-rule.single { background: #fdebe5; color: #c2491f }
.options { display: flex; flex-direction: column }
.option-row {
  display: flex; align-items: center; gap: 12px; padding: 13px 10px; cursor: pointer;
  border-radius: 10px; border: 1px solid transparent; transition: background .12s;
}
.option-row:hover { background: #faf4ea }
.option-row input { width: 20px; height: 20px; accent-color: var(--red); flex: none }
.option-row:has(input:checked) { background: #fcf1e3; border-color: var(--gold-soft) }
.option-name { flex: 1 }
.option-price { color: var(--red); font-weight: 600; font-size: 14px }
.option-price.free { color: var(--muted); font-weight: 500 }
```

คำอธิบายโค้ด

1) .product-head แสดงรูปและข้อมูลเมนูในแนวนอน

2) .addon-rule.multi (ฟ้า) และ .single (ส้ม) แยกสีป้ายกฎการเลือก

3) .option-row:has(input:checked) ไฮไลต์แถวที่ถูกเลือกโดยไม่ต้องใช้ JavaScript

4) .option-price.free แสดงคำว่า "ฟรี" เป็นสีเทา

ตัวอย่างโค้ดที่ 60 public/style.css – หน้าเลือก Add-on: จำนวนและแถบด้านล่าง (บรรทัด 225–239)

public/style.css

```
.quantity-row { display: flex; align-items: center; justify-content: space-between; font-weight: 600 }
.qty-stepper { display: flex; align-items: center; gap: 0; border: 1.5px solid var(--line); border-radius: 12px; overflow: hidden }
.qty-btn { width: 44px; height: 44px; border: 0; background: #f4ece0; color: var(--red); font-size: 22px; cursor: pointer }
.qty-btn:hover { background: var(--gold-soft) }
.qty-stepper input { width: 64px; height: 44px; border: 0; text-align: center; font-weight: 700; font-size: 16px; -moz-appearance: textfield }
.qty-stepper input::-webkit-outer-spin-button, .qty-stepper input::-webkit-inner-spin-button { -webkit-appearance: none; margin: 0 }

.addon-actionbar {
  position: sticky; bottom: 0; display: flex; align-items: center; justify-content: space-between;
  gap: 14px; margin-top: 18px; padding: 14px 16px; border-radius: var(--r);
  background: var(--paper); box-shadow: 0 -6px 20px rgba(30, 10, 5, .2); color: var(--ink);
}
.addon-sum { font-size: 15px }
.addon-sum strong { font-size: 22px; color: var(--red) }
.addon-actionbar .primary-btn { padding: 14px 26px; font-size: 16px }
```

คำอธิบายโค้ด

1) .qty-stepper รวมปุ่ม – / + กับช่องตัวเลขเป็นกล่องเดียว และซ่อนลูกศรเพิ่มลดของช่อง number

2) .addon-actionbar ติดด้านล่างจอ (sticky, bottom: 0) ให้กดเพิ่มลงตะกร้าได้ตลอด

ตัวอย่างโค้ดที่ 61 public/style.css – หน้าตะกร้า (บรรทัด 241–268)

public/style.css

```
241 /* ==================== CART ==================== */
242 .section-title { display: flex; justify-content: space-between; align-items: center; gap: 14px
    }
243 .section-title h1 { margin: 0; color: #fff; font-size: 26px }
244 .pill-count { background: rgba(255, 255, 255, .16); color: #fff; padding: 6px 14px; border-
    radius: 999px; font-size: 14px }
245
246 .cart-list { margin-top: 18px; display: flex; flex-direction: column; gap: 12px }
247 .cart-card { display: flex; align-items: center; gap: 16px; background: var(--paper); color:
    var(--ink); padding: 14px; border-radius: var(--r); box-shadow: var(--shadow-sm) }
248 .cart-card img { width: 108px; height: 80px; object-fit: cover; border-radius: 12px; flex:
    none }
249 .cart-main { flex: 1; min-width: 0 }
250 .cart-main h2 { font-size: 17px; margin: 0 0 6px; font-family: "Noto Sans Thai", sans-serif }
251 .cart-qty { color: var(--red); font-weight: 700 }
252 .cart-base { color: var(--muted); font-size: 13px }
253 .addons { margin-top: 8px; display: flex; flex-wrap: wrap; gap: 6px }
254 .addon-chip { background: #f3ece0; color: #6b5a4f; font-size: 12px; padding: 4px 10px; border-
    radius: 999px }
255 .cart-price { display: flex; flex-direction: column; align-items: flex-end; gap: 10px }
256 .cart-price b { font-size: 18px; color: var(--red) }
257
258 .total-box { background: var(--paper); color: var(--ink); border-radius: var(--r); padding:
    20px; margin-top: 18px; box-shadow: var(--shadow-sm) }
259 .total-row { display: flex; justify-content: space-between; align-items: center; font-size:
    16px }
260 .total-row strong { font-size: 26px; color: var(--red) }
261 .total-box .actions { display: flex; gap: 12px; margin-top: 18px }
262 .total-box .actions form { flex: 1; display: flex }
263 .total-box .actions .secondary-btn { flex: 1 }
264 .total-box .actions .primary-btn { flex: 2 }
265
266 .empty { background: var(--paper); color: var(--ink); border-radius: var(--r); padding: 44px;
    text-align: center; margin-top: 18px }
267 .empty-emoji { font-size: 48px; margin-bottom: 10px }
268 .empty p { color: var(--muted); margin: 0 0 18px }
```

คำอธิบายโค้ด

1) .cart-card จัดรูป ข้อมูล และราคาเป็นแถวเดียวด้วย Flexbox

2) .addon-chip แสดงตัวเลือกเสริมเป็นป้ายมุมมน

3) .total-box แบ่งปุ่ม ล้างตะกร้า : ยืนยัน ในสัดส่วน 1 : 2 ด้วย flex

4) .empty คือกล่องข้อความเมื่อไม่มีข้อมูล

ตัวอย่างโค้ดที่ 62 public/style.css – หน้ารายการที่สั่ง (บรรทัด 270–288)

public/style.css

```
/* ==================== ORDERS (customer) ==================== */
.banner-success {
  background: linear-gradient(145deg, #eafaef, #d7f3e0); color: var(--ok-d);
  border: 1px solid #b6e6c5; border-radius: 14px; padding: 14px 18px; margin-bottom: 18px;
  font-weight: 600;
}
.order-history { background: var(--paper); color: var(--ink); border-radius: var(--r); padding: 18px; margin: 14px 0; box-shadow: var(--shadow-sm) }
.order-head { display: flex; justify-content: space-between; gap: 10px; align-items: center; border-bottom: 1px solid var(--line); padding-bottom: 12px }
.order-id { display: flex; flex-direction: column; gap: 3px }
.order-time { font-size: 12px; color: var(--muted) }
.order-line { display: flex; justify-content: space-between; padding: 10px 0; gap: 10px }
.order-line small, .work-card small { display: block; color: var(--muted); margin-top: 3px; font-size: 12px }
.order-total { text-align: right; border-top: 1px solid var(--line); padding-top: 12px; font-weight: 700; color: var(--red) }

.status { padding: 6px 13px; border-radius: 999px; font-size: 12px; font-weight: 600; white-space: nowrap }
.status-cooking { background: #ffe6cc; color: #b35c0f }
.status-ready { background: #fff2c2; color: #9a7700 }
.status-served { background: #d6f0d9; color: var(--ok-d) }
.status-paid { background: #e6e1da; color: #6b6158 }
```

คำอธิบายโค้ด

1) .banner-success แถบแจ้งส่งออเดอร์สำเร็จสีเขียว

2) .order-history การ์ดออเดอร์แต่ละรายการ

3) .status-cooking / ready / served / paid กำหนดสีป้ายสถานะ ส้ม / เหลือง / เขียว / เทา

ตัวอย่างโค้ดที่ 63 public/style.css – ตะกร้าลอยและข้อความแจ้งเตือน (Toast) (บรรทัด 290–320)

public/style.css

```
/* ==================== FLOATING CART ==================== */
.floating-cart {
  position: fixed; left: 50%; transform: translateX(-50%); bottom: 22px; z-index: 50;
  display: flex; align-items: center; gap: 14px;
  background: linear-gradient(145deg, var(--ok), var(--ok-d)); color: #fff;
  padding: 12px 16px 12px 18px; border-radius: 999px;
  box-shadow: 0 12px 28px rgba(20, 80, 40, .45); font-weight: 600;
  animation: pop .25s ease;
}
.floating-cart:hover { filter: brightness(1.06) }
@keyframes pop { from { transform: translateX(-50%) translateY(20px); opacity: 0 } to {
transform: translateX(-50%) translateY(0); opacity: 1 } }
.fc-icon { position: relative; font-size: 20px }
.fc-count {
  position: absolute; top: -8px; right: -10px; background: var(--gold); color: var(--red-
dark);
  font-size: 11px; min-width: 18px; height: 18px; padding: 0 4px; border-radius: 999px;
  display: grid; place-items: center; font-weight: 700;
}
.fc-text { font-size: 15px }
.fc-total { background: rgba(255, 255, 255, .22); padding: 6px 12px; border-radius: 999px;
font-weight: 700 }

/* ==================== TOAST ==================== */
.toast {
  position: fixed; top: 24px; left: 50%; transform: translateX(-50%); z-index: 60;
  background: #1f2a1f; color: #fff; padding: 14px 22px; border-radius: 999px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, .3); font-size: 15px;
  border: 1px solid rgba(255, 255, 255, .15);
  animation: toastIn .3s ease; transition: opacity .4s, transform .4s;
}
.toast b { color: var(--gold-soft) }
.toast.hide { opacity: 0; transform: translateX(-50%) translateY(-14px) }
@keyframes toastIn { from { opacity: 0; transform: translateX(-50%) translateY(-14px) } to {
opacity: 1; transform: translateX(-50%) translateY(0) } }
```

คำอธิบายโค้ด

1) .floating-cart ใช้ position: fixed ลอยกลางล่างจอ พร้อมแอนิเมชัน pop (@keyframes)

2) .fc-count วงกลมจำนวนชิ้นที่มุมไอคอนตะกร้า

3) .toast คือข้อความแจ้งเตือนด้านบน เมื่อเพิ่มคลาส .hide จะค่อย ๆ จางหายไป

ตัวอย่างโค้ดที่ 64 public/style.css – แถบด้านบนของหน้าพนักงาน (บรรทัด 322–343)

public/style.css

```
/* ==================== STAFF SHARED ==================== */
.staff-body { background: var(--bg-staff); display: flow-root }
.staff-shell { width: min(1200px, 94%); margin: 24px auto 60px }
.staff-top {
  display: flex; align-items: center; justify-content: space-between; gap: 14px;
  background: linear-gradient(160deg, var(--red), var(--red-deep)); color: #fff;
  padding: 16px 22px; border-radius: 18px; box-shadow: var(--shadow-sm);
}
.staff-top h1 { margin: 0; font-size: 21px; display: flex; align-items: center; gap: 10px }
.staff-dot { width: 12px; height: 12px; border-radius: 50%; display: inline-block; box-shadow: 0 0 0 3px rgba(255, 255, 255, .25) }
.staff-dot.cook { background: var(--warn) }
.staff-dot.serve { background: #7fe08a }
.staff-dot.cash { background: var(--gold) }
.staff-top nav { display: flex; gap: 8px; font-size: 14px }
.staff-top nav a { padding: 8px 14px; border-radius: 999px; color: #fff; transition: background .15s }
.staff-top nav a:hover { background: rgba(255, 255, 255, .16) }
.staff-top nav a.active { background: #fff; color: var(--red); font-weight: 700 }

.staff-content { padding: 20px 2px }
.staff-head-row { display: flex; align-items: center; gap: 12px; margin: 4px 0 16px }
.staff-content h2 { font-size: 20px; margin: 0; color: var(--ink) }
.count-pill { background: var(--red); color: #fff; padding: 5px 14px; border-radius: 999px; font-size: 14px; font-weight: 600 }
```

คำอธิบายโค้ด

1) .staff-body เปลี่ยนพื้นหลังหน้าพนักงานเป็นสีครีม
2) .staff-top คือแถบหัวสีแดงของหน้าพนักงาน มีจุดสีบอกแผนก (.staff-dot): ครัวสีส้ม บริการสีเขียว แคชเชียร์สีทอง
3) .count-pill ป้ายจำนวนออเดอร์สีแดง

ตัวอย่างโค้ดที่ 65 public/style.css – การ์ดออเดอร์ของครัวและบริการ (KDS) (บรรทัด 345–368)

public/style.css

```
/* order board (KDS) */
.order-board { display: grid; grid-template-columns: repeat(4, 1fr); gap: 14px }
.work-card {
  border-radius: var(--r); padding: 14px; background: #fff;
  box-shadow: var(--shadow-sm); border-top: 5px solid var(--warn);
  display: flex; flex-direction: column;
}
.work-card.ready { border-top-color: var(--ok) }
.work-card.warn { border-top-color: #e8a02a; box-shadow: 0 0 0 1px #f3d08a inset, var(--shadow-sm) }
.work-card.urgent { border-top-color: var(--danger); box-shadow: 0 0 0 2px rgba(210, 59, 59, .3) }
.work-title { display: flex; align-items: center; justify-content: space-between; border-bottom: 1px solid var(--line); padding-bottom: 8px }
.wt-table { font-weight: 700; font-size: 17px }
.wt-id { color: var(--muted); font-size: 13px }
.work-time { font-size: 12px; color: var(--muted); margin: 8px 0 }
.work-card.urgent .work-time { color: var(--danger); font-weight: 700 }
.work-card ul { list-style: none; padding: 0; margin: 2px 0 14px; flex: 1 }
.work-card li { padding: 7px 0; border-bottom: 1px dashed var(--line); display: flex; flex-wrap: wrap; align-items: baseline; gap: 6px }
.work-card li:last-child { border-bottom: 0 }
.li-name { font-weight: 600 }
.li-qty { color: var(--red); font-weight: 700; margin-left: auto }
.work-card small { width: 100% }

.staff-empty { text-align: center; padding: 56px 20px; color: var(--muted); background: #fff; border-radius: var(--r); box-shadow: var(--shadow-sm) }
.staff-empty .empty-emoji { font-size: 44px; margin-bottom: 8px }
```

คำอธิบายโค้ด

1) .order-board แสดงการ์ดออเดอร์ 4 คอลัมน์

2) .work-card ขอบบนสีส้ม, .ready สีเขียว, .warn สีเหลือง และ .urgent สีแดงพร้อมเวลาสีแดง

3) .li-qty ดันจำนวนไปชิดขวาด้วย margin-left: auto

ตัวอย่างโค้ดที่ 66 public/style.css – หน้าแคชเชียร์: ตารางโต๊ะ (บรรทัด 370–396)

public/style.css

```
/* ==================== CASHIER ==================== */
.cashier-content { max-width: 1000px; margin: 22px auto 60px }
.panel { border-radius: var(--r); overflow: hidden; background: #fff; margin-bottom: 24px; box-shadow: var(--shadow-sm) }
.panel-head { display: flex; justify-content: space-between; align-items: center; gap: 12px; border-bottom: 1px solid var(--line); padding: 16px 18px; flex-wrap: wrap }
.panel-head h2 { font-size: 19px; margin: 0 }

.status-actions { display: flex; align-items: center; gap: 10px; flex-wrap: wrap }
.legend { display: flex; gap: 12px; font-size: 13px; color: var(--muted); margin-right: 4px }
.legend i { display: inline-block; width: 12px; height: 12px; border-radius: 4px; margin-right: 5px; vertical-align: -1px }
.dot-vacant { background: #fff; border: 1.5px solid #cbbfb0 }
.dot-occupied { background: #bcd3ea; border: 1.5px solid #7ba2cd }

.tables-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 10px; padding: 16px }
.table-tile {
  border: 2px solid var(--line); border-radius: 14px; min-height: 76px;
  display: flex; flex-direction: column; align-items: center; justify-content: center; gap: 2px;
  font-size: 13px; background: #fff; cursor: pointer; transition: transform .12s, box-shadow .12s; width: 100%;
}
.table-tile b { font-size: 20px }
.table-tile span { color: var(--muted); font-size: 12px }
.table-tile:hover { transform: translateY(-2px); box-shadow: var(--shadow-sm) }
.table-tile.occupied { background: #d7e6f4; border-color: #9cbfe0 }
.table-tile.occupied span { color: #2f5d8c }
.table-tile.vacant { background: #fff }
.table-tile.selected { outline: 3px solid var(--red); outline-offset: -3px; border-color: var(--red) }
.table-form { display: flex; min-width: 0; margin: 0 }
.table-form .table-tile { height: 100% }
```

คำอธิบายโค้ด

1) .panel / .panel-head กล่องสีขาวพร้อมหัวข้อของแต่ละแผง

2) .tables-grid แสดงโต๊ะ 4 คอลัมน์ .table-tile.occupied เป็นสีฟ้า และ .vacant เป็นสีขาว

3) .table-tile.selected ใช้ outline สีแดงแสดงโต๊ะที่เลือก

ตัวอย่างโค้ดที่ 67 public/style.css – หน้าแคชเชียร์: รายการบิล (บรรทัด 398–408)

public/style.css

```
398 .bill-lines { padding: 10px 18px }
399 .bill-order-head { display: flex; justify-content: space-between; align-items: baseline;
    margin: 12px 0 4px; color: var(--red); font-weight: 700; font-size: 14px; border-bottom: 1px
    dashed var(--line); padding-bottom: 4px }
400 .bill-order-head:first-child { margin-top: 0 }
401 .bill-order-time { color: var(--muted); font-weight: 500; font-size: 12px }
402 .bill-line { display: flex; justify-content: space-between; padding: 7px 0; gap: 10px }
403 .bill-line small { display: block; margin-left: 8px; color: var(--muted); font-size: 12px }
404 .bill-total { border-top: 2px solid var(--ink); margin: 6px 18px 16px; padding-top: 12px;
    display: flex; justify-content: space-between; font-size: 18px }
405 .bill-total b { color: var(--red) }
406 .full-btn { width: calc(100% - 36px); margin: 0 18px 18px }
407 .hint { font-size: 13px; color: var(--muted); padding: 0 18px 16px; line-height: 1.5 }
408 .bill-empty { margin: 18px; box-shadow: none; background: #faf6f0 }
```

คำอธิบายโค้ด

1) .bill-order-head หัวของแต่ละออเดอร์ในบิล คั่นด้วยเส้นประ

2) .bill-total เส้นทึบด้านบนและยอดรวมสีแดง

3) .full-btn ปุ่มเต็มความกว้างของแผง

ตัวอย่างโค้ดที่ 68 public/style.css – หน้ารับชำระเงิน: ยอดเงินและเงินสด (บรรทัด 410–432)

public/style.css

```
/* ==================== PAYMENT ==================== */
.payment-panel { max-width: 760px; margin: auto }
.pay-amount {
  display: flex; justify-content: space-between; align-items: center;
  margin: 18px; padding: 18px 22px; border-radius: 14px;
  background: linear-gradient(145deg, #fcf2e0, #f8e7c6); border: 1px solid var(--gold-soft);
}
.pay-amount span { color: #7a5a1e; font-weight: 600 }
.pay-amount b { font-size: 30px; color: var(--red-deep) }

.method-title { padding: 6px 18px 10px; font-weight: 600 }
.method-buttons { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; padding: 0 18px 8px }
.method { border: 2px solid var(--line); border-radius: 14px; background: #fff; padding: 16px; cursor: pointer; font-weight: 600; font-size: 15px; transition: all .15s }
.method:hover { border-color: var(--gold) }
.method.active { background: var(--info); color: #fff; border-color: var(--info); box-shadow: 0 6px 16px rgba(45, 91, 150, .3) }

.payment-row { display: flex; justify-content: space-between; align-items: center; padding: 14px 18px }
.payment-row label { font-weight: 600 }
.received-wrap { display: flex; align-items: center; gap: 6px; font-weight: 600 }
.payment-row input { width: 180px; border: 1.5px solid var(--line); border-radius: 12px; padding: 12px; text-align: right; font-size: 16px; font-weight: 700 }
.payment-row input:focus { outline: 2px solid var(--gold); border-color: var(--gold) }
.change-row { background: #f7f2ea; margin: 0 18px; border-radius: 12px }
.change-row strong { font-size: 22px; color: var(--ok-d) }
```

คำอธิบายโค้ด

1) .pay-amount กล่องยอดชำระพื้นสีทองอ่อน ตัวเลขขนาดใหญ่

2) .method.active ปุ่มช่องทางที่เลือกเป็นสีน้ำเงิน

3) .change-row แถวเงินทอนตัวเลขสีเขียว

ตัวอย่างโค้ดที่ 69 public/style.css – หน้ารับชำระเงิน: ปุ่มเงินด่วนและ QR (บรรทัด 434–447)

public/style.css

```
434 .quick-cash { display: flex; gap: 8px; padding: 4px 18px 10px; flex-wrap: wrap }
435 .quick-cash button { flex: 1; min-width: 70px; border: 1.5px solid var(--line); background:
    #fff; border-radius: 10px; padding: 10px; cursor: pointer; font-weight: 600; transition: all
    .12s }
436 .quick-cash button:hover { background: var(--gold-soft); border-color: var(--gold) }
437
438 .error { background: #fdebec; color: #b02020; padding: 12px 16px; margin: 0 18px 10px; border-
    radius: 10px; border: 1px solid #f3c7c9 }
439
440 .qr-card { margin: 10px 18px 6px; border: 1px solid var(--line); border-radius: 16px; padding:
    20px; text-align: center; background: #fff }
441 .qr-head { display: flex; align-items: center; justify-content: center; gap: 10px; font-
    weight: 700; margin-bottom: 14px }
442 .qr-ppay { background: linear-gradient(145deg, #003d7a, #0a66c2); color: #fff; padding: 5px
    12px; border-radius: 8px; font-size: 13px; letter-spacing: .5px }
443 .qr-box { display: inline-block; padding: 14px; border: 2px solid var(--line); border-radius:
    14px; background: #fff }
444 .qr-box img, .qr-box canvas { display: block }
445 .qr-amount { margin-top: 14px; font-size: 16px }
446 .qr-amount b { color: var(--red); font-size: 20px }
447 .qr-note { color: var(--muted); font-size: 13px; margin-top: 6px }
```

คำอธิบายโค้ด

1) .quick-cash จัดปุ่มเงินด่วนให้กว้างเท่ากันด้วย flex: 1

2) .error กล่องข้อความผิดพลาดสีแดง

3) .qr-card / .qr-ppay ตกแต่งกรอบ QR พร้อมป้าย PromptPay สีน้ำเงิน

ตัวอย่างโค้ดที่ 70 public/style.css – หน้าชำระเงินสำเร็จและใบเสร็จ (บรรทัด 449–471)

public/style.css

```
449  /* ==================== SUCCESS / RECEIPT ==================== */
450  .success-panel { max-width: 460px; margin: 30px auto; padding: 28px 24px 24px; text-align:
     center }
451  .checkmark {
452    width: 84px; height: 84px; border-radius: 50%; margin: 0 auto 12px;
453    background: linear-gradient(145deg, var(--ok), var(--ok-d)); color: #fff;
454    font-size: 46px; display: grid; place-items: center;
455    box-shadow: 0 8px 22px rgba(35, 124, 62, .4);
456  }
457  .success-panel > h2 { margin: 0 0 20px; font-size: 22px }
458  .receipt { border: 1px dashed #cbbfb0; border-radius: 14px; padding: 20px; text-align: left;
     background: #fffdf8 }
459  .receipt-top { text-align: center; margin-bottom: 14px }
460  .receipt-logo { font-family: "Noto Serif Thai", serif; font-size: 30px; color: var(--red) }
461  .receipt-name { font-weight: 700; font-size: 18px }
462  .receipt-sub { color: var(--muted); font-size: 12px; letter-spacing: 1px }
463  .receipt-row { display: flex; justify-content: space-between; padding: 6px 0; font-size: 14px
     }
464  .receipt-row.big { font-size: 18px; font-weight: 700 }
465  .receipt-row.big b { color: var(--red) }
466  .receipt-divider { border-top: 1px dashed #cbbfb0; margin: 10px 0 }
467  .receipt-thanks { text-align: center; color: var(--muted); font-size: 13px; margin-top: 6px }
468  .success-status { display: flex; justify-content: space-between; align-items: center; padding:
     20px 2px 6px; font-size: 14px }
469  .success-status b { background: #e6e1da; color: #5f564d; border-radius: 10px; padding: 8px
     26px; font-weight: 700 }
470  .success-panel .actions { display: flex; flex-direction: column; gap: 10px; margin-top: 14px }
471  .success-panel .actions > * { width: 100%; margin: 0 }
```

คำอธิบายโค้ด

1) .checkmark วงกลมสีเขียวพร้อมเครื่องหมายถูก
2) .receipt กรอบเส้นประแบบใบเสร็จ และ .receipt-divider เส้นคั่น
3) .success-status แสดงสถานะโต๊ะหลังปิดบิล
4) .success-panel .actions จัดปุ่มเรียงแนวตั้ง กำหนดให้ปุ่มกว้างเต็มแผงและล้าง margin ของ .full-btn เพื่อให้ปุ่มอยู่กึ่งกลาง

ตัวอย่างโค้ดที่ 71 public/style.css – การรองรับหลายขนาดหน้าจอและการพิมพ์ (บรรทัด 473–510)

public/style.css

```
/* ==================== RESPONSIVE ==================== */
@media (max-width: 1000px) {
  .menu-grid { grid-template-columns: repeat(3, 1fr) }
  .order-board { grid-template-columns: repeat(3, 1fr) }
}
@media (max-width: 760px) {
  .app-shell { width: 96%; margin: 12px auto; border-radius: 20px }
  .topbar { flex-wrap: wrap; padding: 12px 16px; gap: 10px }
  .top-links { width: 100%; justify-content: flex-start; flex-wrap: wrap; gap: 6px }
  .content { padding: 20px 16px 80px }
  .hero-row h1 { font-size: 24px }
  .menu-grid { grid-template-columns: repeat(2, 1fr); gap: 12px }
  .menu-thumb img { height: 120px }
  .cat-nav { top: 0; position: static }
  .product-head { flex-direction: column; text-align: center }
  .product-head img { width: 100%; height: 180px }
  .order-board { grid-template-columns: repeat(2, 1fr) }
  .tables-grid { grid-template-columns: repeat(3, 1fr) }
  .staff-top { flex-direction: column; align-items: flex-start }
  .staff-shell { width: 94% }
}
@media (max-width: 460px) {
  .menu-grid { grid-template-columns: repeat(2, 1fr) }
  .order-board { grid-template-columns: 1fr 1fr }
  .tables-grid { grid-template-columns: repeat(2, 1fr) }
  .method-buttons { grid-template-columns: 1fr }
  .total-box .actions { flex-direction: column }
}

/* ==================== PRINT (receipt) ==================== */
@media print {
  body { background: #fff }
  .no-print, .staff-top, .floating-cart, .toast { display: none !important }
  .staff-shell, .cashier-content { margin: 0; width: auto }
  .panel, .success-panel { box-shadow: none; margin: 0 }
  .checkmark { display: none }
  .receipt { border: none }
}
```

คำอธิบายโค้ด

1) @media (max-width: 1000px / 760px / 460px) ลดจำนวนคอลัมน์ของเมนู การ์ดออเดอร์ และโต๊ะ ให้เหมาะกับแท็บเล็ตแนวตั้งและมือถือ

2) @media print ซ่อนแถบเมนู ปุ่ม และพื้นหลัง เหลือเฉพาะใบเสร็จเมื่อสั่งพิมพ์

ตัวอย่างโค้ดที่ 72 public/style.css – หน้าแรกและหน้าเลือกแผนก (บรรทัด 512–548)

public/style.css

```
/* ==================== ROLE LANDING / STAFF SELECT / PIN ==================== */
.role-shell {
  min-height: 100vh; display: flex; flex-direction: column; align-items: center; justify-content: center;
  padding: 32px 18px; gap: 26px;
  background:
    radial-gradient(1000px 420px at 50% -10%, rgba(227,179,79,.18), transparent 60%),
    linear-gradient(160deg, var(--red) 0%, var(--red-deep) 55%, var(--red-dark) 100%);
  color: #fff;
}
.role-brand { text-align: center }
.brand-mark.big {
  width: 76px; height: 76px; border-radius: 20px; margin: 0 auto 14px; font-size: 42px;
}
.role-brand h1 { margin: 0; font-size: 32px }
.role-brand p { margin: 6px 0 0; color: var(--gold-soft); letter-spacing: 3px; text-transform: uppercase; font-size: 12px }
.role-grid { display: grid; grid-template-columns: repeat(2, minmax(0,260px)); gap: 20px; width: 100%; max-width: 560px }
.role-grid.three { grid-template-columns: repeat(3, minmax(0,230px)); max-width: 760px }
.role-card {
  background: var(--paper); color: var(--ink); border-radius: 22px; padding: 30px 22px;
  text-align: center; box-shadow: var(--shadow); transition: transform .16s, box-shadow .16s;
  display: flex; flex-direction: column; align-items: center; gap: 6px; border-top: 6px solid var(--gold);
}
.role-card:hover { transform: translateY(-5px); box-shadow: 0 20px 40px rgba(20,8,4,.4) }
.role-card.customer { border-top-color: var(--ok) }
.role-card.staff { border-top-color: var(--info) }
.role-card.dept.cook { border-top-color: var(--warn) }
.role-card.dept.serve { border-top-color: var(--ok) }
.role-card.dept.cash { border-top-color: var(--gold) }
.role-emoji { font-size: 48px; line-height: 1 }
.role-card h2 { margin: 8px 0 2px; font-size: 24px }
.role-card p { margin: 0; color: var(--muted); font-size: 14px }
.role-table { margin-top: 10px; background: var(--ok); color: #fff; padding: 6px 16px; border-radius: 999px; font-weight: 700 }
.role-lock { margin-top: 10px; background: #eef1f6; color: var(--info); padding: 6px 14px; border-radius: 999px; font-size: 13px; font-weight: 600 }
.role-hint { color: rgba(255,255,255,.8); font-size: 14px; text-align: center; max-width: 520px }
.role-foot { display: flex; align-items: center; gap: 18px; margin-top: 4px }
.back-link.dark { color: rgba(255,255,255,.85) }
.role-foot .secondary-btn { background: rgba(255,255,255,.14); color: #fff }
```

คำอธิบายโค้ด

1) .role-shell จัดเนื้อหากึ่งกลางจอบนพื้นไล่สีแดง

2) .role-grid แสดงการ์ด 2 ใบ (.three = 3 ใบ) และ .role-card แยกสีขอบบนตามบทบาท

3) .role-table / .role-lock คือป้ายหมายเลขโต๊ะและป้าย "ต้องใช้ PIN"

ตัวอย่างโค้ดที่ 73 public/style.css – หน้ากรอก PIN และส่วนเสริม (บรรทัด 550–576)

public/style.css

```
.pin-card {
  background: var(--paper); color: var(--ink); border-radius: 24px; padding: 34px 28px; text-align: center;
  box-shadow: var(--shadow); max-width: 380px; width: 100%; display: flex; flex-direction: column; align-items: center; gap: 6px;
}
.pin-card h1 { margin: 10px 0 2px; font-size: 24px }
.pin-card > p { margin: 0 0 14px; color: var(--muted); font-size: 14px }
.pin-form { width: 100%; display: flex; flex-direction: column; gap: 14px }
.pin-form input {
  width: 100%; text-align: center; font-size: 26px; letter-spacing: 10px; padding: 14px;
  border: 2px solid var(--line); border-radius: 14px; font-weight: 700;
}
.pin-form input:focus { outline: 2px solid var(--gold); border-color: var(--gold) }
.error.center { text-align: center; width: 100%; margin: 0 }

/* staff nav logout + warn banners */
.staff-top nav { align-items: center }
.nav-logout { margin: 0 }
.nav-logout button { background: rgba(255,255,255,.16); color: #fff; border: 0; border-radius: 999px; padding: 8px 14px; cursor: pointer; font-weight: 600 }
.nav-logout button:hover { background: rgba(255,255,255,.28) }
.banner-warn { background: #fff4e0; color: #9a5b00; border: 1px solid #f3d08a; border-radius: 14px; padding: 14px 18px; margin-bottom: 18px; font-weight: 600 }
.bill-wait { margin: 0 18px 10px; color: #9a5b00; background: #fff4e0; border: 1px solid #f3d08a; border-radius: 10px; padding: 10px 14px; font-size: 14px; text-align: center }
button.blue-btn[disabled] { opacity: .5; cursor: not-allowed; filter: none }

@media (max-width: 620px) {
  .role-grid, .role-grid.three { grid-template-columns: 1fr; max-width: 360px }
  .role-brand h1 { font-size: 26px }
}
```

คำอธิบายโค้ด

1) .pin-card กล่องกรอก PIN ตัวเลขใหญ่ เว้นระยะตัวอักษร (letter-spacing)

2) .banner-warn / .bill-wait แถบเตือนสีส้ม

3) button.blue-btn[disabled] ทำปุ่มรับชำระเงินให้จางลงเมื่อยังกดไม่ได้

ส่วนที่ 2 คู่มือการใช้งานหน้าเว็บ (User Manual)

ส่วนนี้อธิบายวิธีใช้งานหน้าเว็บของแต่ละบทบาทตามลำดับการทำงานจริงของร้าน ตั้งแต่ลูกค้าสั่งอาหาร ครัวทำอาหาร พนักงานเสิร์ฟ จนถึงแคชเชียร์รับชำระเงินและปิดโต๊ะ ภาพหน้าจอแต่ละภาพมีกรอบสีแดงพร้อมหมายเลขกำกับ และมีคำอธิบายหมายเลขอยู่ใต้ภาพ

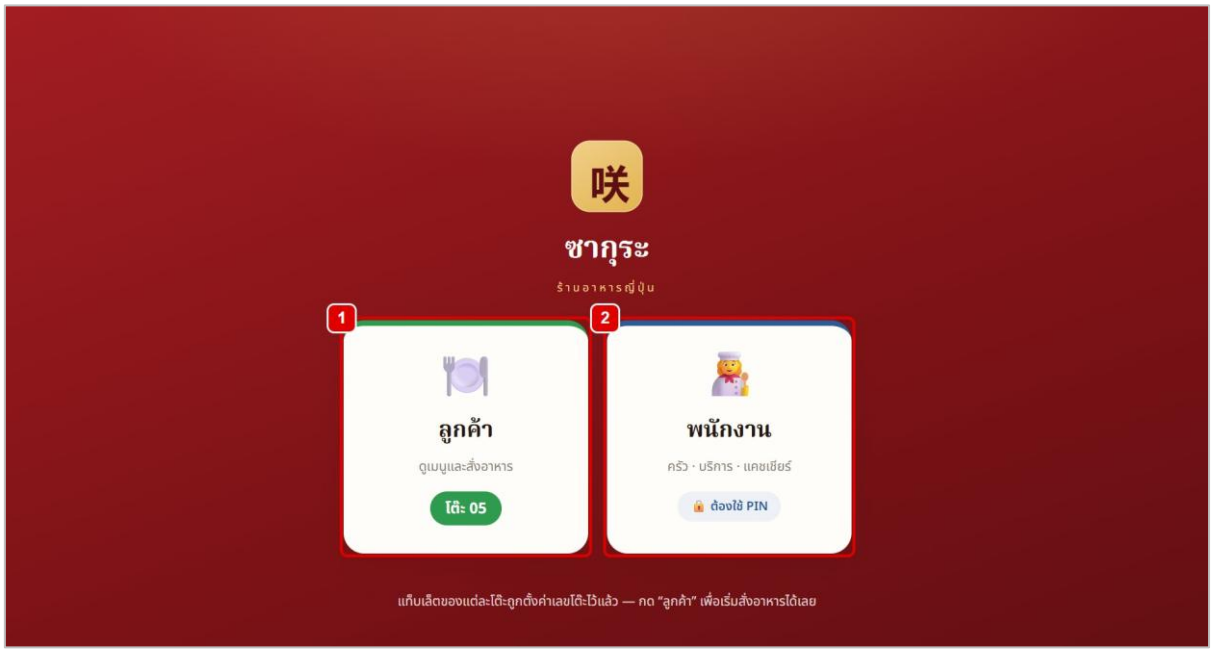
ขั้นตอนโดยรวมของระบบ: ลูกค้าเลือกอาหารและยืนยันการสั่ง ⟶ ครัวทำอาหารแล้วกด "เสร็จสิ้น" (สถานะ กำลังทำ ⟶ พร้อมเสิร์ฟ) ⟶ พนักงานบริการนำอาหารไปเสิร์ฟแล้วกด "เสิร์ฟแล้ว" ⟶ แคชเชียร์รับชำระเงินและปิดบิล (สถานะ ชำระแล้ว และโต๊ะกลับเป็น ว่าง)

2.1 คู่มือการใช้งานสำหรับลูกค้า

ลูกค้าใช้งานผ่านแท็บเล็ตที่วางประจำโต๊ะ ซึ่งพนักงานตั้งค่าหมายเลขโต๊ะไว้ล่วงหน้าแล้ว ลูกค้าไม่ต้องติดตั้งแอปพลิเคชันหรือสมัครสมาชิก สามารถเลือกเมนู ปรับแต่งรายการ สั่งอาหาร และติดตามสถานะอาหารได้ด้วยตนเอง ตัวอย่างในหัวข้อนี้ใช้โต๊ะ 05

2.1.1 หน้าเลือกผู้ใช้งาน

เมื่อเปิดเว็บไซต์ที่แท็บเล็ตประจำโต๊ะ ระบบจะแสดงหน้าแรกพร้อมชื่อร้าน "ซากุระ" และให้เลือกประเภทผู้ใช้งาน การ์ดของลูกค้าแสดงหมายเลขโต๊ะที่แท็บเล็ตถูกตั้งค่าไว้



รูปที่ 1 หน้าเลือกผู้ใช้งาน

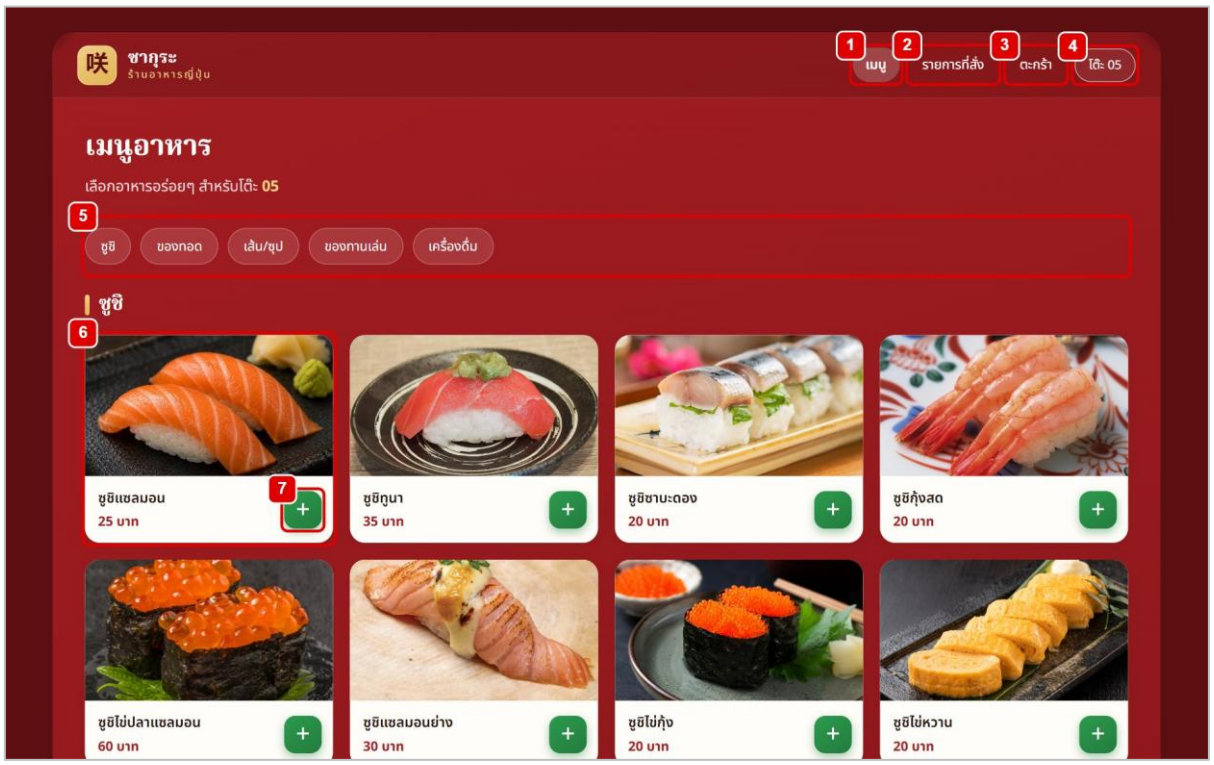
จากรูปที่ 1 หน้าเลือกผู้ใช้งาน มีองค์ประกอบดังนี้

หมายเลข 1 : การ์ด "ลูกค้า" กดเพื่อเริ่มดูเมนูและสั่งอาหาร โดยแสดงหมายเลขโต๊ะ (โต๊ะ 05) ไว้ด้านล่าง

หมายเลข 2 : การ์ด "พนักงาน" สำหรับพนักงานครัว บริการ และแคชเชียร์ ต้องกรอก PIN ก่อนเข้าใช้งาน

2.1.2 หน้าเมนูอาหาร

หน้าเมนูแสดงรายการอาหารทั้งหมด 50 รายการ แบ่งเป็น 5 หมวด แต่ละรายการมีรูปภาพ ชื่อ และราคา ลูกค้าเลื่อนดูหรือกดปุ่มหมวดหมู่เพื่อข้ามไปยังหมวดที่ต้องการได้



รูปที่ 2 หน้าเมนูอาหาร

จากรูปที่ 2 หน้าเมนูอาหาร มีองค์ประกอบดังนี้

หมายเลข 1 : เมนู "เมนู" กลับมาที่หน้ารายการอาหาร

หมายเลข 2 : เมนู "รายการที่สั่ง" ดูออเดอร์ที่ส่งเข้าครัวแล้วและติดตามสถานะ

หมายเลข 3 : เมนู "ตะกร้า" ดูรายการอาหารที่เลือกไว้แต่ยังไม่ได้สั่ง เมื่อมีสินค้าจะแสดงตัวเลขจำนวนชิ้นต่อท้าย

หมายเลข 4 : ป้ายแสดงหมายเลขโต๊ะของแท็บเล็ตเครื่องนี้

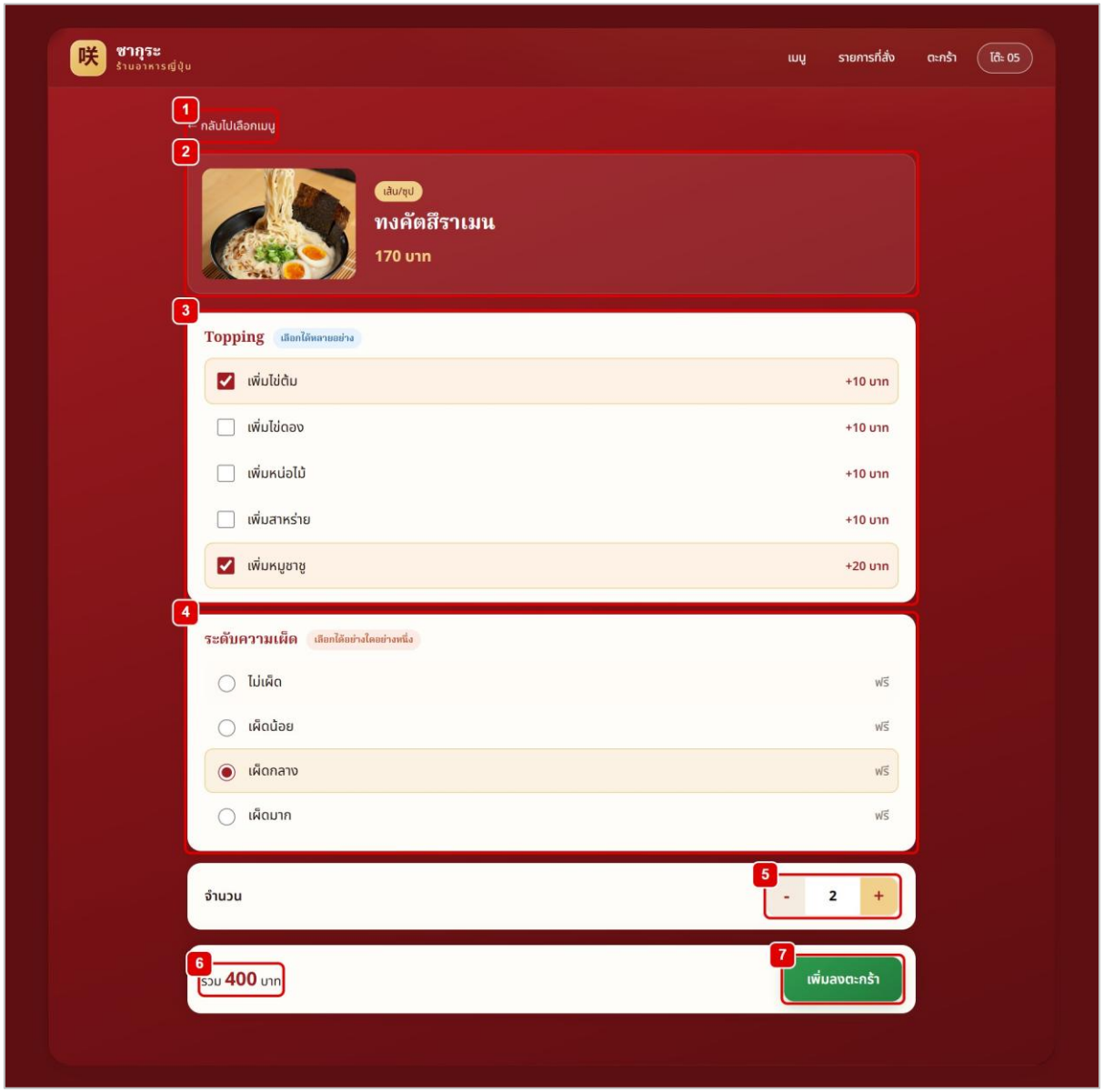
หมายเลข 5 : ปุ่มหมวดหมู่ ซูชิ / ของทอด / เส้น/ซุป / ของทานเล่น / เครื่องดื่ม กดเพื่อเลื่อนไปยังหมวดนั้น

หมายเลข 6 : การ์ดเมนูอาหาร แสดงรูป ชื่อ และราคา

หมายเลข 7 : ปุ่ม + กดเพื่อเลือกอาหารรายการนั้น ระบบจะพาไปหน้าปรับแต่งรายการ

2.1.3 หน้าเลือกตัวเลือกเสริม (Add-on)

เมื่อกดปุ่ม + ระบบจะแสดงตัวเลือกเสริมตามหมวดของอาหาร ตัวอย่างในรูปคือ ทงคัตสึราเมน (หมวด เส้น/ซุป) ซึ่งเลือก Topping ได้หลายอย่างและต้องเลือกระดับความเผ็ด 1 ระดับ ราคารวมด้านล่างจะเปลี่ยนตามตัวเลือกและจำนวนทันที



รูปที่ 3 หน้าเลือกตัวเลือกเสริมของเมนูทงคัตสึราเมน

จากรูปที่ 3 หน้าเลือกตัวเลือกเสริมของเมนูทงคัตสึราเมน มีองค์ประกอบดังนี้

หมายเลข 1 : ลิงก์ "⟵ กลับไปเลือกเมนู" ยกเลิกและกลับไปหน้าเมนู

หมายเลข 2 : ข้อมูลเมนูที่เลือก ได้แก่ รูป หมวดหมู่ ชื่อ และราคา

หมายเลข 3 : กลุ่ม "Topping" ป้ายสีฟ้า "เลือกได้หลายอย่าง" ติ๊กได้มากกว่า 1 ช่อง แต่ละตัวเลือกแสดงราคาที่เพิ่ม (ในรูปเลือก เพิ่มไข่ต้ม +10 บาท และ เพิ่มหมูชาชู +20 บาท)

หมายเลข 4 : กลุ่ม "ระดับความเผ็ด" ป้ายสีส้ม "เลือกได้อย่างใดอย่างหนึ่ง" เลือกได้เพียง 1 ระดับ ไม่มีค่าใช้จ่ายเพิ่ม (แสดงคำว่า ฟรี)

หมายเลข 5 : ปุ่ม – / + ปรับจำนวนที่ต้องการ (1–99)

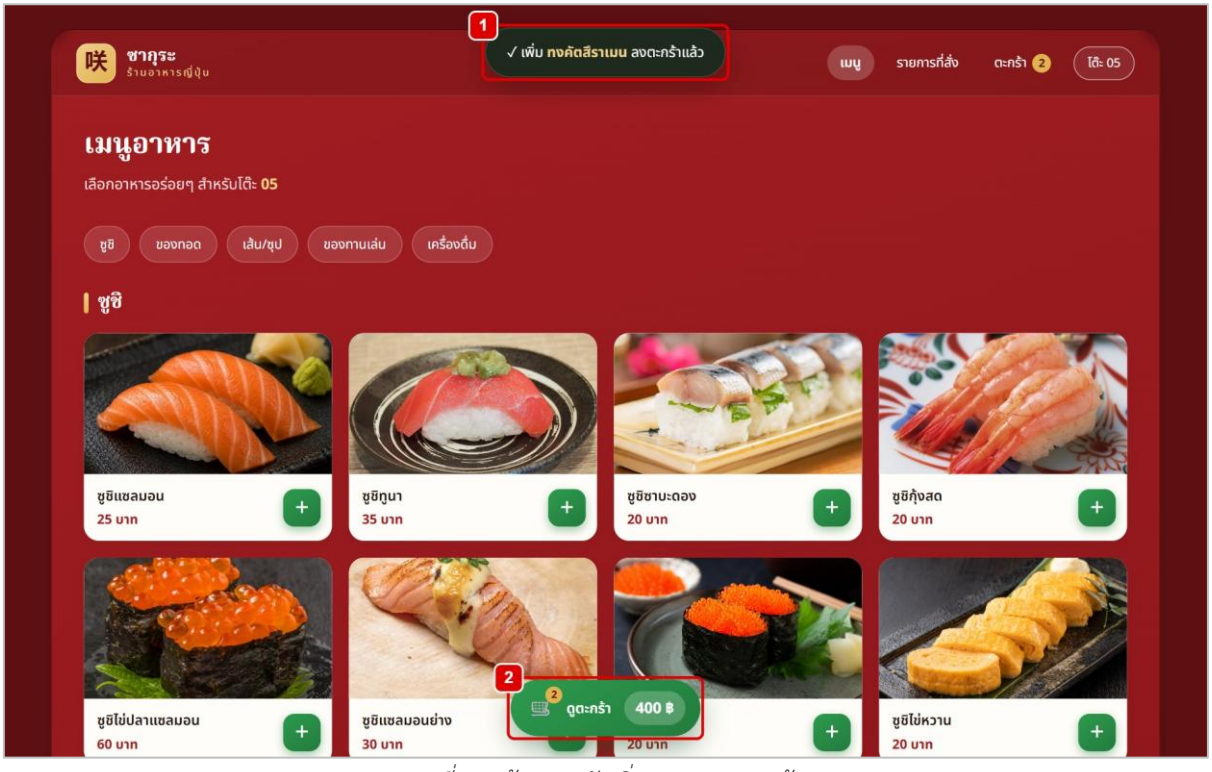
หมายเลข 6 : ราคารวมของรายการนี้ คำนวณจาก (ราคาอาหาร + ตัวเลือกเสริม) × จำนวน ในรูปคือ (170 + 10 + 20) × 2 = 400 บาท

หมายเลข 7 : ปุ่ม "เพิ่มลงตะกร้า" บันทึกรายการลงตะกร้าแล้วกลับไปหน้าเมนู

สำหรับหมวด ซูชิ จะมีตัวเลือก "เครื่องเคียง" (เพิ่มวาซาบิ เพิ่มโชยุ เพิ่มขิงดอง) และหมวด ของทอด / ของทานเล่น จะมีตัวเลือก "ซอส" และ "ผงคลุก"

2.1.4 การแจ้งเตือนเมื่อเพิ่มลงตะกร้า

หลังกดเพิ่มลงตะกร้า ระบบจะกลับมาที่หน้าเมนูเพื่อให้ลูกค้าเลือกอาหารต่อได้ทันที



รูปที่ 4 หน้าเมนูหลังเพิ่มอาหารลงตะกร้า

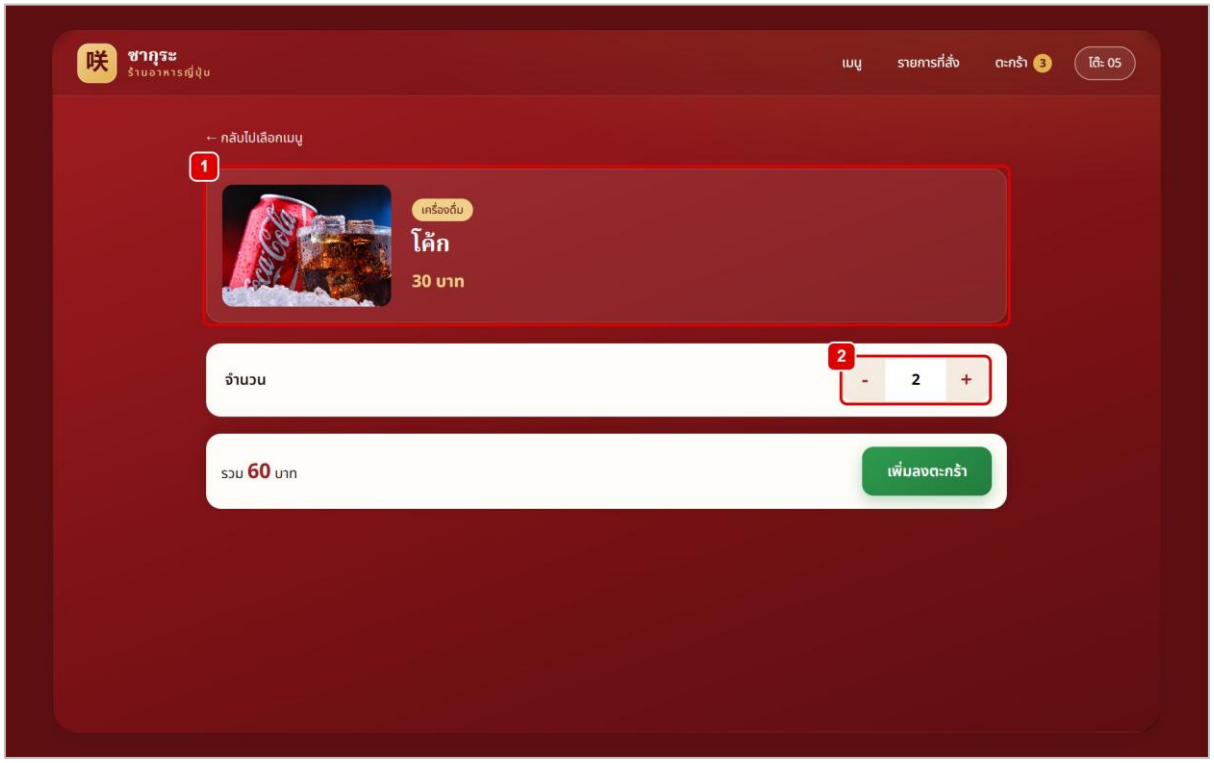
จากรูปที่ 4 หน้าเมนูหลังเพิ่มอาหารลงตะกร้า มีองค์ประกอบดังนี้

หมายเลข 1 : ข้อความแจ้งเตือน "✓ เพิ่ม ทงคัตสึราเมน ลงตะกร้าแล้ว" แสดงด้านบนประมาณ 2–3 วินาทีแล้วหายไปเอง

หมายเลข 2 : ปุ่มตะกร้าลอยด้านล่าง แสดงจำนวนชิ้นและยอดรวมในตะกร้า กดเพื่อไปหน้าตะกร้าได้จากทุกตำแหน่งของหน้า

2.1.5 การเลือกเมนูที่ไม่มีตัวเลือกเสริม

เมนูในหมวดเครื่องดื่มไม่มีตัวเลือกเสริม เมื่อกดปุ่ม + จึงแสดงเพียงข้อมูลเมนูและการเลือกจำนวน



รูปที่ 5 หน้าเลือกจำนวนของเครื่องดื่ม

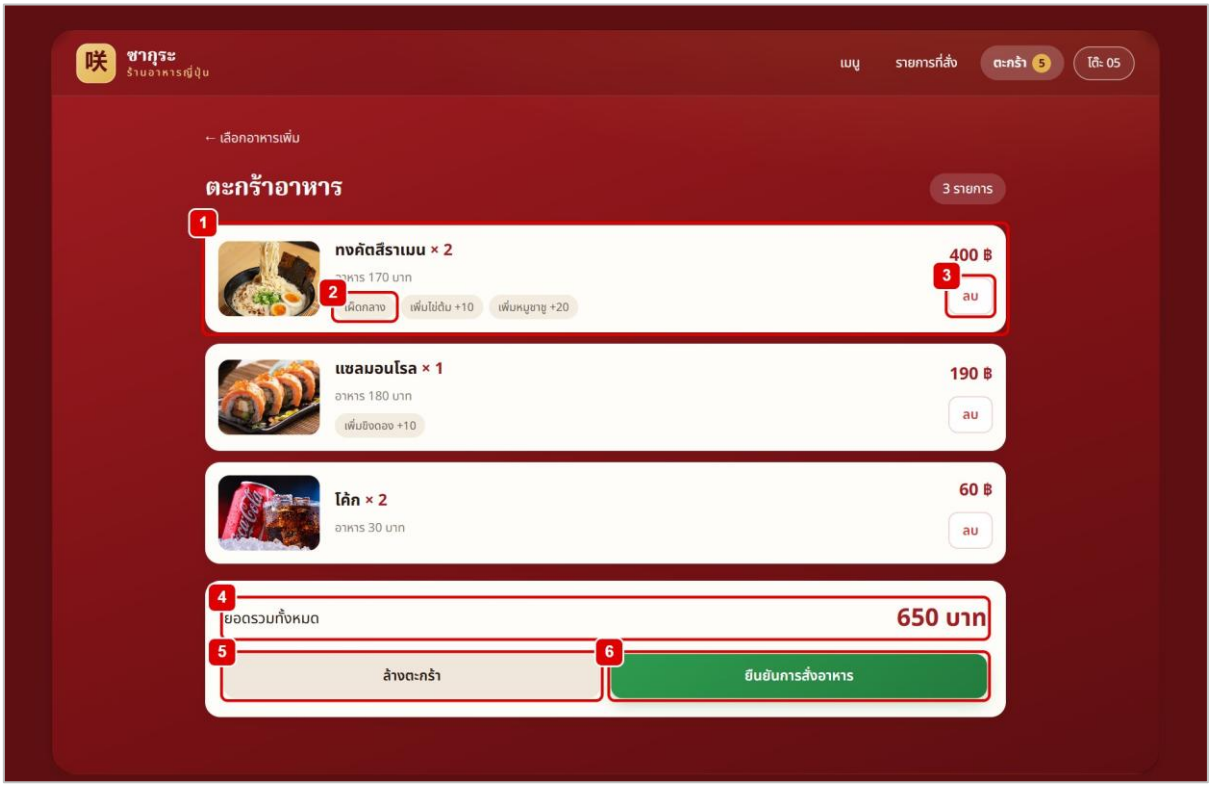
จากรูปที่ 5 หน้าเลือกจำนวนของเครื่องดื่ม มีองค์ประกอบดังนี้

หมายเลข 1 : ข้อมูลเครื่องดื่มที่เลือก (โค้ก 30 บาท)

หมายเลข 2 : เลือกจำนวน 2 แก้ว ราคารวมจะเป็น 60 บาท จากนั้นกด "เพิ่มลงตะกร้า"

2.1.6 หน้าตะกร้าอาหาร

ลูกค้ากดเมนู "ตะกร้า" หรือปุ่มตะกร้าลอยเพื่อตรวจสอบรายการก่อนสั่ง หน้านี้แสดงทุกรายการที่เลือกไว้พร้อมตัวเลือกเสริมและราคา



รูปที่ 6 หน้าตะกร้าอาหาร

จากรูปที่ 6 หน้าตะกร้าอาหาร มีองค์ประกอบดังนี้

หมายเลข 1 : การ์ดรายการอาหาร แสดงรูป ชื่อ จำนวน (× 2) และราคาอาหารต่อหน่วย

หมายเลข 2 : ป้ายตัวเลือกเสริมที่เลือกไว้ พร้อมราคาที่เพิ่ม

หมายเลข 3 : ปุ่ม "ลบ" ลบเฉพาะรายการนั้นออกจากตะกร้า

หมายเลข 4 : ยอดรวมทั้งหมดของตะกร้า (650 บาท)

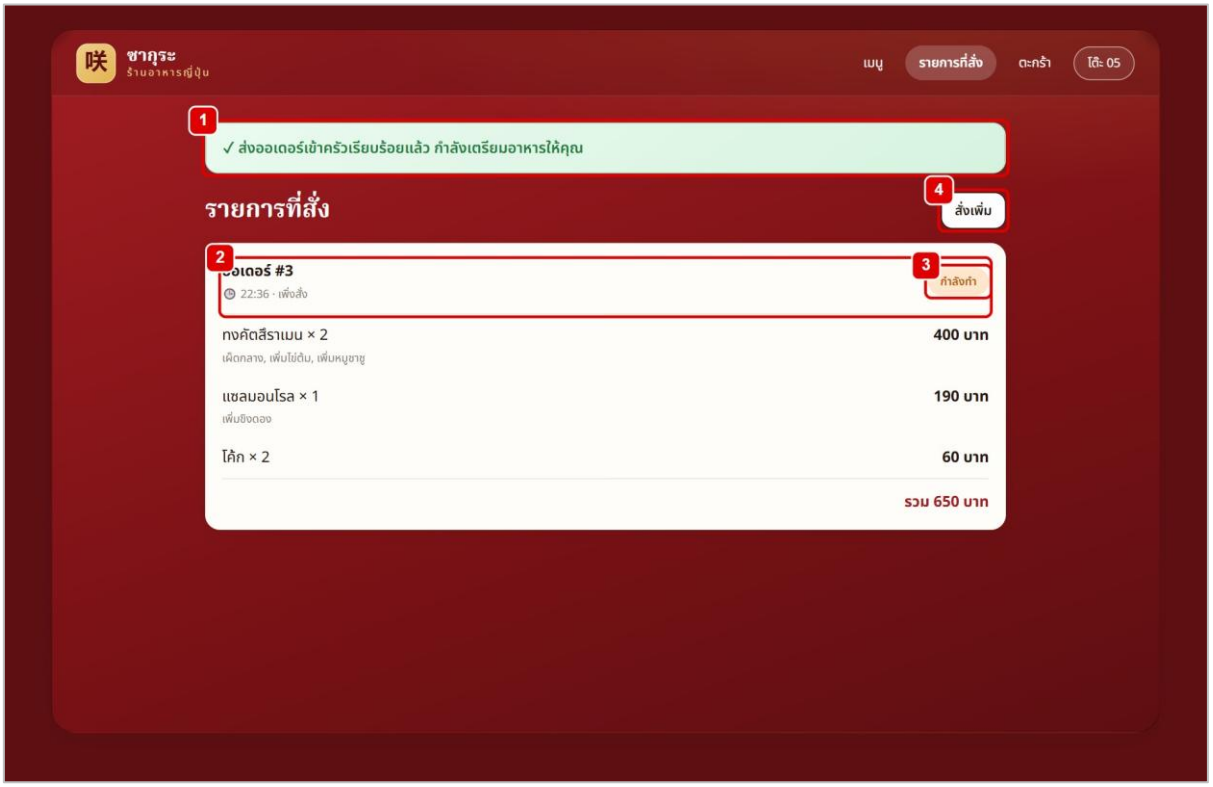
หมายเลข 5 : ปุ่ม "ล้างตะกร้า" ลบทุกรายการในตะกร้า

หมายเลข 6 : ปุ่ม "ยืนยันการสั่งอาหาร" ส่งรายการทั้งหมดเข้าครัว

หากต้องการแก้จำนวนหรือตัวเลือกของรายการที่เลือกไปแล้ว ให้กด "ลบ" รายการนั้นและเลือกใหม่จากหน้าเมนู

2.1.7 หน้ารายการที่สั่ง

หลังกดยืนยัน ระบบจะส่งออเดอร์เข้าครัว ล้างตะกร้า และพาไปหน้ารายการที่สั่ง ลูกค้าสั่งเพิ่มได้หลายครั้ง แต่ละครั้งจะเป็นออเดอร์ใหม่



รูปที่ 7 หน้ารายการที่สั่งหลังยืนยันออเดอร์

จากรูปที่ 7 หน้ารายการที่สั่งหลังยืนยันออเดอร์ มีองค์ประกอบดังนี้

หมายเลข 1 : แถบแจ้ง "✓ ส่งออเดอร์เข้าครัวเรียบร้อยแล้ว กำลังเตรียมอาหารให้คุณ"

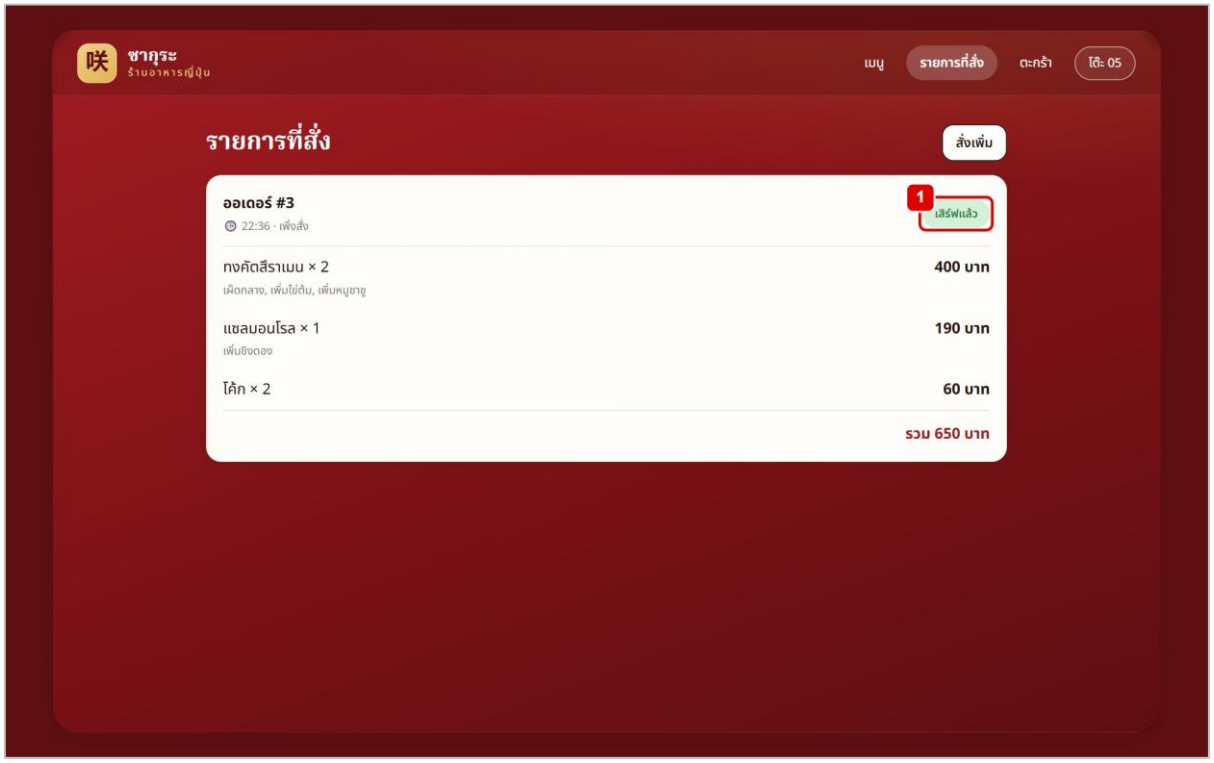
หมายเลข 2 : หมายเลขออเดอร์ (#3) เวลาที่สั่ง และเวลาที่ผ่านไป

หมายเลข 3 : ป้ายสถานะของออเดอร์ เริ่มต้นเป็น "กำลังทำ" (สีส้ม)

หมายเลข 4 : ปุ่ม "สั่งเพิ่ม" กลับไปหน้าเมนูเพื่อสั่งอาหารเพิ่ม

2.1.8 การติดตามสถานะอาหาร

หน้ารายการที่สั่งอัปเดตเองแบบเรียลไทม์โดยไม่ต้องกดรีเฟรช เมื่อพนักงานครัวหรือพนักงานบริการเปลี่ยนสถานะ ป้ายสถานะจะเปลี่ยนตามทันที



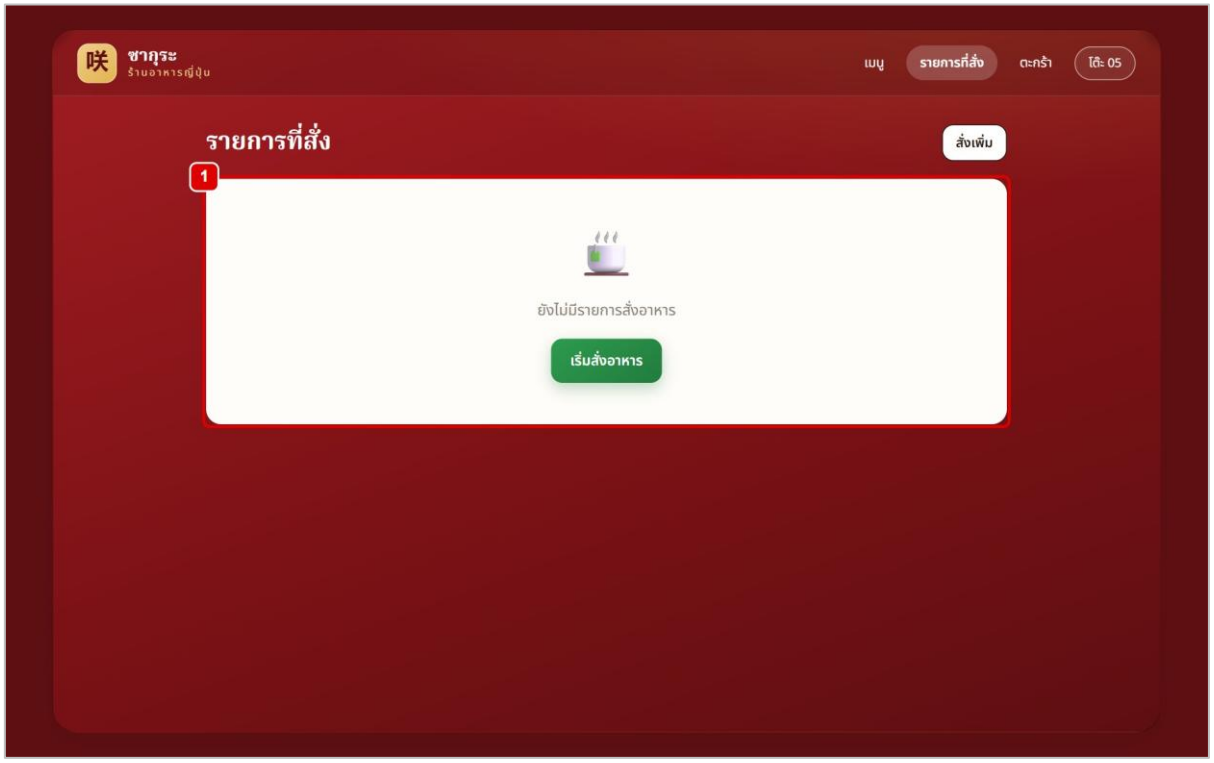
รูปที่ 8 หน้ารายการที่สั่งเมื่ออาหารเสิร์ฟแล้ว

จากรูปที่ 8 หน้ารายการที่สั่งเมื่ออาหารเสิร์ฟแล้ว มีองค์ประกอบดังนี้

หมายเลข 1 : ป้ายสถานะเปลี่ยนเป็น "เสิร์ฟแล้ว" (สีเขียว) สถานะที่เป็นไปได้ ได้แก่ กำลังทำ (สีส้ม) ⟶ พร้อมเสิร์ฟ (สีเหลือง) ⟶ เสิร์ฟแล้ว (สีเขียว)

2.1.9 หลังชำระเงินเรียบร้อย

เมื่อแคชเชียร์ปิดบิลของโต๊ะแล้ว ประวัติออเดอร์จะหายจากแท็บเล็ตทันที เพื่อให้ลูกค้ากลุ่มถัดไปที่นั่งโต๊ะนี้เริ่มสั่งอาหารใหม่ได้ (ข้อมูลยังเก็บอยู่ในฐานข้อมูลสำหรับทำรายงาน)



รูปที่ 9 หน้ารายการที่สั่งหลังปิดบิล

จากรูปที่ 9 หน้ารายการที่สั่งหลังปิดบิล มีองค์ประกอบดังนี้

หมายเลข 1 : ข้อความ "ยังไม่มีรายการสั่งอาหาร" พร้อมปุ่ม "เริ่มสั่งอาหาร"

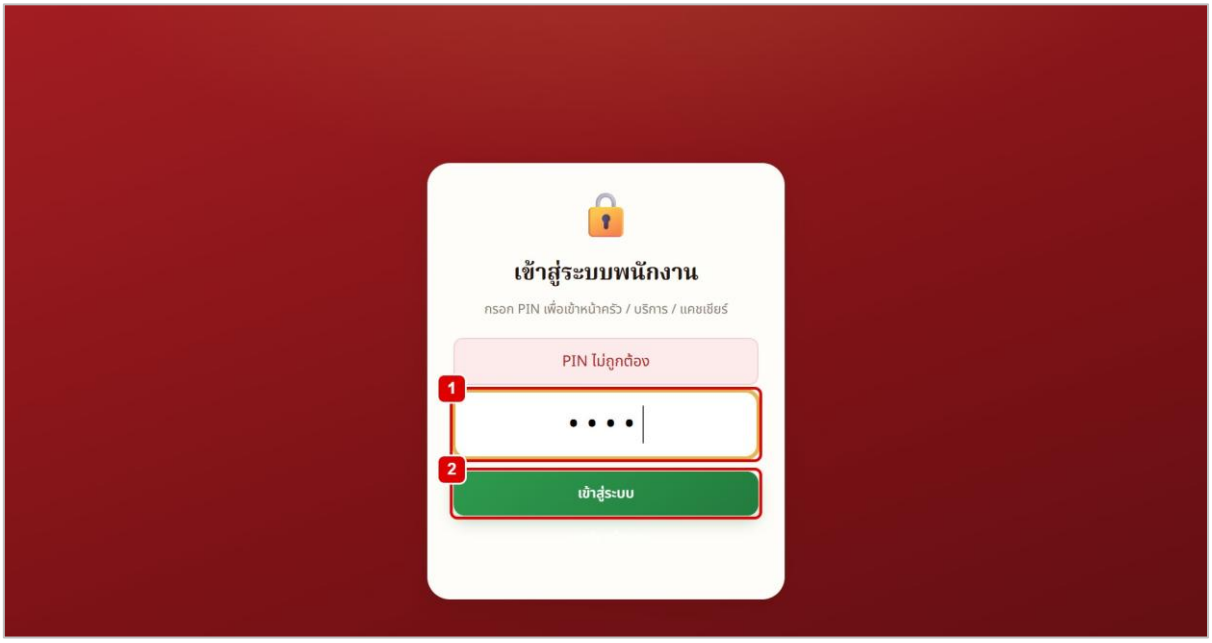
หมายเหตุสำหรับพนักงาน: หน้าของลูกค้าไม่มีปุ่มกลับไปหน้าแรก หากพนักงานต้องการกลับไปหน้าเลือกผู้ใช้งาน ให้กดค้างที่โลโก้ **咲** มุมซ้ายบนประมาณ 1 วินาที

2.2 คู่มือการใช้งานสำหรับพนักงาน (เข้าสู่ระบบ)

พนักงานทุกแผนกเข้าใช้งานจากการ์ด "พนักงาน" ในหน้าแรก โดยต้องกรอก PIN ของร้านก่อน (ค่าเริ่มต้นคือ 1234) เมื่อกรอกถูกต้องครั้งหนึ่งแล้ว ระบบจะจำการเข้าสู่ระบบไว้จนกว่าจะกดออกจากระบบ

2.2.1 หน้าเข้าสู่ระบบพนักงาน

ระบบแสดงกล่องกรอก PIN ตัวเลขจะถูกซ่อนเป็นจุดขณะพิมพ์



รูปที่ 10 หน้าเข้าสู่ระบบพนักงาน

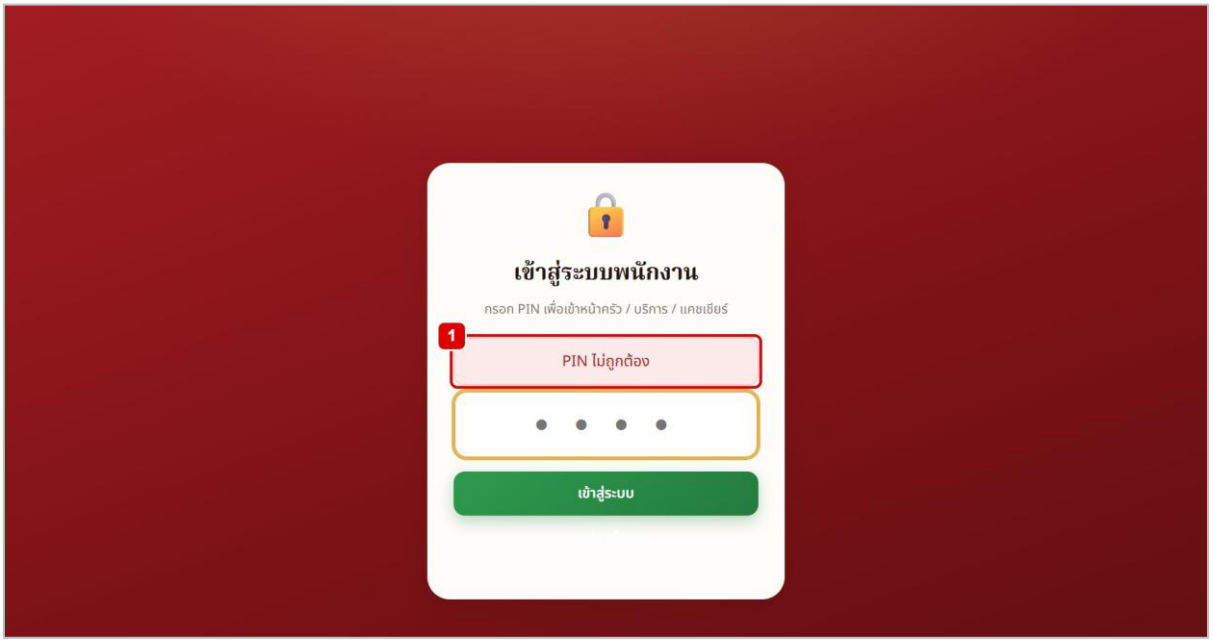
จากรูปที่ 10 หน้าเข้าสู่ระบบพนักงาน มีองค์ประกอบดังนี้

หมายเลข 1 : ช่องกรอก PIN (แท็บเล็ตจะแสดงแป้นตัวเลขให้อัตโนมัติ)

หมายเลข 2 : ปุ่ม "เข้าสู่ระบบ"

2.2.2 กรณีกรอก PIN ไม่ถูกต้อง

หากกรอก PIN ผิด ระบบจะไม่ให้เข้าหน้าพนักงานและแสดงข้อความเตือน ให้กรอกใหม่อีกครั้ง



รูปที่ 11 ข้อความเตือนเมื่อกรอก PIN ไม่ถูกต้อง

จากรูปที่ 11 ข้อความเตือนเมื่อกรอก PIN ไม่ถูกต้อง มีองค์ประกอบดังนี้

หมายเลข 1 : ข้อความ "PIN ไม่ถูกต้อง"

2.2.3 หน้าเลือกแผนก

หลังเข้าสู่ระบบ พนักงานเลือกแผนกที่ต้องการทำงาน



รูปที่ 12 หน้าเลือกแผนกของพนักงาน

จากรูปที่ 12 หน้าเลือกแผนกของพนักงาน มีองค์ประกอบดังนี้

หมายเลข 1 : การ์ด "ครัว" เข้าหน้ารับออเดอร์และทำอาหาร

หมายเลข 2 : การ์ด "บริการ" เข้าหน้าอาหารพร้อมเสิร์ฟ

หมายเลข 3 : การ์ด "แคชเชียร์" เข้าหน้าสถานะโต๊ะและชำระเงิน

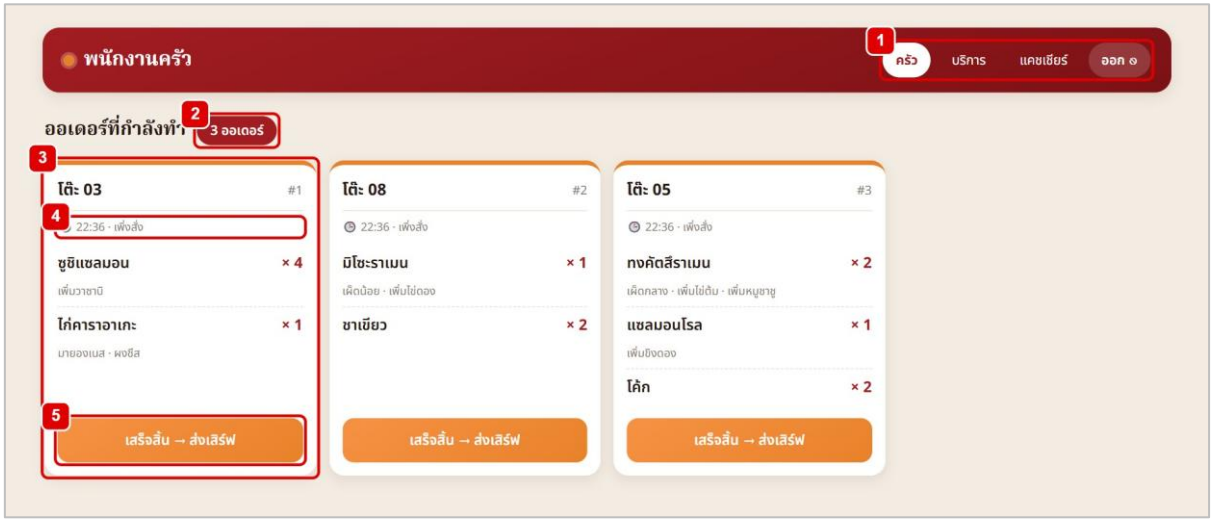
หมายเลข 4 : ปุ่ม "ออกจากระบบพนักงาน" เมื่อเลิกใช้งาน (หน้าครัว บริการ และแคชเชียร์มีปุ่ม "ออก" ที่แถบด้านบนเช่นกัน)

2.3 คู่มือการใช้งานสำหรับพนักงานครัว

หน้าจอครัวแสดงออเดอร์ที่ลูกค้าสั่งเข้ามาและยังทำไม่เสร็จ เรียงตามลำดับเวลาที่สั่ง (มาก่อนทำก่อน) ออเดอร์ใหม่จะขึ้นบนจอเองทันทีที่ลูกค้ากดยืนยัน

2.3.1 หน้าออเดอร์ที่กำลังทำ

ในตัวอย่างมีออเดอร์จาก 3 โต๊ะ ได้แก่ โต๊ะ 03, 08 และ 05



รูปที่ 13 หน้าจอพนักงานครัว

จากรูปที่ 13 หน้าจอพนักงานครัว มีองค์ประกอบดังนี้

หมายเลข 1 : แถบเมนูสลับไปหน้า ครัว / บริการ / แคชเชียร์ และปุ่ม "ออก" จากระบบพนักงาน

หมายเลข 2 : จำนวนออเดอร์ที่กำลังทำทั้งหมด

หมายเลข 3 : การ์ดออเดอร์ แสดงหมายเลขโต๊ะ หมายเลขออเดอร์ ชื่ออาหาร จำนวน และตัวเลือกเสริมใต้ชื่ออาหาร (เช่น เพิ่มวาซาบิ, มายองเนส · ผงชีส)

หมายเลข 4 : เวลาที่สั่งและเวลาที่ผ่านไป หากรอนานเกิน 8 นาทีขอบการ์ดจะเปลี่ยนเป็นสีเหลือง และเกิน 15 นาทีจะเป็นสีแดงพร้อมเวลาสีแดง เพื่อเตือนให้เร่งทำ

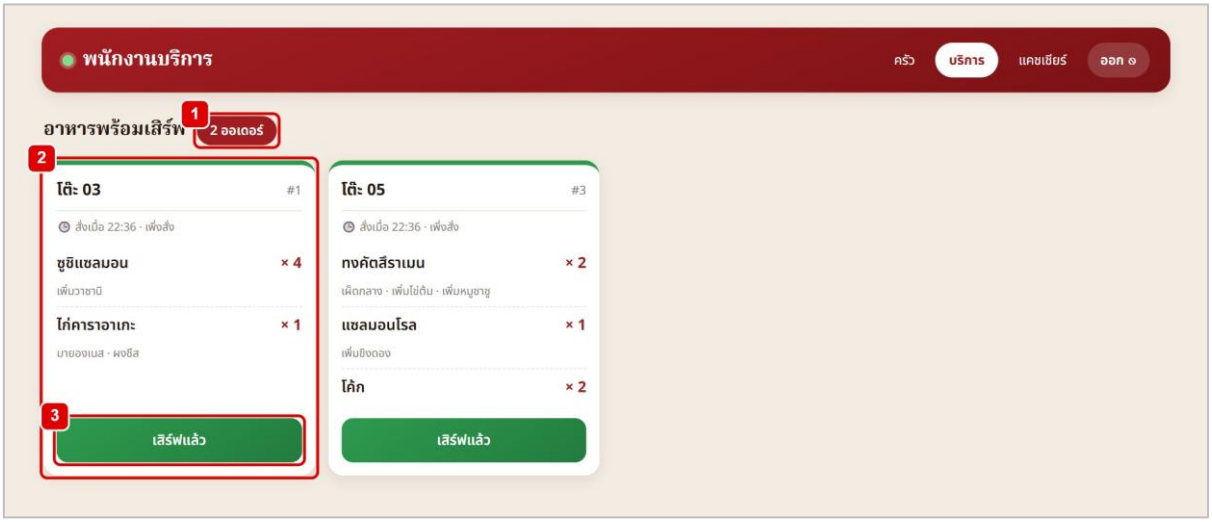
หมายเลข 5 : ปุ่ม "เสร็จสิ้น ⟶ ส่งเสิร์ฟ" กดเมื่อทำอาหารของออเดอร์นั้นเสร็จครบ การ์ดจะย้ายไปหน้าพนักงานบริการ และสถานะบนแท็บเล็ตของลูกค้าเปลี่ยนเป็น "พร้อมเสิร์ฟ"

2.4 คู่มือการใช้งานสำหรับพนักงานบริการ

พนักงานบริการดูรายการอาหารที่ครัวทำเสร็จแล้ว นำไปเสิร์ฟที่โต๊ะ และกดยืนยันว่าเสิร์ฟเรียบร้อย

2.4.1 หน้าอาหารพร้อมเสิร์ฟ

การ์ดในหน้านี้มีขอบบนสีเขียว แสดงเฉพาะออเดอร์ที่ครัวกด "เสร็จสิ้น" แล้ว



รูปที่ 14 หน้าจอพนักงานบริการ

จากรูปที่ 14 หน้าจอพนักงานบริการ มีองค์ประกอบดังนี้

หมายเลข 1 : จำนวนออเดอร์ที่รอเสิร์ฟ

หมายเลข 2 : การ์ดออเดอร์ แสดงหมายเลขโต๊ะ เวลาที่สั่ง รายการอาหาร และตัวเลือกเสริม ใช้ตรวจสอบอาหารก่อนนำไปเสิร์ฟ

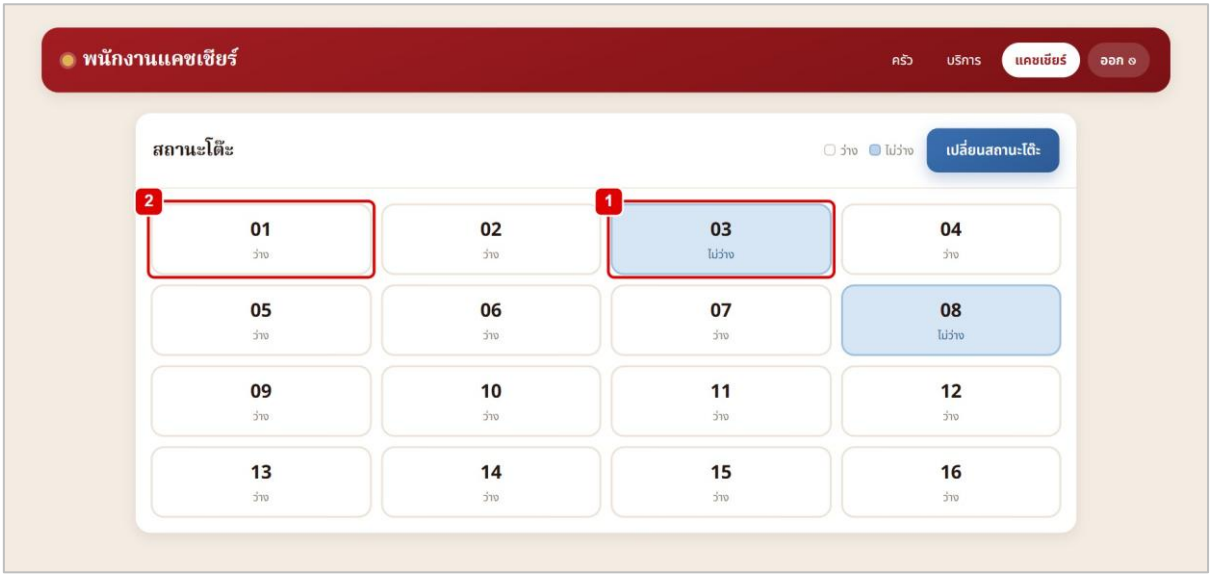
หมายเลข 3 : ปุ่ม "เสิร์ฟแล้ว" กดหลังนำอาหารไปวางที่โต๊ะเรียบร้อย การ์ดจะหายจากหน้านี้ และสถานะบนแท็บเล็ตของลูกค้าเปลี่ยนเป็น "เสิร์ฟแล้ว"

2.5 คู่มือการใช้งานสำหรับพนักงานแคชเชียร์

พนักงานแคชเชียร์ดูภาพรวมสถานะของโต๊ะทั้ง 16 โต๊ะ ตรวจสอบบิล รับชำระเงินด้วยเงินสดหรือ QR พร้อมเพย์ และปรับสถานะโต๊ะ

2.5.1 หน้าสถานะโต๊ะ

เมื่อเข้าหน้าแคชเชียร์ ระบบแสดงตารางโต๊ะ 01–16 โต๊ะที่มีลูกค้าสั่งอาหารจะเปลี่ยนเป็นสถานะไม่ว่างโดยอัตโนมัติ



รูปที่ 15 หน้าสถานะโต๊ะของแคชเชียร์

จากรูปที่ 15 หน้าสถานะโต๊ะของแคชเชียร์ มีองค์ประกอบดังนี้

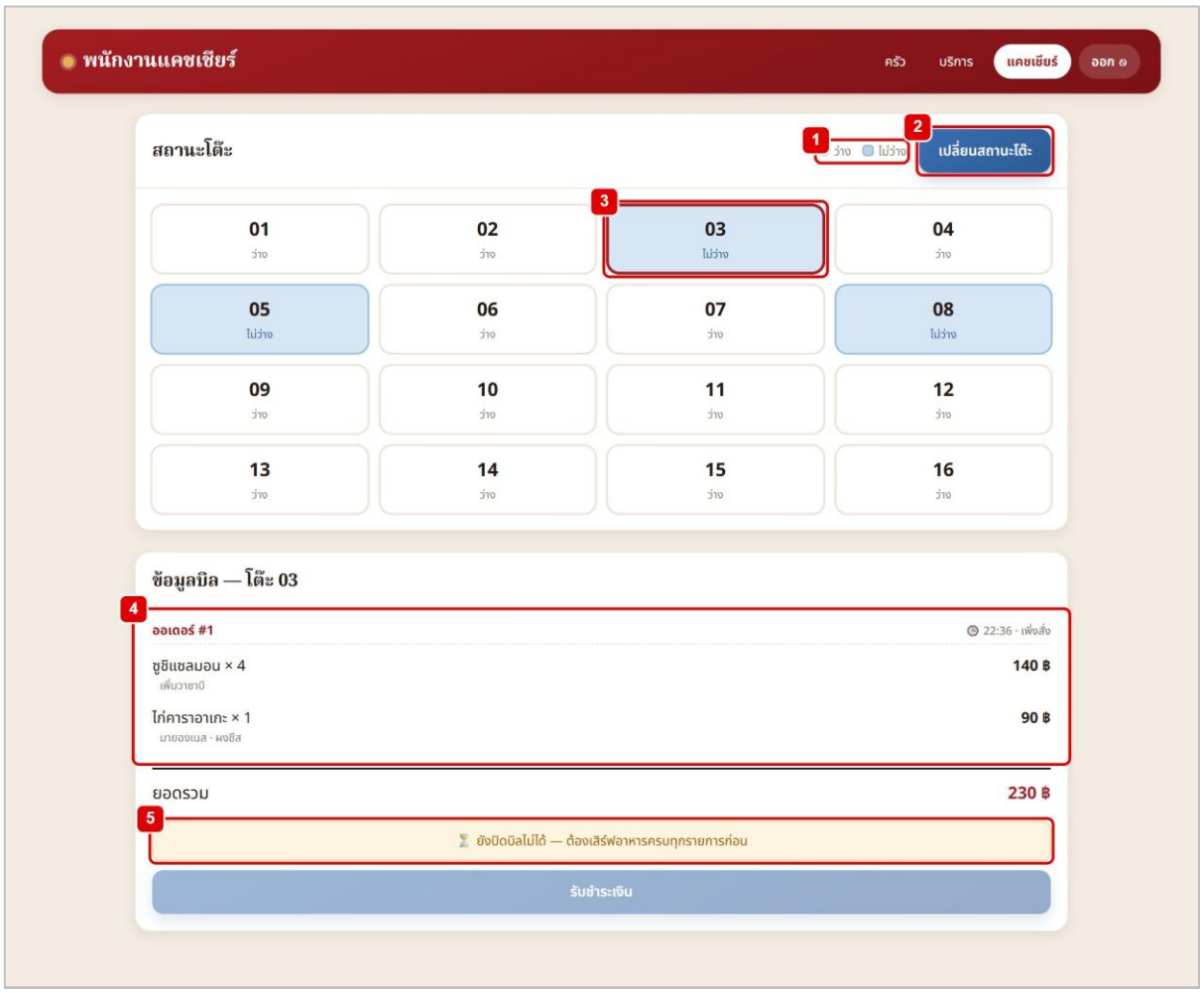
หมายเลข 1 : โต๊ะสีฟ้า สถานะ "ไม่ว่าง" (มีลูกค้า)

หมายเลข 2 : โต๊ะสีขาว สถานะ "ว่าง"

กดที่โต๊ะใดก็ได้เพื่อดูข้อมูลบิลของโต๊ะนั้น

2.5.2 ดูบิลของโต๊ะที่ยังเสิร์ฟไม่ครบ

ระบบจะไม่อนุญาตให้ปิดบิลถ้ายังมีอาหารที่ยังไม่ได้เสิร์ฟ ในตัวอย่างโต๊ะ 03 ครัวทำเสร็จแล้วแต่ยังไม่ได้เสิร์ฟ



รูปที่ 16 ข้อมูลบิลของโต๊ะที่ยังเสิร์ฟอาหารไม่ครบ

จากรูปที่ 16 ข้อมูลบิลของโต๊ะที่ยังเสิร์ฟอาหารไม่ครบ มีองค์ประกอบดังนี้

หมายเลข 1 : คำอธิบายสีของโต๊ะ ว่าง / ไม่ว่าง

หมายเลข 2 : ปุ่ม "เปลี่ยนสถานะโต๊ะ" เข้าสู่โหมดปรับสถานะโต๊ะเอง (อธิบายในหัวข้อการเปลี่ยนสถานะโต๊ะ)

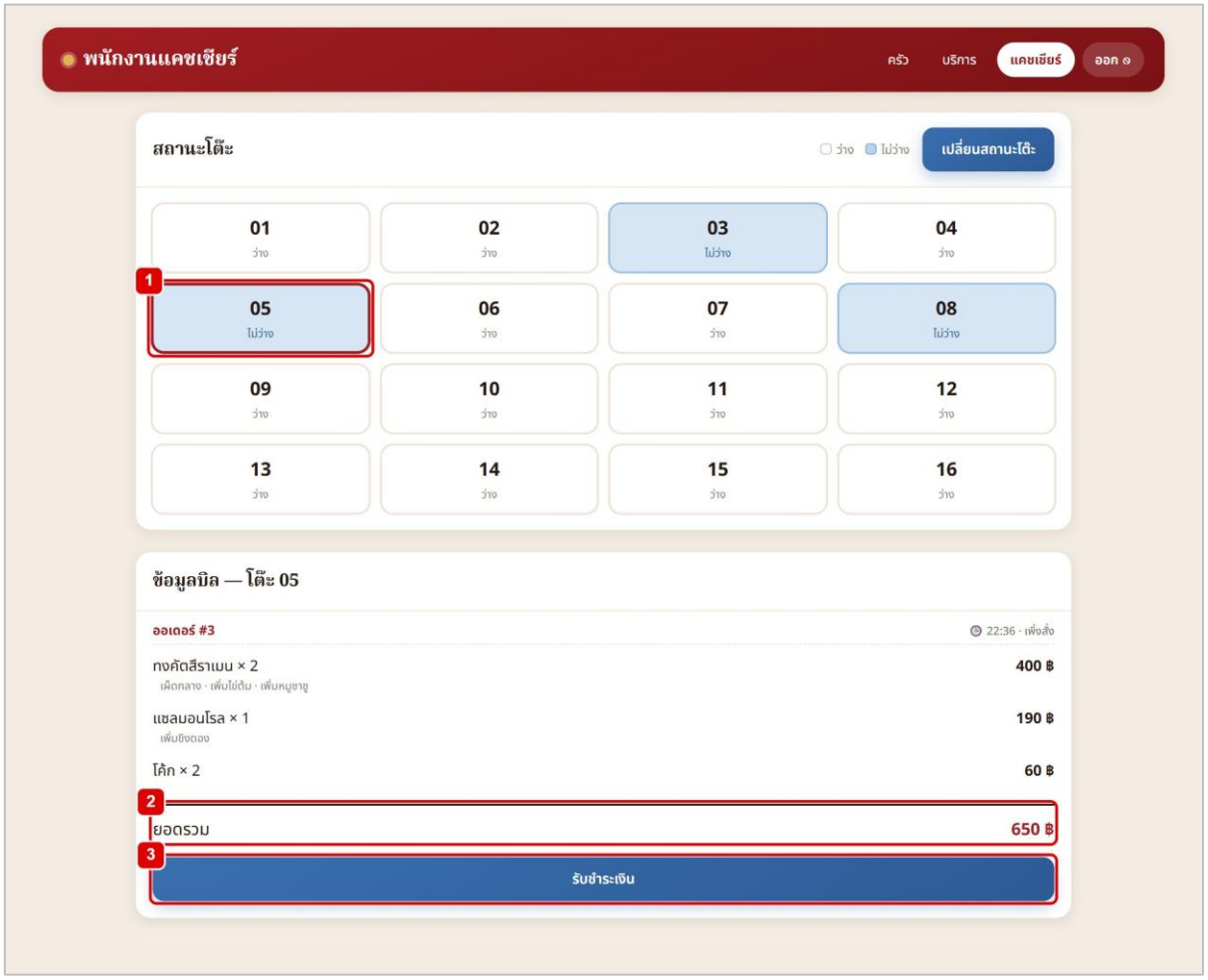
หมายเลข 3 : โต๊ะที่เลือกอยู่ แสดงกรอบสีแดง

หมายเลข 4 : รายการอาหารของโต๊ะ แยกตามออเดอร์ พร้อมเวลา ตัวเลือกเสริม และราคา ด้านล่างแสดงยอดรวม

หมายเลข 5 : ข้อความ " ⏳ ยังปิดบิลไม่ได้ — ต้องเสิร์ฟอาหารครบทุกรายการก่อน" และปุ่มรับชำระเงินจะจางและกดไม่ได้

2.5.3 ดูบิลของโต๊ะที่พร้อมชำระเงิน

เมื่อทุกออเดอร์ของโต๊ะมีสถานะ "เสิร์ฟแล้ว" ปุ่มรับชำระเงินจะกดได้



รูปที่ 17 ข้อมูลบิลของโต๊ะที่พร้อมชำระเงิน

จากรูปที่ 17 ข้อมูลบิลของโต๊ะที่พร้อมชำระเงิน มีองค์ประกอบดังนี้

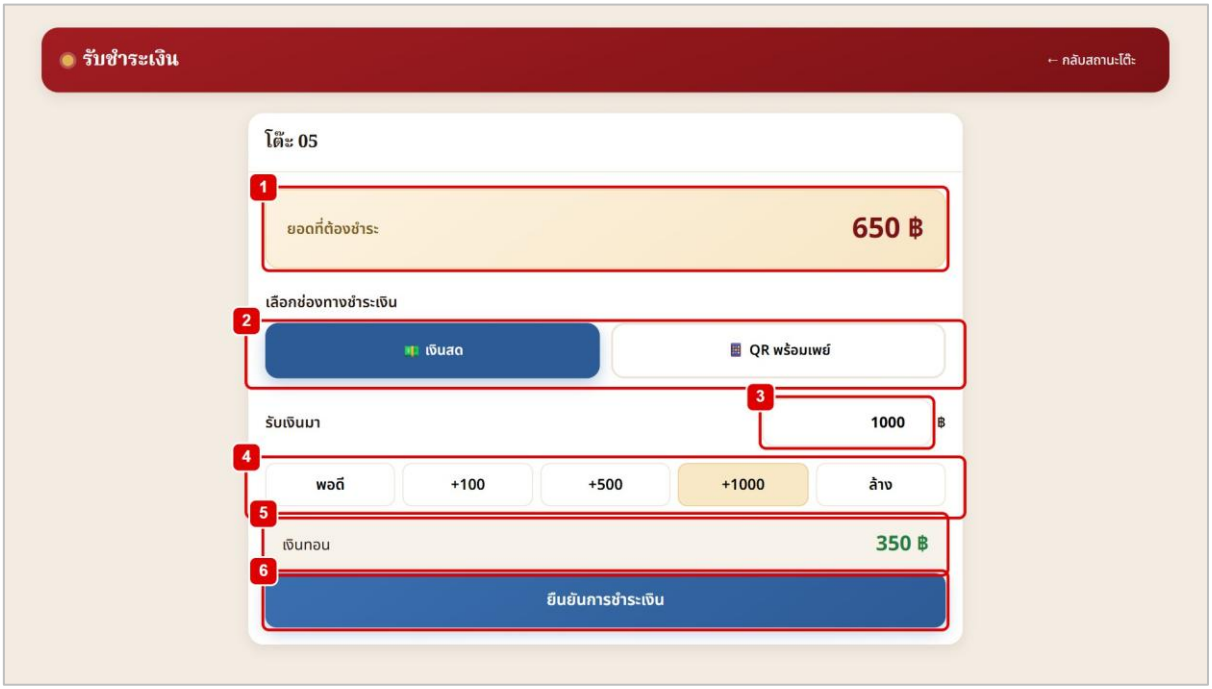
หมายเลข 1 : โต๊ะ 05 ที่เลือก

หมายเลข 2 : ยอดรวมของบิล 650 บาท

หมายเลข 3 : ปุ่ม "รับชำระเงิน" ไปหน้ารับชำระเงินของโต๊ะ

2.5.4 รับชำระเงินด้วยเงินสด

หน้ารับชำระเงินเลือกช่องทาง "เงินสด" ไว้ให้เป็นค่าเริ่มต้น ช่องรับเงินจะใส่ยอดพอดีไว้ให้ก่อน



รูปที่ 18 หน้ารับชำระเงินด้วยเงินสด

จากรูปที่ 18 หน้ารับชำระเงินด้วยเงินสด มีองค์ประกอบดังนี้

หมายเลข 1 : ยอดที่ต้องชำระของโต๊ะ

หมายเลข 2 : ปุ่มเลือกช่องทางชำระเงิน " เงินสด" หรือ " QR พร้อมเพย์" (ปุ่มที่เลือกอยู่เป็นสีน้ำเงิน)

หมายเลข 3 : ช่อง "รับเงินมา" กรอกจำนวนเงินที่ได้รับจากลูกค้า

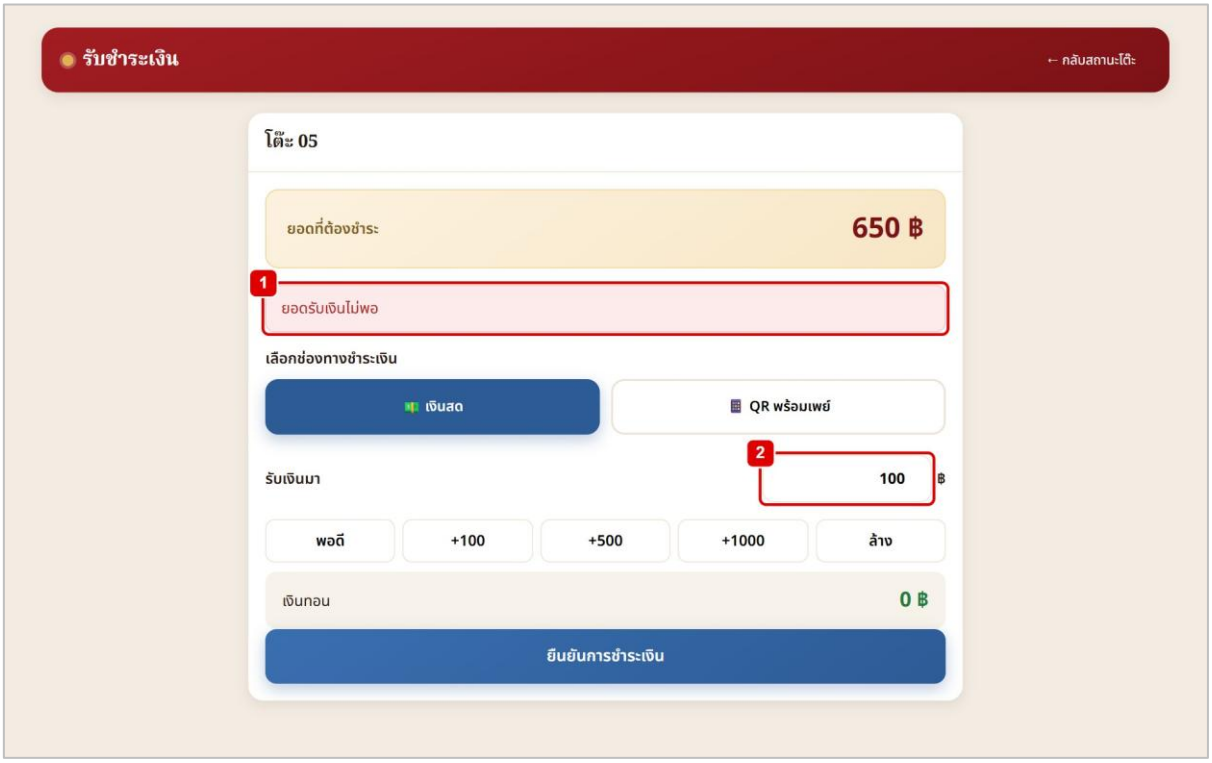
หมายเลข 4 : ปุ่มเงินด่วน: "พอดี" ใส่ยอดพอดี, "+100 / +500 / +1000" บวกเพิ่มจากยอดเดิม, "ล้าง" ตั้งเป็น 0 (ในรูปกด ล้าง แล้ว +1000)

หมายเลข 5 : เงินทอน คำนวณให้อัตโนมัติ (1,000 − 650 = 350 บาท)

หมายเลข 6 : ปุ่ม "ยืนยันการชำระเงิน" บันทึกการชำระเงินและปิดบิล

2.5.5 กรณีรับเงินไม่พอ

ถ้ากรอกจำนวนเงินที่รับน้อยกว่ายอดที่ต้องชำระแล้วกดยืนยัน ระบบจะไม่ปิดบิลและแสดงข้อความเตือน



รูปที่ 19 ข้อความเตือนเมื่อรับเงินไม่พอ

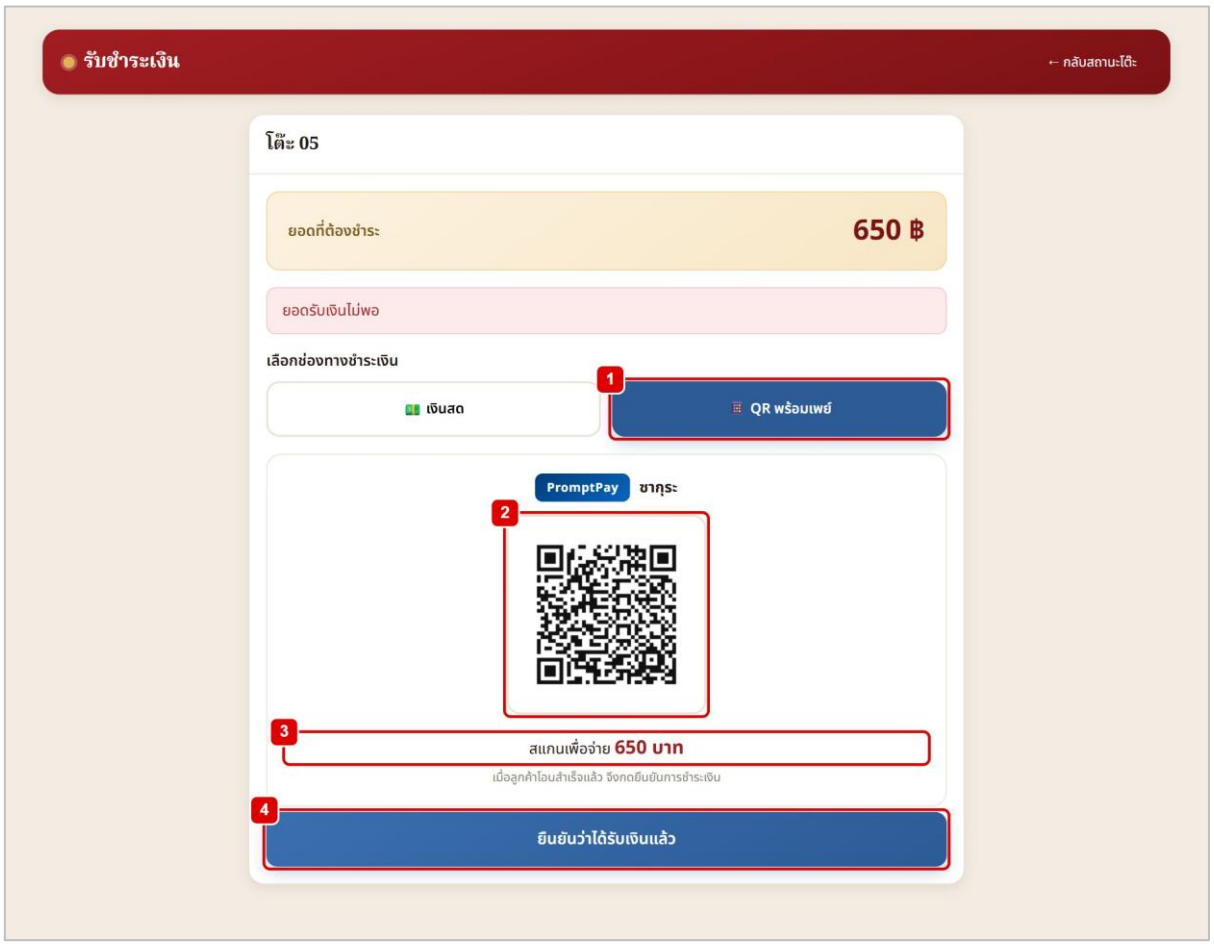
จากรูปที่ 19 ข้อความเตือนเมื่อรับเงินไม่พอ มีองค์ประกอบดังนี้

หมายเลข 1 : ข้อความ "ยอดรับเงินไม่พอ"

หมายเลข 2 : ช่องรับเงินแสดงยอดที่กรอกไว้ (100 บาท) ให้แก้ไขเป็นจำนวนที่ถูกต้องแล้วกดยืนยันอีกครั้ง

2.5.6 รับชำระเงินด้วย QR พร้อมเพย์

เมื่อกดเลือก "QR พร้อมเพย์" ระบบจะสร้าง QR Code ที่ระบุยอดเงินของบิลไว้แล้ว ลูกค้าสแกนด้วยแอปธนาคารได้ทันทีโดยไม่ต้องพิมพ์ยอดเอง



รูปที่ 20 หน้ารับชำระเงินด้วย QR พร้อมเพย์

จากรูปที่ 20 หน้ารับชำระเงินด้วย QR พร้อมเพย์ มีองค์ประกอบดังนี้

หมายเลข 1 : ปุ่ม "QR พร้อมเพย์" ที่เลือกอยู่

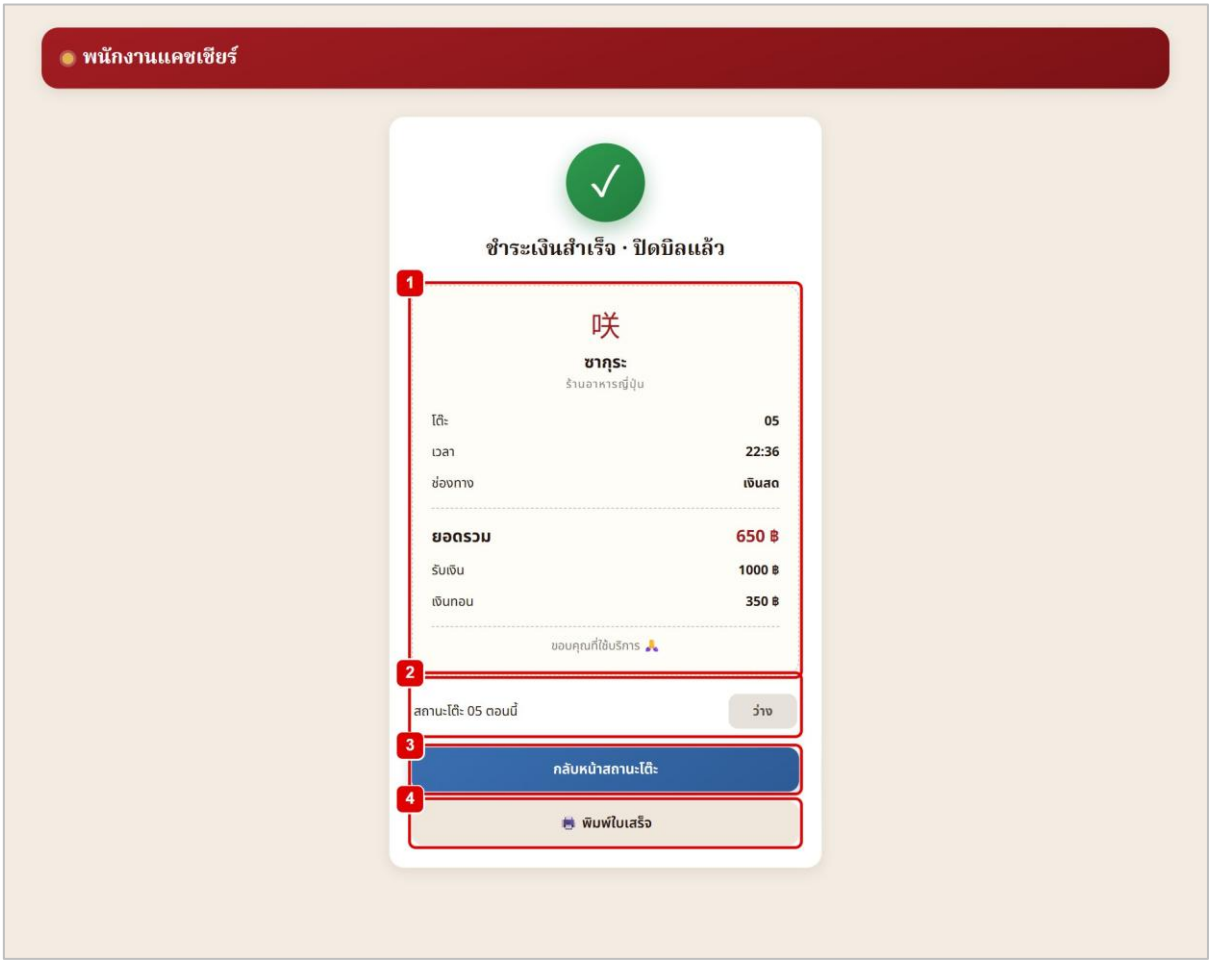
หมายเลข 2 : QR Code พร้อมเพย์ของร้าน ให้ลูกค้าสแกนเพื่อโอนเงิน

หมายเลข 3 : ยอดเงินที่ต้องสแกนจ่าย

หมายเลข 4 : ปุ่ม "ยืนยันว่าได้รับเงินแล้ว" กดหลังตรวจสอบว่าเงินเข้าบัญชีร้านแล้วเท่านั้น

2.5.7 ชำระเงินสำเร็จ

หลังยืนยันการชำระเงิน ระบบบันทึกประวัติการชำระ ปิดทุกออเดอร์ของโต๊ะ และคืนสถานะโต๊ะเป็นว่าง



รูปที่ 21 หน้าชำระเงินสำเร็จและใบเสร็จ

จากรูปที่ 21 หน้าชำระเงินสำเร็จและใบเสร็จ มีองค์ประกอบดังนี้

หมายเลข 1 : ใบเสร็จ แสดงโต๊ะ เวลา ช่องทาง ยอดรวม เงินที่รับ และเงินทอน (กรณีเงินสด)

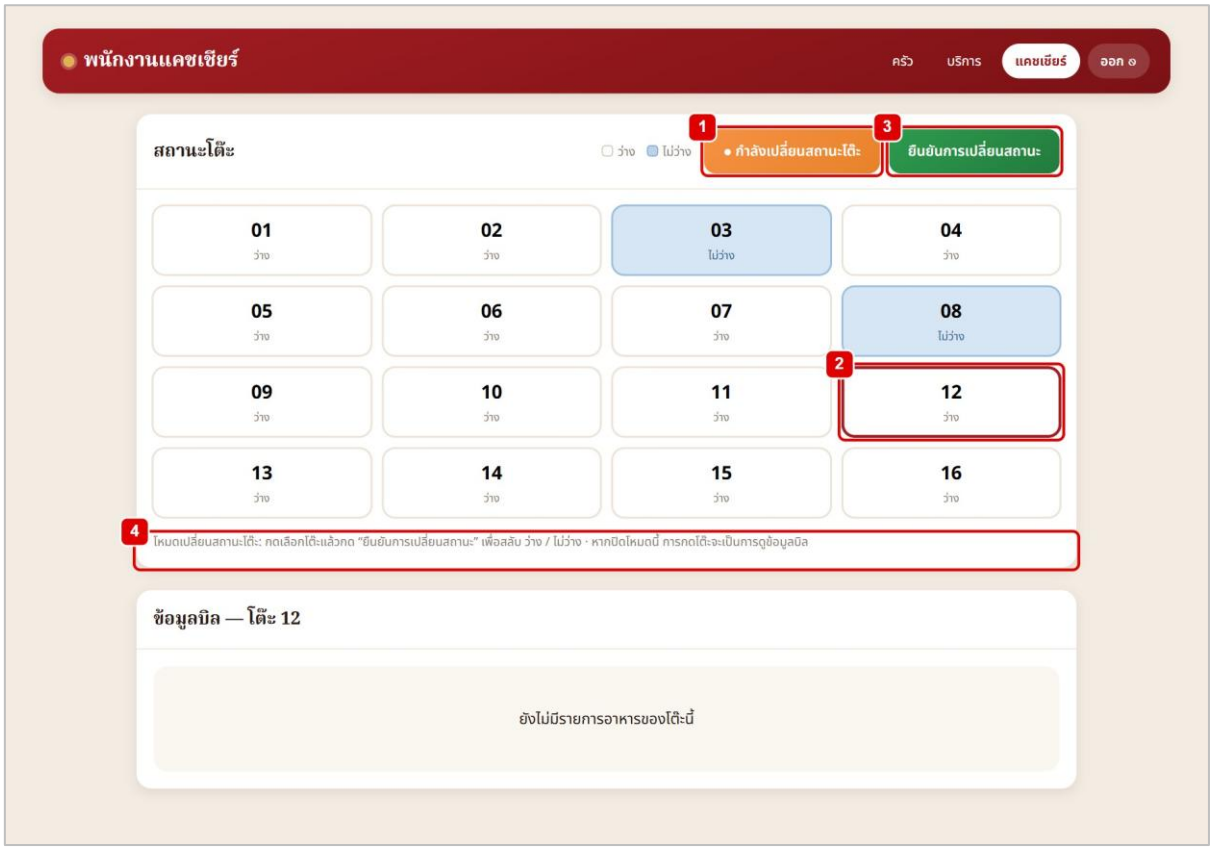
หมายเลข 2 : สถานะโต๊ะหลังปิดบิล เปลี่ยนเป็น "ว่าง"

หมายเลข 3 : ปุ่ม "กลับหน้าสถานะโต๊ะ"

หมายเลข 4 : ปุ่ม " พิมพ์ใบเสร็จ" สั่งพิมพ์เฉพาะส่วนใบเสร็จ

2.5.8 การเปลี่ยนสถานะโต๊ะ

ใช้ในกรณีที่ต้องการปรับสถานะโต๊ะเอง เช่น ลูกค้าเข้ามานั่งแล้วแต่ยังไม่ได้สั่งอาหาร หรือต้องการปลดโต๊ะที่ค้างสถานะ ขั้นตอนคือ กดปุ่ม "เปลี่ยนสถานะโต๊ะ" ⟶ กดเลือกโต๊ะ ⟶ กด "ยืนยันการเปลี่ยนสถานะ"



รูปที่ 22 โหมดเปลี่ยนสถานะโต๊ะ

จากรูปที่ 22 โหมดเปลี่ยนสถานะโต๊ะ มีองค์ประกอบดังนี้

หมายเลข 1 : ปุ่มเปลี่ยนเป็นสีส้ม "● กำลังเปลี่ยนสถานะโต๊ะ" แสดงว่าอยู่ในโหมดนี้

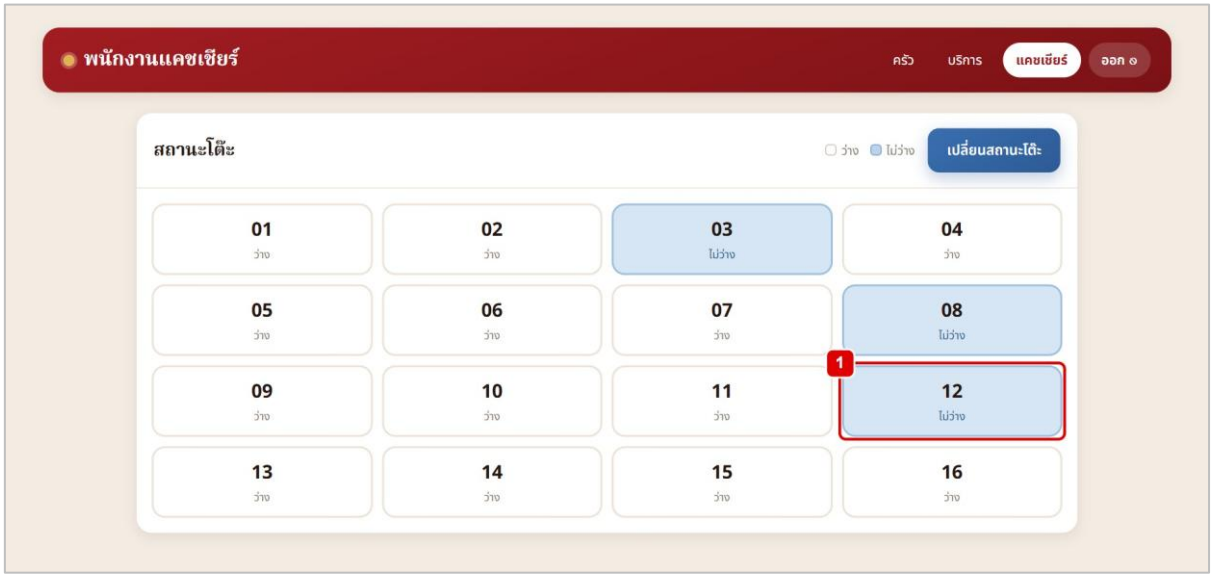
หมายเลข 2 : โต๊ะที่เลือก (โต๊ะ 12) แสดงกรอบสีแดง การกดเลือกยังไม่เปลี่ยนสถานะทันที

หมายเลข 3 : ปุ่ม "ยืนยันการเปลี่ยนสถานะ" สลับสถานะของโต๊ะที่เลือกระหว่าง ว่าง ⟷ ไม่ว่าง

หมายเลข 4 : คำแนะนำการใช้งานโหมดเปลี่ยนสถานะ

2.5.9 หลังเปลี่ยนสถานะโต๊ะ

เมื่อกดยืนยัน ระบบจะกลับสู่หน้าสถานะโต๊ะปกติ



รูปที่ 23 หน้าสถานะโต๊ะหลังเปลี่ยนสถานะ

จากรูปที่ 23 หน้าสถานะโต๊ะหลังเปลี่ยนสถานะ มีองค์ประกอบดังนี้

หมายเลข 1 : โต๊ะ 12 เปลี่ยนจาก "ว่าง" เป็น "ไม่ว่าง" (สีฟ้า) การเปลี่ยนสถานะด้วยวิธีนี้ไม่มีผลกับออเดอร์ของโต๊ะ